

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ТАРАСА ШЕВЧЕНКА

ФАКУЛЬТЕТ РАДІОФІЗИКИ, ЕЛЕКТРОНІКИ ТА КОМП'ЮТЕРНИХ СИСТЕМ

Кафедра Комп'ютерної інженерії

«На правах рукопису»

Робота допущена до захисту в ЕК
рішенням кафедри комп'ютерної інженерії
від _____ 2026 року, протокол № _____.
В.О. Завідувач кафедри кандидат фіз.-мат. наук, доцент
Юрій БОЙКО

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА

на тему:

«Система адаптивної фільтрації та автоматизованого відновлення зображень для
CMOS-сенсорів наукового призначення»

Виконав:

студент 4-го курсу
денної форми здобуття освіти
спеціальності 123 Комп'ютерна інженерія
ОПП Інженерія комп'ютерних систем і мереж
Ібатуллін Ільдар Шамільович _____

Науковий керівник:

кандидат фізико-математичних наук, доцент
Барабанов Олександр Валерійович _____

Рецензент:

кандидат фізико-математичних наук, доцент
Шкавро Анатолій Григорович _____

Засвідчую, що у цій бакалаврській роботі
немає запозичень з праць інших авторів без
відповідних посилань

Студент _____ Ільдар ІБАТУЛЛІН

Київ – 2026

РЕФЕРАТ

Ібатуллін І. Ш. Система адаптивної фільтрації та автоматизованого відновлення зображень для CMOS-сенсорів наукового призначення. – Кваліфікаційна робота бакалавра. Київський національний університет імені Тараса Шевченка, факультет радіофізики, електроніки та комп'ютерних систем, кафедра комп'ютерної інженерії. – Київ, 2026.

Ключові слова: CMOS-сенсор, функція розсіювання точки, PSF, деконволюція, алгоритм Річардсона-Люсі, адаптивна фільтрація, метрика Tenengrad, Ringing Score, флуоресцентна мікроскопія, дистанційне зондування.

Мета роботи – розробка системи адаптивної фільтрації та автоматизованого відновлення зображень для CMOS-сенсорів наукового призначення, що забезпечує автоматичну ідентифікацію моделі функції розсіювання точки (PSF), оптимізацію її параметрів та компенсацію спотворень методами деконволюції без ручного налаштування.

У роботі проведено аналіз архітектури CMOS-сенсорів типу Active Pixel Sensor, фізичних механізмів виникнення шумів (темновий струм, FPN, PRNU, shot noise) та явища перехресних завад (crosstalk). Розроблено математичні моделі шести аналітичних PSF: гаусової, Коші (Лоренца), диску Ейрі, Моффата, розмиття руху та розсіювання. Обґрунтовано чотириетапний алгоритм адаптивної деконволюції: автоматична ідентифікація моделі PSF на основі статистичного аналізу ексцесу градієнтів та коефіцієнта напрямковості; оптимізація масштабного параметра γ^* методом золотого перерізу з мінімізацією розширеної цільової функції на основі метрики Tenengrad; ітеративна деконволюція алгоритмом Річардсона-Люсі; адаптивне двомасштабне нерізде маскування.

Програмний модуль реалізовано мовою Python з використанням бібліотек NumPy, SciPy, OpenCV, scikit-image та Astropy. Модуль підтримує формати TIFF (8 та 16 біт), PNG, JPEG та FITS і оснащений графічним інтерфейсом на базі Tkinter/matplotlib.

Тестування проведено на одинадцяти реальних зображеннях чотирьох доменних класів: планетарна зйомка, глибокий космос, дистанційне зондування та флуоресцентна мікроскопія. У всіх тестових випадках зафіксовано позитивний приріст метрики Tenengrad у діапазоні 70–646 %. Алгоритм автоматичної ідентифікації PSF забезпечив коректний вибір моделі у 100 % тестів. Критерій відсутності артефактів (Ringing Score < 8.0) виконується у трьох ключових випадках, включаючи приріст різкості 645.7 % для флуоресцентного зображення клітин U2OS.

Тестове зображення Місяця, що використовується у роботі, є авторською фотографією автора – студента 4-го курсу Ібатулліна І. Ш. Панорама Місяця (Приклад 4.1) у квітні 2026 р. набула широкого міжнародного розповсюдження у зв'язку з місією «Артеміда 2»: видання PolitiFact, Snopes та ряд міжнародних ЗМІ підтвердили, що ця та інші фотографії автора були помилково атрибутовані космічній місії NASA і є земними астрофотографіями, знятими з Києва за допомогою аматорського телескопа GSO 150/750 та камери Canon 550D.

Методи: математичне моделювання, алгоритм деконволюції Річардсона-Люсі, методи скалярної оптимізації, статистичний аналіз показників різкості, комп'ютерне моделювання та тестування.

Кваліфікаційна робота містить 69 сторінок основного тексту, 18 рисунків, 4 таблиці, 30 джерел.

ЗМІСТ

РЕФЕРАТ.....	2
ЗМІСТ.....	4
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	6
ВСТУП.....	8
РОЗДІЛ 1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА	
ЗАДАЧІ.....	10
1.1 Архітектура CMOS-сенсорів та процес формування сигналу.....	10
1.2 Класифікація шумів та їх математичні моделі.....	12
1.3 Явище перехресних завад (Crosstalk) та його вплив на точкові об'єкти.....	13
1.4 Огляд існуючих програмних рішень та постановка завдання.....	15
РОЗДІЛ 2 МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ АДАПТИВНОЇ СИСТЕМИ	
ВІДНОВЛЕННЯ.....	18
2.1 Математичні моделі функцій розсіювання точки.....	18
2.2 Метрики оцінки якості та різкості зображень.....	22
2.3 Алгоритм адаптивної деконволюції та оптимізація параметрів.....	24
РОЗДІЛ 3 ПРОГРАМНА РЕАЛІЗАЦІЯ ТА АРХІТЕКТУРА СИСТЕМИ.....	
3.1 Обґрунтування стеку технологій.....	32
3.2 Проєктування архітектури та функціональної структури програмного модуля.....	35
3.3 Реалізація модулів селекції та фільтрації.....	38
3.3.1 Модуль оцінки масштабу розмиття.....	39
3.3.2 Модуль автоматичної ідентифікації PSF.....	40
3.3.3 Модуль оптимізації масштабного параметра PSF.....	41

3.3.4 Реалізація алгоритму Річардсона-Люсі.....	42
3.3.5 Модуль адаптивного загострення.....	43
3.3.6 Реалізація графічного інтерфейсу.....	43
РОЗДІЛ 4 ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ ТА РЕЗУЛЬТАТИ.....	46
4.1 Методика тестування.....	46
4.2 Аналіз кількісних показників.....	48
4.3 Візуалізація та інтерпретація результатів.....	52
4.3.1 Планетарна зйомка.....	53
4.3.2 Знімки глибокого космосу.....	55
4.3.3 Дистанційне зондування Землі.....	57
4.3.4 Флуоресцентна мікроскопія.....	59
ВИСНОВКИ.....	65
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	67
ДОДАТОК А Лістинг модуля обробки зображень.....	70
ДОДАТОК Б Лістинг графічного інтерфейсу користувача.....	80
ДОДАТОК В Лістинг модуля порівняльного аналізу методів.....	87
ДОДАТОК Г Лістинг побудови одновимірних профілів моделей PSF.....	96
ДОДАТОК Д Лістинг побудови двовимірних теплових карт моделей PSF.....	98

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

Скорочення	Розшифрування
ADU	Analog-to-Digital Unit (одиниця сигналу АЦП)
BPM	Bad Pixel Map (карта дефектних пікселів)
CMOS	Complementary Metal-Oxide-Semiconductor (КМОН-структура)
Crosstalk	Перехресні завади (небажане перетікання сигналу між пікселями)
FFT	Fast Fourier Transform (швидке перетворення Фур'є)
FPN	Fixed Pattern Noise (шум фіксованого патерну)
FWHM	Full Width at Half Maximum (повна ширина на половині максимуму)
MSE	Mean Squared Error (середньоквадратична помилка)
PSF	Point Spread Function (функція розсіювання точки)
PRNU	Photo Response Non-Uniformity (нерівномірність фотовідгуку)
QE	Quantum Efficiency (квантова ефективність)
RL	Richardson-Lucy (ітеративний алгоритм Річардсона-Люсі)

SNR	Signal-to-Noise Ratio (співвідношення сигнал/шум)
АЦП	Аналого-цифровий перетворювач
ПЗЗ	Прилад із зарядовим зв'язком (CCD)

ВСТУП

Сучасні CMOS-сенсори (Complementary Metal-Oxide-Semiconductor) набули широкого застосування у галузях, що висувають підвищені вимоги до точності реєстрації зображень: астрономічних спостереженнях, медичній візуалізації, дистанційному зондуванні Землі та системах машинного зору. Порівняно з традиційними ПЗЗ-матрицями, CMOS-технологія забезпечує нижче енергоспоживання, вищу швидкість зчитування та можливість інтеграції додаткової логіки безпосередньо на кристалі сенсора. Разом із тим CMOS-матриці мають властиві їм систематичні спотворення: темновий струм, нерівномірність фотовідгуку пікселів (PRNU), фіксований патерн шуму (FPN) та явище перехресних завад (crosstalk). Тому відсутність належного калібрування призводить до систематичних помилок у фотометрії та зниження роздільної здатності.

Актуальність теми обумовлена відсутністю програмних рішень, здатних автоматично адаптувати алгоритми відновлення до конкретного типу спотворень без ручного налаштування параметрів. Існуючі підходи орієнтовані або на корекцію лише одного класу артефактів, або потребують апріорної інформації про характеристики сенсора.

Практичну значущість задачі підтверджує і реальний досвід автора: знімки Місяця, що слугують тестовим матеріалом у цій роботі, зняті за допомогою любительської камери на 150мм телескопі і пройшли ручну обробку, яка є прямим практичним аналогом методів, що розробляються. Якість отриманих зображень виявилась настільки високою, що у квітні 2026 р., під час місії «Артеміда 2», ці фотографії були масово сприйняті в соціальних мережах як офіційні знімки NASA – факт, підтверджений виданнями PolitiFact і Snopes. Це наочно демонструє, що коректна фільтрація та відновлення зображень CMOS-сенсорів дозволяють аматорському обладнанню досягати якості, яку важко відрізнити від результатів професійних космічних місій.

Мета роботи – розробка системи адаптивної фільтрації та автоматизованого відновлення зображень для CMOS-сенсорів наукового призначення, що забезпечує автоматичну ідентифікацію моделі PSF, оптимізацію її параметрів та компенсацію спотворень методами деконволюції.

Для досягнення поставленої мети необхідно вирішити такі завдання: провести аналіз архітектури CMOS-сенсорів, фізичних механізмів виникнення шумів та явища crosstalk; розробити математичні моделі PSF різних типів і визначити метрики оцінки якості зображень; обґрунтувати алгоритм адаптивної деконволюції та метод автоматизованої оптимізації параметрів PSF; спроектувати архітектуру програмного модуля та реалізувати підсистеми автоматичної селекції моделі PSF і відновлення зображень; виконати експериментальні дослідження та провести кількісний аналіз ефективності розробленої системи.

Методи дослідження: математичне моделювання процесів формування та спотворення зображень, алгоритм деконволюції Річардсона-Люсі, методи скалярної оптимізації, статистичний аналіз показників різкості та тестування.

Об'єкт дослідження – процеси формування, спотворення та відновлення цифрових зображень у CMOS-сенсорах наукового призначення. Предмет дослідження – методи та алгоритми адаптивної ідентифікації моделі PSF і деконволюції для автоматизованого відновлення зображень.

Практичне значення отриманих результатів полягає у розробці програмного інструментарію для автоматизованого відновлення зображень, який може бути застосований в астрономії, системах дистанційного зондування та медицині.

Структура та обсяг роботи. Кваліфікаційна робота складається зі вступу, чотирьох розділів, висновків, списку використаних джерел та додатків. Перший розділ присвячено аналізу предметної області та постановці задачі. Другий розділ містить математичне забезпечення адаптивної системи відновлення. Третій розділ описує програмну реалізацію та архітектуру системи. Четвертий розділ містить експериментальні дослідження та аналіз результатів.

РОЗДІЛ 1

АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Архітектура CMOS-сенсорів та процес формування сигналу

Сучасні системи науково-технічної зйомки базуються на використанні твердотільних перетворювачів світла, серед яких домінуюче положення займають сенсори, виготовлені за технологією комплементарних метал-оксид-напівпровідників (CMOS). На відміну від ПЗЗ-матриць (CCD), де зчитування заряду відбувається послідовно через обмежену кількість вихідних вузлів, архітектура CMOS типу Active Pixel Sensor (APS) передбачає наявність підсилювача безпосередньо в кожному пікселі [1]. Це забезпечує високу швидкість зчитування та низьке енергоспоживання, що є критичним для задач астрономії коротких часових масштабів і супутникового моніторингу.

На рис. 1.1 зображено типовий піксель сенсора, що складається з фотодіода та трьох або чотирьох транзисторів: транзистора скидання (Reset Tx), транзистора вибору (Select Tx) та вихідного джерела струму (Source Follower). У 4Т-архітектурі додається транзистор передачі заряду (Transfer Tx), що дозволяє реалізувати корельоване подвійне дискретизування (Correlated Double Sampling, CDS) безпосередньо на кристалі.

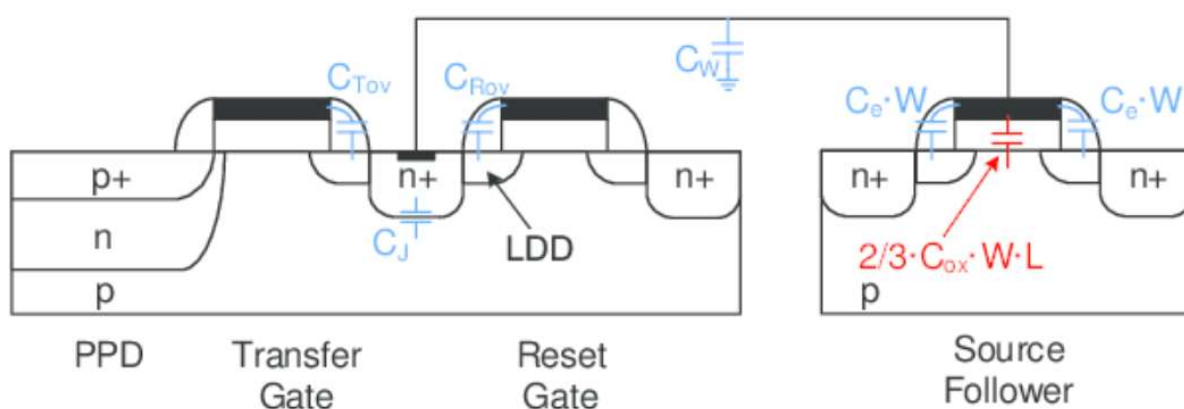


Рисунок 1.1 – Схематична структура та еквівалентна схема 4Т CMOS-пікселя

Процес формування зображення є послідовністю перетворень енергії фотонів у цифровий код. Падіння квантів світла на фоточутливу область пікселя (фотодіод) викликає генерацію електронно-діркових пар внаслідок внутрішнього фотоефекту. Кількість накопиченого заряду теоретично пропорційна добутку інтенсивності світлового потоку, квантової ефективності пікселя QE та часу експозиції t :

$$Q = QE \cdot \Phi \cdot t, \quad (1.1)$$

де Q – кількість накопичених фотоелектронів, Φ – інтенсивність світлового потоку, QE – квантова ефективність пікселя, t – час експозиції.

Накопичений заряд передається на вузол зберігання, де перетворюється у напругу підсилювачем Source Follower. Далі напруга надходить на аналого-цифровий перетворювач (АЦП), що формує цифрове значення пікселя у відносних одиницях (ADU, Analog-to-Digital Units). У сучасних наукових CMOS-сенсорах глибина квантування становить 12–16 біт, що забезпечує динамічний діапазон понад 60 дБ [2].

На практиці ідеальна лінійна передавальна характеристика (1.1) спотворюється низкою фізичних факторів. Загальна модель вихідного сигналу пікселя $\{i, j\}$ має вигляд:

$$y_{\{i,j\}} = t \cdot \eta_{\{i,j\}} \cdot \phi_{\{i,j\}} + D_{\{i,j\}(t,T)} + O_{\{i,j\}} + N_{\{read\}} + N_{\{shot\}}, \quad (1.2)$$

де $\eta_{\{i,j\}}$ – квантова ефективність пікселя з урахуванням нерівномірності (PRNU); $\phi_{\{i,j\}}$ – потік фотонів; $D_{\{i,j\}(t,T)}$ – темновий струм, залежний від температури T та часу витримки t ; $O_{\{i,j\}}$ – зміщення (Bias/Offset); $N_{\{read\}}$ – шум зчитування; $N_{\{shot\}}$ – дробовий шум фотонів.

Модель (1.2) є вихідною точкою для побудови алгоритмів калібрування, що розглядаються в подальших розділах.

1.2 Класифікація шумів та їх математичні моделі

У науковій літературі [3, 4] прийнято класифікувати шуми CMOS-сенсорів на часові (temporal noise), що змінюються від кадру до кадру, та просторові (spatial noise), які залишаються постійними для конкретного екземпляра сенсора.

Темновий струм (Dark Current) – сигнал, що генерується в пікселі за відсутності освітлення внаслідок теплової генерації електронів у кремнієвій підкладці. Темновий струм D підпорядковується рівнянню Арреніуса:

$$D(T) = D_0 \cdot e^{\left\{-\frac{E_a}{k_B T}\right\}}, \quad (1.3)$$

де D_0 – константа, E_a – енергія активації ($\sim 0,55$ - $0,65$ еВ для кремнію), k_B – стала Больцмана, T – абсолютна температура.

Подвоєння темного струму відбувається приблизно при збільшенні температури на 6 – 8 °C. У наукових камерах (наприклад, Teledyne COSMOS [5]) застосовуються системи глибокого охолодження, проте повністю усунути цей шум апаратно неможливо. Темновий струм додає адитивну постійну складову до сигналу пікселя, а пов'язаний з ним дробовий шум підпорядковується статистиці Пуассона.

Шум фіксованого патерну (Fixed Pattern Noise, FPN) – просторово-корельована варіація вихідного сигналу різних пікселів при однаковому освітленні. FPN виникає через технологічний розкид параметрів транзисторів у кожному пікселі та нерівномірність роботи підсилювачів стовпців [3]. На зображенні FPN проявляється як «вертикальні смуги» або стаціонарна зернистість. Для компенсації FPN на апаратному рівні використовують корельоване подвійне дискретизування (CDS), а на програмному – віднімання темного кадру (Dark Frame Subtraction).

Нерівномірність фотовідгуку (Photo Response Non-Uniformity, PRNU) – мультиплікативний шум, що характеризує різну чутливість (квантову ефективність) пікселів до світла. PRNU стає помітним при високих рівнях сигналу і пов'язаний з неоднорідністю товщини фотодіодів та фільтрів Байєра. Для корекції PRNU

застосовують метод «плоского поля» (Flat-Field Correction), однак він не враховує спектральні особливості об'єктів [6].

Дробовий шум фотонів (Shot Noise) – фундаментальний квантовий шум, зумовлений дискретною природою світла. Його стандартне відхилення σ пропорційне кореню квадратному від кількості зареєстрованих фотонів N : $\sigma = \sqrt{N}$. Цей шум не піддається усуненню калібруванням і визначає теоретичну межу точності вимірювань.

1.3 Явище перехресних завад (Crosstalk) та його вплив на точкові об'єкти

Однією з найскладніших проблем для прецизійної фотометрії та астрометрії є явище перехресних завад (crosstalk) – небажаного перетікання сигналу з одного пікселя до сусідніх. Відповідно до досліджень [7, 8], розрізняють три основні механізми виникнення crosstalk.

Оптичний crosstalk виникає внаслідок проникнення фотонів під мікролінзи сусідніх пікселів або відбиття від внутрішніх шарів підкладки. Його вплив зростає зі зменшенням розміру пікселя та є особливо суттєвим для тонких кремнієвих підкладок (BSI-сенсорів, Back-Side Illuminated).

Електричний (дифузійний) crosstalk обумовлений дифузією фотоелектронів, згенерованих у збідненій або нейтральній областях одного пікселя, у потенціальну яму сусіднього. Цей ефект домінує при реєстрації фотонів з великою довжиною хвилі (ближній ІЧ-діапазон), що глибше проникають у кремній.

Міжканальний (ємнісний) crosstalk – взаємний вплив електричних кіл зчитування сусідніх рядків або стовпців при високій щільності компонування матриці.

Особливу увагу crosstalk-ефектам приділено у звітах про калібрування інструментів телескопа James Webb [9]. Встановлено, що при накопиченні великого заряду в пікселі змінюється електростатичний потенціал у підкладці, що призводить до ефекту «міграції заряду» (Charge Migration). Це спричиняє так званий «brighter-

fatter effect»: чим яскравіше точкове джерело, тим ширшою стає його функція розсіювання точки (PSF), що суттєво ускладнює фотометричне калібрування яскравих зірок.

Математично вплив crosstalk на зображення описується через ядро PSF $h(x, y)$. Реальна PSF CMOS-сенсора є суперпозицією оптичної компоненти і компоненти розсіювання носіїв заряду:

$$h(x, y) = h_{\{opt\}}(x, y) * h_{\{ct\}}(x, y), \quad (1.4)$$

де $h_{\{opt\}}(x, y)$ – суперпозиція оптичної компоненти, $h_{\{ct\}}(x, y)$ – розсіювання носіїв заряду, $*$ позначає двовимірну згортку.

Ширина результуючої PSF перевищує дифракційну межу і нелінійно залежить від рівня сигналу при значних ступенях заповнення потенціальної ями.

Для підтвердження наявності описаних спотворень у реальних умовах проведено тестову зйомку галактики Андромеда (M31) за допомогою повнокадрового сенсора камери Sony a7s II, встановленої на телескоп GSO 150/750 з монтуванням Arsenal EQ5. Зйомка виконувалась з високими параметрами чутливості камери (ISO 8000) та тривалої витримки (30 с) без застосування калібрувальних кадрів. На рис. 1.2 чітко спостерігаються «гарячі пікселі» та розмиті профілі зірок, що підтверджує необхідність розробки системи компенсації розмиття.

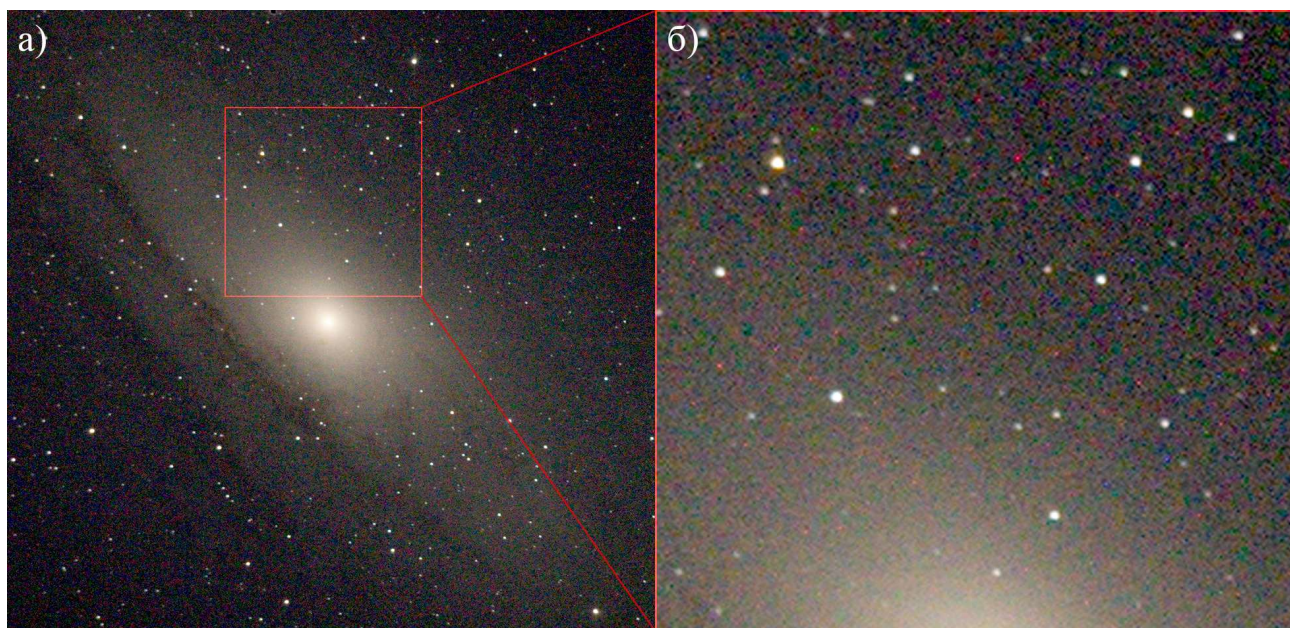


Рисунок 1.2 – Приклад спотворень на некаліброваному зображенні ядра галактики Андромеди (M31): а) загальний вигляд об'єкта; б) збільшений фрагмент з видимими "гарячими пікселями" та розмиттям зірок внаслідок crosstalk

1.4 Огляд існуючих програмних рішень та постановка завдання

Для мінімізації впливу описаних артефактів розроблено широкий спектр програмних та апаратних методів. Їх можна поділити на два класи: методи корекції адитивних та мультиплікативних спотворень і методи відновлення різкості (деконволюції).

Стандартний калібрувальний пайплайн, що описаний у [10], базується на отриманні трьох типів калібрувальних кадрів: кадру зміщення (Bias), темного кадру (Dark) і кадру «плоского поля» (Flat-Field) – та лінійній корекції кожного науково-знімального кадру. Бібліотека Astropy (Python) [11] є стандартом для таких операцій в астрономії, а пакет IRAF/AstroPy реалізує відповідний пайплайн для наземних обсерваторій. Проте ці методи ефективні лише проти стаціонарних шумів (FPN, PRNU) і не усувають ефекти розмиття та crosstalk.

Для відновлення різкості зображень точкових джерел застосовуються методи деконволюції. Найпоширенішим є ітеративний алгоритм Річардсона-Люсі [12], оснований на байєсівській максимізації правдоподібності за умови пуассонівського розподілу шуму. Він реалізований у бібліотеках `scikit-image` та `DeconvolutionLab2` (ImageJ/Fiji). Серед недоліків методу – поява артефактів у вигляді «кілець» (ringing artifacts) при надмірній кількості ітерацій. Алгоритм Вінера (Wiener Filter) забезпечує більш швидке відновлення у частотній області, але не гарантує невід'ємності результату та поступається методу Річардсона-Люсі при пуассонівському шумі.

Останніми роками набувають популярності методи на основі глибокого навчання. У роботі [13] представлено нейромережу CycleMRSNet для корекції нерівномірності чутливості, яка перевершує класичні підходи за рядом метрик. Перспективним є також застосування методів стисненого зондування (Compressive Sensing) для калібрування сенсорів [14]. Однак більшість підходів машинного навчання вимагають великих обсягів тренувальних даних, що є суттєвим обмеженням для спеціалізованих наукових інструментів.

Незважаючи на велику кількість наявних інструментів, жоден із них не вирішує задачу комплексно: більшість рішень є або вузькоспеціалізованими, або не адаптуються автоматично до конкретного типу PSF без ручного налаштування параметрів. Алгоритми деконволюції з фіксованою гаусовою моделлю PSF часто призводять до появи артефактів при обробці зображень з важкохвостими розподілами яскравості (Cauchy, Moffat, Airy). Існуючі рішення не виконують автоматичний вибір моделі PSF на основі статистичних характеристик вхідного зображення.

Наведений аналіз визначає наступні завдання кваліфікаційної роботи:

- провести аналіз математичних моделей PSF різних типів (Gaussian, Cauchy, Airy, Moffat, motion, scattering) та метрик оцінки якості зображень;

- розробити алгоритм автоматичної ідентифікації моделі PSF на основі статистичного аналізу градієнтів та спектральних характеристик вхідного зображення;
- обґрунтувати метод оптимізації параметрів PSF на основі максимізації метрики різкості Tenengrad;
- спроектувати та реалізувати програмний модуль адаптивної деконволюції з підтримкою 8- та 16-бітних зображень у форматах TIFF, FITS та стандартних растрових форматах;
- провести експериментальне тестування системи на реальних даних та виконати кількісний аналіз ефективності за метриками Tenengrad і Laplacian Variance.

РОЗДІЛ 2

МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ АДАПТИВНОЇ СИСТЕМИ ВІДНОВЛЕННЯ

2.1 Математичні моделі функцій розсіювання точки

Як зазначено в підрозділі 1.1, процес реєстрації зображення CMOS-сенсором описується моделлю лінійної згортки. Зареєстроване зображення $g(x, y)$ подається рівнянням Фредгольма першого роду [15]:

$$g(x, y) = \iint_{-\infty}^{\infty} f(\alpha, \beta) h(x - \alpha, y - \beta) d\alpha d\beta + n(x, y), \quad (2.1)$$

де $f(\alpha, \beta)$ – істинний розподіл яскравості об'єкта у фокальній площині; $h(x - \alpha, y - \beta)$ – функція розсіювання точки (PSF); $n(x, y)$ – адитивний шум, що включає темновий струм, шум зчитування та дробовий шум фотонів; α, β – просторові змінні інтегрування.

У скороченому записі (2.1) набуває вигляду:

$$g = h * f + n, \quad (2.2)$$

де символ $*$ позначає операцію двовимірної згортки. Завданням системи відновлення є розв'язання оберненої задачі: знаходження оцінки $f(x, y)$ за спотвореним зображенням $g(x, y)$ та моделлю PSF $h(x, y)$.

Реальна PSF оптико-сенсорної системи залежить від типу оптичних аберацій, атмосферних умов, характеристик сенсора та рівня сигналу. Для параметричної апроксимації PSF в рамках даної роботи розглянуто шість аналітичних моделей.

Гаусова модель є стандартною для апроксимації атмосферного розмиття в астрономії та розсіювання в мікроскопії [16, 17]:

$$h_G(x,y) = \left(\frac{1}{2\pi\sigma^2}\right) e^{\left\{-\frac{x^2+y^2}{2\sigma^2}\right\}}, \quad (2.3)$$

де параметр σ – стандартне відхилення, що визначає ширину піка. Модель має «легкі» хвости (light tails) і добре описує дифракційно обмежені системи при помірних рівнях сигналу.

Модель Коші (Лоренца) характеризується «важкими» хвостами (heavy tails) і краще описує профіль зірок в умовах атмосферної турбуленції:

$$h_C(x,y) = \left(\frac{1}{(2\pi\gamma^2)}\right) \left(1 + \frac{(x^2+y^2)}{\gamma^2}\right)^{-1.5}, \quad (2.4)$$

де γ – масштабний параметр. Профіль Коші краще апроксимує зірки при атмосферній турбуленції. Як показано в роботах [16, 17], гаусова апроксимація систематично недооцінює яскравість у хвостах профілю зірок, що призводить до похибок фотометрії.

Диск Ейрі є теоретично точною PSF дифракційно обмеженої оптичної системи з круглою апертурою [15]:

$$h_A(x,y) = \left(\frac{\left(2 J_1\left(\frac{\pi r}{R}\right)\right)}{\left(\frac{\pi r}{R}\right)}\right)^2, \quad r = \sqrt{(x^2 + y^2)}, \quad (2.5)$$

де J_1 – функція Бесселя першого роду першого порядку; R – радіус першого темного кільця дифракційної картини (радіус Ейрі), пов'язаний з довжиною хвилі λ та числовою апертурою NA : $R = 0.61\lambda/NA$.

Профіль Моффата широко використовується в астрономії для апроксимації профілю зірок при атмосферних спотвореннях, оскільки, як доведено в роботі [16, 17], він є більш гнучким, ніж гаусова модель:

$$h_{M(x,y)} = \frac{\beta-1}{\pi\alpha^2} \left(1 + \frac{(x^2+y^2)}{\alpha^2}\right)^{-\beta}, \quad (2.6)$$

де α – масштабний параметр, β – показник степеня, що керує формою хвостів. При $\beta \rightarrow \infty$ профіль Моффата наближається до гаусового, при $\beta = 1$ до профілю Лоренца. Типові значення для астрономічних зображень: $\beta \in [2.5; 4.5]$.

Модель розмиття руху (Motion Blur) описує PSF, що виникає при відносному переміщенні об'єкта або камери під час експозиції. При умові, що точка (x, y) лежить на відрізку довжиною L під кутом θ до горизонталі, і 0 в іншому випадку.

$$h_{mot(x,y)} = \frac{1}{L}, \quad (2.7)$$

де L – довжина розмиття у пікселях, θ – кут розмиття.

Модель розсіювання (Scattering) описує PSF, зумовлену розсіюванням фотонів у підкладці сенсора або в середовищі (характерна для ближнього ІЧ-діапазону та BSI-сенсорів) [7, 9]:

$$h_{sc(x,y)} = \frac{2\pi}{k^2} \exp(-k * \sqrt{x^2 + y^2}), \quad (2.8)$$

де k – коефіцієнт загасання розсіяного світла.

Всі шість моделей нормовані так, що $\iint h(x, y) dx dy = 1$, що забезпечує збереження сумарної яскравості (flux conservation) при деконволюції.

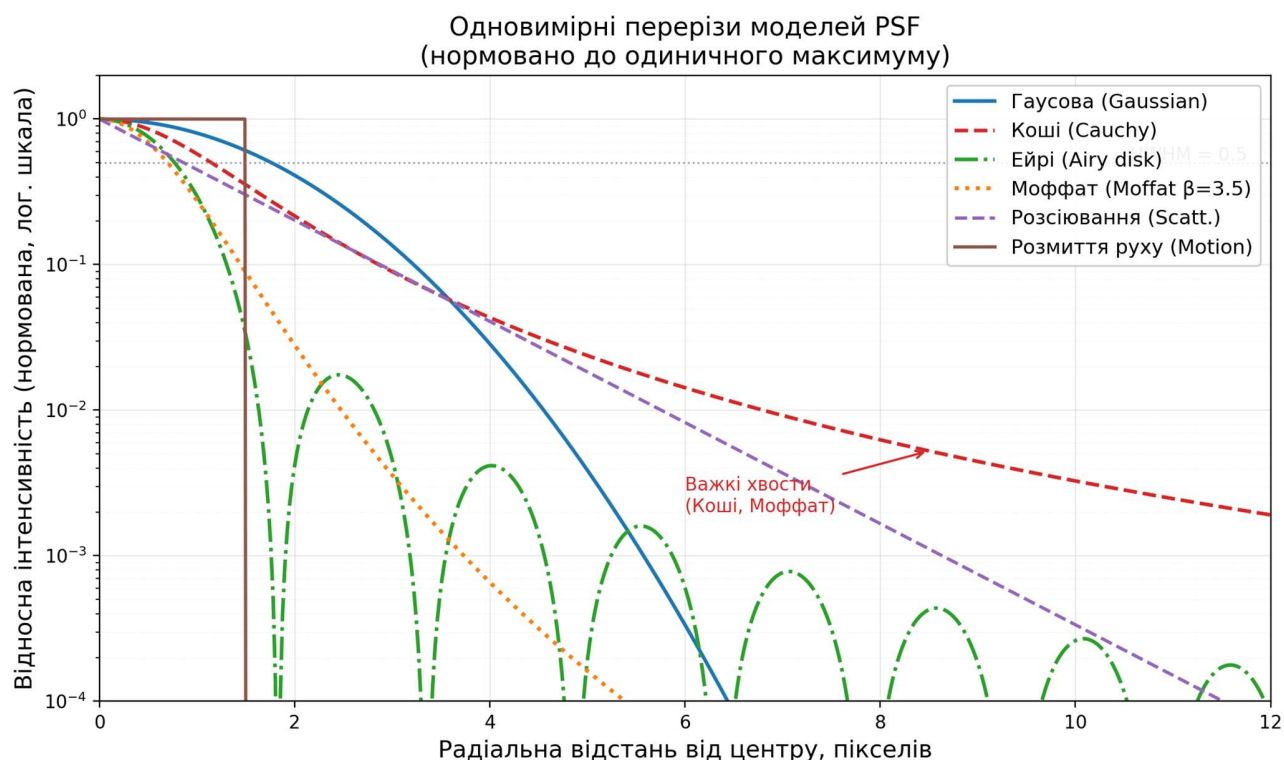


Рисунок 2.1 – Одномерні радіальні перерізи аналітичних моделей PSF у логарифмічному масштабі. Всі профілі нормовано до одиничного максимуму.

Горизонтальна штрихова лінія відповідає рівню 0.5 (FWHM). Важкохвости розподіли (Коші, Моффат) суттєво перевищують гаусовий профіль на відстанях понад 4 пікселі від центру, що визначає вибір відповідної моделі PSF для прецизійної деконволюції

З рис. 2.1 видно, що на відстані 8 пікселів від центру інтенсивність профілю Коші на 2–3 порядки перевищує гаусовий профіль з аналогічним FWHM. Саме ця різниця обумовлює появу ringing-артефактів при застосуванні гаусової деконволюції до зображень з важкохвостою PSF.

Двовимірні ядра функцій розсіювання точки (PSF)
(логірифімічна кольорова шкала, нормовано до максимуму)

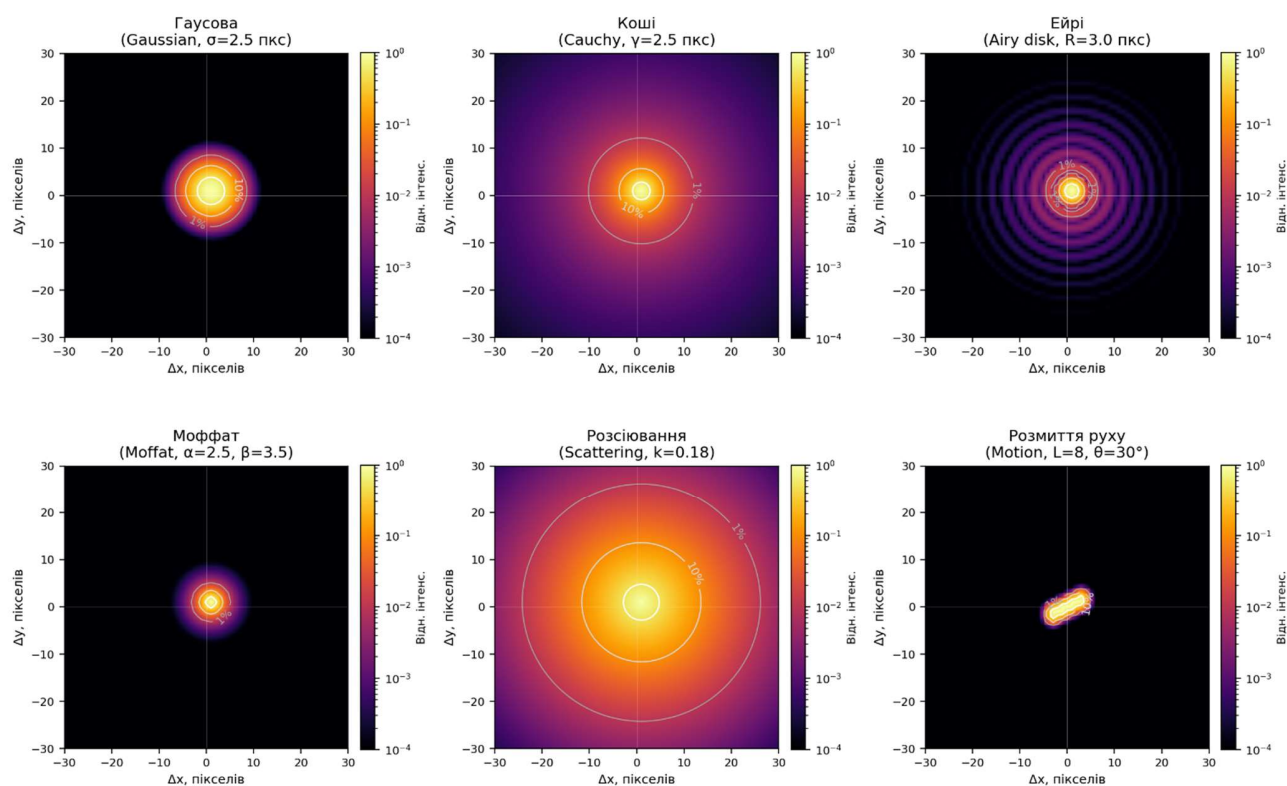


Рисунок 2.2 – Двовимірні ядра моделей PSF у логарифмічній кольоровій шкалі.
Контурні лінії відповідають рівням відносної інтенсивності 50% (HWHM), 10% та 1%. Розмір відображеної області: ± 30 пікселів від центру

2.2 Метрики оцінки якості та різкості зображень

Для автоматизованої оптимізації параметрів PSF та кількісної оцінки ефективності відновлення використовується набір метрик різкості зображення. На відміну від таких метрик, як PSNR або SSIM, що вимагають еталонного зображення, наведені нижче метрики є безеталонними (no-reference) і можуть обчислюватись безпосередньо для вхідного зображення.

Метрика Tenengrad ґрунтується на обчисленні градієнта яскравості за допомогою оператора Собеля та є основною метрикою для оптимізації параметрів PSF у цій роботі [18]:

$$T(f) = \left(\frac{1}{(MN)}\right) * \Sigma\Sigma \left([G_x(x, y)]^2 + [G_y(x, y)]^2\right), \quad (2.9)$$

де $[G_x(x, y)]^2$ і $[G_y(x, y)]^2$ – горизонтальна та вертикальна компоненти градієнта зображення f , обчислені згорткою з ядрами Собеля S_x та S_y відповідно; M, N – розміри зображення.

Метрика Tenengrad чутлива до широкого спектру просторових частот і є монотонно зростаючою функцією різкості: чим більше значення $T(f)$, тим різкішим є зображення. Висока обчислювальна ефективність і надійність при пуассонівському шумі обумовлюють вибір саме цієї метрики як цільової функції при оптимізації параметрів PSF.

Варіація лапласіану вимірює концентрацію другої похідної яскравості [18]:

$$L(f) = Var(\nabla^2 f) = \left(\frac{1}{(MN)}\right) * \Sigma\Sigma (\nabla^2 f(x, y) - \mu_L)^2, \quad (2.10)$$

де $\nabla^2 f$ – лапласіан зображення, μ_L – його середнє значення. Метрика $L(f)$ є більш чутливою до дрібних текстурних деталей і доповнює метрику Tenengrad при аналізі результатів відновлення.

Оцінка артефактів дзвону (Ringing Score) виявляє надмірну деконволюцію, що призводить до появи паразитних коливань яскравості навколо яскравих деталей. Для цього використовується ексцес (excess kurtosis) розподілу значень лапласіану:

$$R(f) = kurt(\nabla^2 f) = \left(\frac{\mu_4(\nabla^2 f)}{\sigma^4(\nabla^2 f)}\right) - 3, \quad (2.11)$$

де $\mu_4(\nabla^2 f)$ – четвертий центральний момент розподілу значень лапласіану зображення (верхній індекс «4» позначає степінь у визначенні центрального моменту: $\mu_4 = E[(X - E[X])^4]$); $\sigma^4(\nabla^2 f)$ – четвертий степінь стандартного

відхилення того самого розподілу (верхній індекс «4» при σ забезпечує безрозмірність відношення). Нормування на σ^4 робить метрику інваріантною до масштабу яскравості зображення, тобто її значення не залежить від загальної контрастності кадру.

При відсутності артефактів розподіл лапласіану є близьким до нормального ($R \approx 0$). Поява виражених ringing-артефактів збільшує ексцес ($R > 8$), що сигналізує про необхідність зменшення кількості ітерацій або послаблення параметра PSF [26].

Коефіцієнт напрямковості градієнта дозволяє виявити розмиття руху (motion blur) шляхом аналізу асиметрії розподілу градієнтів за напрямками [26]:

$$D(f) = \frac{(\max(\mu|G_x|, \mu|G_y|))}{(\min(\mu|G_x|, \mu|G_y|) + \epsilon)}, \quad (2.12)$$

де $\mu|G_x|$ і $\mu|G_y|$ – середні абсолютні значення горизонтальної та вертикальної компонент градієнта відповідно; $\epsilon = 10^{-9}$ мале число для уникнення ділення на нуль. Значення $D(f) > 2.5$ є ознакою переважно однонаправленого розмиття і є критерієм для автоматичного вибору моделі motion blur.

2.3 Алгоритм адаптивної деконволюції та оптимізація параметрів

Центральним елементом розробленої системи є алгоритм адаптивної деконволюції, що реалізує три послідовних етапи: автоматичну ідентифікацію моделі PSF, оптимізацію її параметрів та ітеративне відновлення зображення з подальшим адаптивним загостренням.

Етап 1. Автоматична ідентифікація моделі PSF.

Вибір моделі PSF виконується на основі статистичного аналізу характеристик вхідного зображення. Алгоритм ідентифікації діє за таким деревом рішень.

Спочатку обчислюється коефіцієнт напрямковості $D(f)$ за формулою (2.12). Якщо $D(f) > 2.5$, то констатується наявність переважно однонаправленого

розмиття і вибирається модель motion blur. При цьому кут розмиття θ визначається автоматично з аналізу спектру потужності зображення: обчислюється двовимірне перетворення Фур'є, після придушення постійної складової ідентифікується орієнтація головної осі інерції розподілу потужності на вищих просторових частотах.

Якщо $D(f) \leq 2.5$, то виконується аналіз ексцесу розподілу градієнтів вхідного зображення:

$$\kappa = \left(\frac{\mu_4(G_x \cup G_y)}{\sigma^4(G_x \cup G_y)} \right) - 3, \quad (2.13)$$

де конкатенація $G_x \cup G_y$ означає формування єдиного одновимірного масиву, що містить усі значення горизонтальних градієнтів $G_x(x,y)$ та вертикальних градієнтів $G_y(x,y)$ по всіх пікселях зображення. Таке об'єднання виконується поелементним злиттям двох матриць в один вектор без будь-якого розрізнення за напрямком: κ обчислюється для спільного розподілу $|G_x| \cup |G_y|$. Це подвоює статистичну вибірку порівняно з аналізом лише одного напрямку градієнта, знижує чутливість до просторово-анізотропних текстур (наприклад, горизонтальних смуг атмосфери Юпітера або вертикальних волокон актину) та дає більш стабільну оцінку ексцесу для зображень малого розміру. Відповідно до значення ексцесу κ модель PSF вибирається за правилом:

$\kappa > 10.0 \rightarrow$ модель Коші (дуже важкі хвости, сильна турбулентність); $5.0 < \kappa \leq 10.0 \rightarrow$ модель Моффата (помірно важкі хвости); $1.5 < \kappa \leq 5.0 \rightarrow$ диск Ейрі (близько до дифракційної межі); $\kappa \leq 1.5 \rightarrow$ гаусова модель (атмосферне розмиття, мікроскопія).

Фізичний зміст застосування ексцесу саме до розподілу градієнтів полягає у такому: якщо зображення розмите важкохвостою PSF (Коші або Моффат), то переважна більшість пікселів мають дуже малий градієнт (розмите «тіло»), але окремі контрастні переходи зберігають значні значення градієнта (хвости PSF). Таке

поєднання «маса у нулі + рідкісні великі значення» і є визначенням лептокуртичного розподілу. Навпаки, при гаусовому розмитті градієнти рівномірніше розподілені навколо нуля, і ексцес є низьким.

Порогові значення $\kappa > 10.0, 5.0 < \kappa \leq 10.0, 1.5 < \kappa \leq 5.0$ та $\kappa \leq 1.5$ є авторською евристикою, сформованою шляхом суб'єктивного підбору на основі аналізу реальних астрономічних зображень різних класів: зоряних полів, планетарних туманностей та знімків планет. Для кожної категорії об'єктів відома фізично обґрунтована модель PSF (наприклад, профіль Моффата для атмосферних спотворень [16, 17], диск Ейрі для дифракційно обмежених систем [15]), що дозволило зіставити реальні значення ексцесу κ , отримані з цих зображень, з очікуваною моделлю PSF та визначити граничні значення розподілу. Отримані пороги підтверджено на тестовому наборі реальних зображень у розділі 4. Конкатенація $G_x \cup G_y$ означає, що аналізується один спільний розподіл усіх градієнтних значень по обох осях, що підвищує статистичну вибірку вдвічі та знижує чутливість до просторово-анізотропних текстур.

Етап 2. Оптимізація параметрів PSF.

Після вибору моделі PSF виконується автоматична оптимізація її масштабного параметра γ (або σ, α, k залежно від типу моделі). Оптимальне значення γ^* знаходиться шляхом мінімізації негативної метрики Tenengrad відновленого зображення:

$$\gamma^* = \arg \min_{\gamma \in [\gamma_{min}, \gamma_{max}]} \left(-T(\hat{f}_\gamma) \right), \quad (2.14)$$

де $\hat{f}_\gamma$ – результат попереднього відновлення.

Задача (2.14) формально є одновимірною задачею безумовної мінімізації (або максимізації метрики різкості) на обмеженому відрізку $[\gamma_{min}, \gamma_{max}]$. Інтервал пошуку визначається автоматично функцією на основі попередньої оцінки масштабу розмиття σ_{est} : нижня межа $\gamma_{min} \approx 0.3$ пікс. відповідає дифракційній

межі, верхня γ_{max} залежить від домену і становить від 4.5 пікс. (planetary) до 12.0 пікс. (deep_space).

Для мінімізації застосовується метод золотого перерізу (golden-section search) – один із класичних методів скалярної оптимізації на обмеженому відрізку без використання похідних [28]. На кожному кроці метод звужує інтервал пошуку у співвідношенні золотого перерізу $\phi = (\sqrt{5} - 1)/2 \approx 0.618$, розміщуючи дві пробні точки симетрично всередині поточного інтервалу. Завдяки цьому кожна нова ітерація вимагає лише одного нового обчислення цільової функції (друга пробна точка переноситься з попереднього кроку). Метод гарантує збіжність за $O(\log(1/\epsilon))$ обчислень цільової функції для будь-якої унімодальної функції у заданому інтервалі. Точність $\epsilon = 10^{-3}$ пікселя обрана з таких міркувань: при розмірі пікселя сенсора порядку 1–5 мкм похибка оцінки γ менша за 10^{-3} пікс. не дає помітного покращення метрики Tenengrad відновленого зображення, натомість подальше зменшення ϵ лінійно збільшує кількість ітерацій оптимізатора. На практиці цей вибір забезпечує збіжність за 12–18 обчислень цільової функції, що є прийнятним для інтерактивного режиму роботи.

Практична унімодальність функції $-T(\hat{f}_\gamma)$ обумовлена фізичним змістом: при надто малому γ ядро PSF не компенсує реальне розмиття (різкість не зростає), при надто великому γ ядро «перекомпенсує» PSF і вносить нові артефакти (різкість знижується). Між цими крайнощами існує єдиний максимум. На практиці для кожного значення γ у ході оптимізації будується ядро PSF, виконується 5 попередніх ітерацій RL-деконволюції та обчислюється Tenengrad. Кількість попередніх ітерацій 5 обрана як компроміс між обчислювальною вартістю одного виклику цільової функції та точністю оцінки: більша кількість ітерацій лише незначно впливає на положення оптимального γ^* , але суттєво збільшує час оптимізації. На реальних тестових зображеннях метод сходиться за 12–18 викликів цільової функції.

Етап 3. Ітеративна деконволюція алгоритмом Річардсона-Люсі.

Після знаходження γ^* будується оптимальне ядро h^* і виконується повне відновлення зображення за ітеративною формулою Річардсона-Люсі [12, 19]:

$$\hat{f}^{(k+1)}(x,y) = \hat{f}^{(k)}(x,y) * \left(h^{*(-x,-y)} * \left(\frac{(g(x,y))}{(h^* * \hat{f}^k)(x,y)} \right) \right), \quad (2.15)$$

де $k = 0, 1, \dots, K - 1$ – номер ітерації; K задана кількість ітерацій. Початкове наближення $f(0) = g$.

Формула (2.15) є точним дискретним аналогом безперервного рівняння Річардсона-Люсі, виведеного з умови максимізації логарифму правдоподібності при пуассонівському розподілі шуму [12, 19]. Множник у квадратних дужках є відношенням спостережуваного зображення g до «передбаченого» зображення $h^* * \hat{f}^k$; якщо поточна оцінка $\hat{f}^{(k)}(x,y)$ точна, це відношення дорівнює одиниці і оцінка не змінюється. Якщо оцінка занижена в деякій точці, відношення > 1 і оцінка зростає; якщо завищена – відношення < 1 і оцінка зменшується. Операція згортки множника з перевернутим ядром $h^{*(-x,-y)}$ виконує «зворотну проєкцію» похибки у простір об'єкта. Алгоритм гарантує дві важливі властивості: невід'ємність результату (оскільки добуток двох невід'ємних величин є невід'ємним) та збереження сумарного потоку ($\sum \hat{f}^k = \text{const}$ при кожній ітерації), що критично для фотометричних застосувань в астрономії та мікроскопії. Збіжність контролюється через Ringing Score (2.11): перевищення порогу $R > 8$ сигналізує про початок нестабільного режиму підсилення шуму, що є характерним недоліком алгоритму RL при надмірній кількості ітерацій [26]. Практичне число ітерацій k для різних доменів: 5 для астрономічних та супутникових зображень з яскравими точковими джерелами; 15–20 для флуоресцентної мікроскопії, де PSF добре апроксимується диском Ейрі і алгоритм збігається повільніше.

Етап 4. Адаптивне багатомасштабне загострення.

Після деконволюції застосовується додатковий етап загострення для підсилення деталей, що не відновлюються деконволюцією (низькоконтрастні

текстури, градієнтні переходи). Метод базується на багатомасштабному нерізкому маскуванні (Multiscale Unsharp Masking):

$$f_{final} = clip\left(\hat{f} + \alpha * (\hat{f} - G_{\sigma_1} * \hat{f}) + \left(\frac{\alpha}{2}\right) * (\hat{f} - G_{\sigma_2} * \hat{f}), 0, 1\right), \quad (2.16)$$

де $G_{\sigma_1} * \hat{f}$ – гаусове розмиття відновленого зображення з параметром σ ; $\sigma_1 = 0.5$ пікс. (дрібний масштаб), $\sigma_2 = 1.5$ пікс. (середній масштаб); α – параметр інтенсивності загострення.

Метод нерізкого маскування (Unsharp Masking) у класичному варіанті додає до зображення різницю між ним самим та його розмитою версією: $\hat{f} + \alpha * (\hat{f} - G_{\sigma_1} * \hat{f})$. Формула (2.16) є двомасштабним узагальненням: перший доданок $\alpha * (\hat{f} - G_{\sigma_1} * \hat{f})$ підсилює деталі дрібного масштабу (текстури, тонкі межі), другий доданок $\left(\frac{\alpha}{2}\right) * (\hat{f} - G_{\sigma_2} * \hat{f})$ підсилює деталі середнього масштабу (контури об'єктів, великі краї). Параметри $\sigma_1 = 0.5$ пікс. та $\sigma_2 = 1.5$ пікс. обрані так, щоб охоплювати просторові частоти від ~ 0.3 до ~ 0.7 від частоти Найквіста, де зосереджена більша частина інформації про деталі після RL-деконволюції. Менший коефіцієнт $\frac{\alpha}{2}$ при другому члені обмежує підсилення низькочастотних контурів, запобігаючи появі ефекту «Mach bands» (паразитне посвітління/затемнення уздовж країв). Функція $clip(\cdot, 0, 1)$ забезпечує збереження динамічного діапазону: перепідсилення або від'ємні значення, які могли б виникнути при великому α , відсікаються на межах. Додатковий коефіцієнт $\frac{\alpha}{2}$ (замість α) для другого масштабу обраний емпірично на основі балансу між підсиленням контурних деталей та уникненням Mach-ефекту: при однакових коефіцієнтах середньочастотне підсилення домінує і зображення набуває «малюнкowego» вигляду.

Значення α обирається адаптивно залежно від стандартного відхилення σ_f яскравості зображення:

$$\alpha = \text{clip}(1.0 + (0.4 - \sigma_f) * 4.0, 0.3, 3.0), \quad (2.17)$$

де α – параметр інтенсивності загострення. Це забезпечує сильніше загострення для темних зображень і слабше для яскравих. Якщо Ringing Score $R(f) > 8$, параметр α додатково зменшується вдвічі для запобігання підсиленню наявних артефактів.

Формула (2.17) реалізує просту лінійну залежність: чим менше стандартне відхилення σ_f яскравості зображення, тим більший коефіцієнт загострення α застосовується. Точка рівноваги $\sigma_f = 0.4$ відповідає зображенню з помірним контрастом (нормалізований діапазон $[0, 1]$), де $\alpha = 1.0$. При $\sigma_f = 0.15$ (дуже темне зображення) α досягає верхньої межі 3.0. При $\sigma_f = 0.65$ (яскраве насичене зображення, типово для планетарної зйомки) α знижується до нижньої межі 0.3. Нахил коефіцієнта 4.0 обраний так, що зміна σ_f на 0.1 призводить до зміни α на 0.4 – достатньо для помітної адаптації без стрибків між кадрами. Межі $\text{clip}(\cdot, 0.3, 3.0)$ встановлені емпірично: нижня межа 0.3 гарантує мінімальне базове загострення навіть для яскравих зображень, де деконволюція вже відновила більшість деталей; верхня межа 3.0 запобігає надмірному підсиленню шуму для дуже темних зображень. Якщо Ringing Score $R > 8$, параметр α додатково зменшується вдвічі – це означає, що зображення вже містить кільцеві артефакти від деконволюції, і додаткове загострення лише підсилить їх.

Таким чином, розроблений алгоритм не вимагає жодного ручного налаштування параметрів: усі рішення щодо вибору моделі PSF, її параметрів та інтенсивності загострення приймаються автоматично на основі статистичного аналізу вхідного зображення.

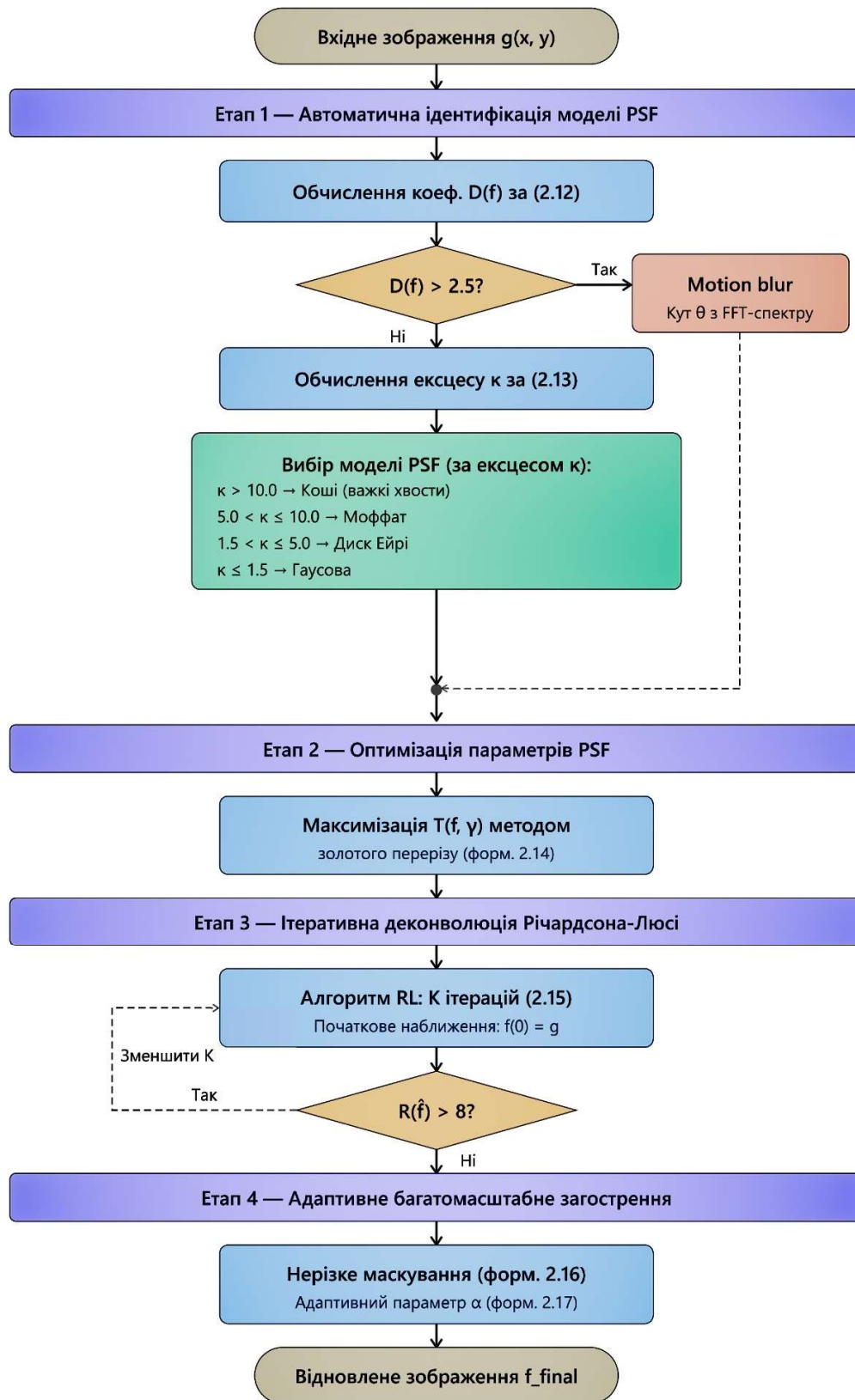


Рисунок 2.2 – Блок-схема чотириетапного алгоритму адаптивної деконволюції

РОЗДІЛ 3

ПРОГРАМНА РЕАЛІЗАЦІЯ ТА АРХІТЕКТУРА СИСТЕМИ

3.1 Обґрунтування стеку технологій

Вибір технологічного стеку для реалізації системи адаптивної фільтрації та відновлення зображень CMOS-сенсорів наукового призначення визначався сукупністю вимог: підтримкою 16-бітних зображень у форматах TIFF та FITS, ефективним виконанням операцій двовимірної згортки великих масивів, наявністю апробованих реалізацій чисельної оптимізації, а також можливістю розгортання у середовищах без спеціалізованого апаратного забезпечення. Проведений аналіз наявних платформ засвідчив переваги мови програмування Python версії 3.10+ як основного інструментарію.

Мова Python є де-факто стандартом для наукових обчислень і обробки зображень завдяки розвиненій екосистемі бібліотек, що безпосередньо підтримують необхідні операції [11]. Критичні за продуктивністю обчислення виконуються у скомпільованому коді C/Fortran через відповідні Python-обгортки, що забезпечує прийнятну швидкість обробки без необхідності низькорівневого програмування. Нижче обґрунтовано вибір кожної ключової бібліотеки.

NumPy [27] є фундаментальним пакетом для роботи з багатовимірними масивами даних. У межах розробленої системи він використовується для всіх внутрішніх представлень зображень у форматі float32/float64, побудови координатних сіток для обчислення ядер PSF, а також для ефективних векторизованих обчислень статистичних показників градієнтів.

SciPy [28] надає дві критично важливі функції: `fftconvolve` для виконання двовимірної згортки у частотній області із застосуванням алгоритму FFT (що є обчислювально ефективнішим за пряму згортку при великих розмірах ядра), а також `minimize_scalar` із методом `bounded` (золотий переріз) для одновимірної скалярної оптимізації масштабного параметра PSF. Метод `minimize_scalar` із методом `bounded`

гарантує збіжність за скінченне число кроків для унімодальних функцій у заданому інтервалі.

OpenCV [29] застосовується для операцій вводу/виводу зображень у форматах PNG, JPEG, TIFF з підтримкою 8- та 16-бітних даних, обчислення операторів Собеля та Лапласіана, а також перетворення кольорового простору між BGR та CIE-Lab. Використання OpenCV замість PIL/Pillow обумовлено підтримкою 16-бітних зображень (cv2.IMREAD_UNCHANGED) та суттєво вищою продуктивністю при роботі з numpy-масивами.

scikit-image [30] використовується виключно для операції гаусового розмиття (filters.gaussian) на етапі адаптивного нерізкого маскуванню. Ця функція забезпечує коректну обробку граничних умов та ефективну реалізацію для довільних значень sigma.

Astropy [11] підключається як опціональна залежність (через блок try/except) для підтримки формату FITS (Flexible Image Transport System), що є стандартом для астрономічних спостережень. Завантаження FITS-файлів виконується через astropy.io.fits з автоматичною нормалізацією за перцентилями 0.1 та 99.9 для усунення граничних артефактів.

Бібліотека Tkinter у поєднанні з matplotlib (бекенд TkAgg) утворює графічний інтерфейс користувача. Tkinter є стандартною бібліотекою Python та не потребує встановлення. Matplotlib забезпечує відображення зображень до та після обробки з коректною передачею кольору, зокрема grayscale-режим для одноканальних зображень.

Таблиця 3.1 – Стек технологій та обґрунтування вибору

Компонент	Бібліотека / версія	Призначення	Обґрунтування вибору
Обчислення масивів	NumPy $\geq$ 1.24	Внутрішнє представлення зображень та ядер PSF	Векторизовані операції на рівні C; де-факто стандарт наукових обчислень
Згортка та оптимізація	SciPy $\geq$ 1.11	FFT-згортка (fftconvolve), скалярна мінімізація	Ефективна FFT-реалізація $O(N \log N)$; метод golden-section гарантує збіжність
Обробка зображень	OpenCV $\geq$ 4.8	I/O форматів, оператори Собеля/Лапласіана, CIE-Lab	Підтримка 16-бітних TIFF; висока продуктивність numpy-інтерфейсу
Фільтрація	scikit-image $\geq$ 0.21	Гаусове розмиття для unsharp masking	Коректна обробка границь; сумісність з numpy float32
Астрономічний I/O	Astropy $\geq$ 5.3 (optional)	Читання FITS-файлів	Галузевий стандарт; авто-нормалізація за перцентилями
GUI	Tkinter + matplotlib	Графічний інтерфейс, порівняльний перегляд	Стандартна бібліотека без зовнішніх залежностей; TkAgg для вбудованих canvas

Загальна схема залежностей побудована за принципом мінімалізму: кожна бібліотека виконує чітко визначену роль і не дублює функціональність інших компонентів. Astropy оголошена необов'язковою залежністю, що дозволяє розгортати систему без астрономічного оточення у задачах медичної візуалізації та дистанційного зондування. Весь стек є крос-платформним і тестованим на операційних системах Windows 10/11.

3.2 Проєктування архітектури та функціональної структури програмного модуля

Програмний модуль реалізовано у вигляді двох Python-файлів з чітко розмежованими зонами відповідальності. Файл `diploma_engine.py` містить весь обчислювальний конвеєр – від завантаження зображення до виведення відновленого результату, – а `diploma_gui.py` реалізує виключно графічний інтерфейс користувача, взаємодіючи з двигуном через публічний API класу `RestorationEngine`. Такий поділ забезпечує незалежне тестування обчислювального ядра, можливість пакетного запуску без GUI, а також спрощує подальший розвиток інтерфейсу.

Архітектура обчислювального ядра організована у вигляді п'яти основних класів та набору допоміжних функцій. Нижче описано функціональне призначення кожного компонента.

Клас `EngineConfig` є структурою даних (dataclass) без методів, що інкапсулює всі налаштовувані параметри конвеєра: домен зйомки, модель PSF, розмір ядра, кількість ітерацій, межі пошуку параметра масштабу, прапорці режимів обробки та рівень діагностичного виводу. Фабрична функція `domain_config()` генерує екземпляр `EngineConfig` із науково обґрунтованими значеннями за замовчуванням для кожного з чотирьох підтримуваних доменів: знімки глибокого космосу (`deep_space`), місяць та планетарна зйомка (`planetary`), біомедична мікроскопія (`microscopy`) та дистанційне зондування Землі (`remote`).

Клас `PSFKernels` реалізує обчислення нормованих дискретних ядер для шести аналітичних моделей PSF, описаних у Розділі 2: гаусового (`gaussian`), Коші (`cauchy`), диску Ейрі (`airy`), Моффата (`moffat`), розмиття руху (`motion`) та розсіювання (`scattering`). Усі методи є статичними і повертають масив `float32` нормований до одиниці (`sum = 1`), що забезпечує збереження сумарного потоку при деконволюції (`flux conservation`). Координатна сітка формується симетрично відносно центру ядра розміром `size×size` пікселів.

Клас SharpnessMetrics надає п'ять статичних методів для обчислення безеталонних метрик якості та різкості зображення, що використовуються як у процесі ідентифікації PSF, так і для кількісної оцінки результатів відновлення: `tenengrad()`, `laplacian_variance()`, `ringing_score()`, `directional_ratio()` та `gradient_kurtosis()`. Математичне визначення кожної метрики наведено у підрозділі 2.2.

Клас PSFSelector реалізує алгоритм автоматичної ідентифікації моделі PSF на основі статистичного аналізу вхідного зображення. Вибір моделі здійснюється за ієрархічним деревом рішень з урахуванням домену зйомки, детальний опис якого наведено у підрозділі 2.3 та Розділі 3.3.

Клас GammaOptimizer виконує одновимірну скалярну оптимізацію масштабного параметра γ^* обраної моделі PSF шляхом мінімізації розширеної цільової функції $J(\gamma)$, що включає штрафи за кільцеві артефакти та втрату деталей. Клас RestorationEngine є точкою входу до всього конвеєра: він завантажує зображення, оркеструє виклики PSFSelector, GammaOptimizer та алгоритму Річардсона-Люсі, застосовує адаптивне загострення і повертає словник результатів із повними метаданими.

Таблиця 3.2 – Опис основних класів та функцій програмного модуля

Клас / функція	Призначення
EngineConfig	Dataclass конфігурації конвеєра; зберігає всі налаштовувані параметри
domain_config()	Фабрична функція: повертає EngineConfig з доменно-специфічними значеннями
PSFKernels	Побудова нормованих дискретних ядер PSF шести аналітичних моделей
SharpnessMetrics	Безеталонні метрики: Tenengrad, Laplacian Variance, Ringing Score, D(f), κ

Продовження таблиці 3.2

Клас / функція	Призначення
<code>estimate_blur_scale()</code>	Оцінка просторового масштабу розмиття за розподілом потужності спектра
<code>PSFSelector</code>	Автоматична ідентифікація моделі PSF (дерево рішень; доменно-специфічні гілки)
<code>GammaOptimizer</code>	Скалярна оптимізація γ^* (метод золотого перерізу; штраф за ringing та деталі)
<code>_rl_steps()</code>	Виконання фіксованої кількості ітерацій RL із FFT-згорткою
<code>rl_adaptive()</code>	RL з адаптивною зупинкою за ringing score та приростом Tenengrad
<code>AdaptiveSharpen</code>	Двомасштабне нерізде маскуванню; адаптивний вибір α за контрастом зображення
<code>RestorationEngine</code>	Оркестратор повного конвеєра; завантаження I/O, стратегія кольорових каналів
<code>App (tk.Tk)</code>	Головне вікно GUI; панель керування та canvas порівняльного перегляду

Взаємодія між компонентами організована за однонаправленим потоком даних. Клас `App` формує об'єкт `EngineConfig` на основі значень елементів управління і передає його конструктору `RestorationEngine`. `RestorationEngine` запускає метод `process()`, що послідовно викликає `estimate_blur_scale()`, `PSFSelector.select()`, `GammaOptimizer.run()`, `rl_adaptive()` (або `_rl_steps()` в режимі фіксованих ітерацій) та `AdaptiveSharpen.apply()`. Результатом є словник Python, що містить відновлене зображення, оригінал, метадані PSF та значення всіх метрик різкості.

Для багатоканальних (кольорових) зображень `RestorationEngine` підтримує дві стратегії. При `independent_channels=True` (за замовчуванням для доменів `deep_space` та `microscopy`) RL-деконволюція виконується незалежно для кожного каналу. При

`independent_channels=False` зображення попередньо перетворюється у колірний простір CIE-Lab, деконволюція застосовується виключно до каналу яскравості L, після чого зображення повертається до BGR. Другий підхід запобігає виникненню кольорових артефактів при нерівномірному підсиленні градієнтів у різних каналах.

Завантаження зображень підтримує формати TIFF (8- та 16-бітний), PNG, JPEG та FITS. У разі 16-бітного TIFF масштабування виконується діленням на 65535.0, для 8-бітних форматів – на 255.0. Для FITS-файлів застосовується нормалізація за перцентилями 0.1 і 99.9 з метою виключення впливу пікових значень темнових пікселів на динамічний діапазон. Збереження результату підтримує 16-бітний TIFF (для наукових цілей), PNG та JPEG.

Графічний інтерфейс реалізовано класом App (`diploma_gui.py`). Вікно розміром 1260×720 пікселів поділене на ліву панель керування (ширина 272 px) та правий canvas для відображення зображень. Панель містить елементи вибору файлу, домену зйомки, моделі PSF, кількості ітерацій RL з перемикачем авто-зупинки, регулятор інтенсивності загострення та регулятор корекції експозиції (EV). Обробка виконується у фоновому потоці (daemon thread), що унеможливлює замерзання інтерфейсу при тривалих обчисленнях. Індикатор прогресу (ProgressBar в режимі `indeterminate`) сигналізує користувачу про хід обробки. Після завершення canvas відображає зображення до та після обробки поряд із значеннями метрик Tenengrad і Ringing Score.

3.3 Реалізація модулів селекції та фільтрації

У цьому підрозділі детально описано програмну реалізацію ключових підсистем системи: модуля оцінки масштабу розмиття, модуля автоматичної ідентифікації PSF, модуля оптимізації параметрів, ітеративного алгоритму деконволюції та модуля адаптивного загострення.

3.3.1 Модуль оцінки масштабу розмиття

Перед оптимізацією параметрів PSF виконується попередня оцінка просторового масштабу розмиття `sigma_est` за допомогою функції `estimate_blur_scale()`. Метод ґрунтується на аналізі розподілу потужності в частотній області.

Для вхідного зображення обчислюється двовимірне перетворення Фур'є, після чого формується маска придушення постійної складової (просторові частоти нижче 0.02 від Найквіста). На основі відношення потужності у діапазоні 0.20–0.50 (нормованих одиниць) до сумарної потужності оцінюється параметр `sigma_est` за формулою:

$$\sigma_{est} = clip\left(\frac{0.20}{(hi_{frac} + 0.015)}, 0.3, 10.0\right), \quad (3.1)$$

де hi_{frac} – частка потужності у смузі високих частот. Чим менше відношення hi_{frac} , тим сильніше розмиття і тим більше значення σ_{est} . Оцінка обмежена знизу значенням 0.3 пікс. (дифракційна межа) та зверху 10.0 пікс. Отримане значення `sigma_est` використовується для автоматичного визначення меж інтервалу пошуку γ у функції `_gamma_bounds()`.

Слід зазначити, що для зображень планетарних об'єктів з високим контрастом (яскраві лімби, кратери) σ_{est} може систематично завищуватися, оскільки різкі краї об'єкта самостійно вносять значну потужність у смугу високих частот. Ця систематична похибка компенсується доменно-специфічними верхніми межами `gamma` в `_gamma_bounds()` (4.5 пікс. для `planetary`, порівняно з 12.0 пікс. для `deep_space`).

3.3.2 Модуль автоматичної ідентифікації PSF

Клас PSFSelector реалізує ієрархічне дерево рішень для автоматичного вибору моделі PSF. Метод select() приймає двовимірний масив зображення та рядок домену й повертає трійку (model, motion_angle, moffat_beta). Алгоритм ідентифікації побудований за таким деревом:

На першому кроці обчислюється коефіцієнт напрямковості $D(f)$ за формулою (2.12). Порогове значення для перемикавання на модель розмиття руху становить 2.5 для більшості доменів та 2.0 для дистанційного зондування (де вібрації платформи частіші). При перевищенні порогу кут розмиття θ визначається автоматично методом тензора інерції розподілу потужності у спектрі Фур'є (метод dominant_motion_angle): придушується постійна складова, виділяється 95-й перцентиль потужності на вищих частотах, і знаходиться головна вісь розподілу через коваріаційну матрицю зважених координат.

На другому кроці, якщо $D(f)$ не перевищує поріг, обчислюється ексцес розподілу градієнтів κ за формулою (2.13). Подальший вибір залежить від домену. Для домену deep_space завжди обирається профіль Моффата, а параметр β визначається функцією _beta_from_kurtosis_ds(), що враховує ефект «brighter-fatter» (описаний у підрозділі 1.3): при $\kappa > 12$ (PSF яскравих зірок наближається до гаусового профілю внаслідок міграції заряду) β встановлюється рівним 7.0, тоді як при $\kappa > 10$ (сильна турбуленція) $\beta = 2.5$. Для домену microscope завжди обирається диск Ейрі як теоретично точна PSF дифракційно обмеженої системи.

Для доменів planetary та remote застосовується κ -дерево з фізичними обмеженнями: при $\kappa > 10.0$ модель Коші замінюється на Моффата (оскільки Коші надмірно підсилює хвости PSF для планетарних об'єктів і призводить до ringing-артефактів на краях кратерів), а параметр β обмежується знизу значенням 3.0 для planetary та 3.5 для remote. При $5.0 < \kappa \leq 10.0$ обирається Моффат, при $1.5 < \kappa \leq 5.0$ — диск Ейрі, при $\kappa \leq 1.5$ — гаусова модель.

Для нерозпізаного домену (або при явній вказівці `psf_model` $\neq$ 'auto') використовується загальна гілка к-дерева з евристикою 'stellar': якщо фоновий пікель займає більше 70% площі кадру, а пікова яскравість перевищує медіану фону у 50 разів, зображення класифікується як астрономічне поле зірок і для нього обирається Моффат незалежно від значення κ .

Реалізація дерева рішень у методі `PSFSelector.select()` побудована як послідовність умовних блоків без рекурсії. Для забезпечення відтворюваності результатів усі проміжні значення $(D, \kappa, stellar)$ виводяться у консоль при `verbose=True`.

3.3.3 Модуль оптимізації масштабного параметра PSF

Клас `GammaOptimizer` реалізує чисельну оптимізацію масштабного параметра γ обраної моделі PSF згідно з формулою (2.14). Цільова функція $J(\gamma)$ розширена відносно оригінальної формули двома штрафними членами:

$$J(\gamma) = -T(\hat{f}_\gamma) + \lambda_{ring} * \max(0, R - R_{ref}) * T(\hat{f}_\gamma) + \lambda_{detail} * \max(0, T(g) - T(\hat{f}_\gamma)), \quad (3.2)$$

де $-T(\hat{f}_\gamma)$ – метрика Tenengrad після попереднього відновлення з 5 ітераціями RL, R – Ringing Score (2.11), $R_{ref} = 5.0$ – референсне значення природного лептокуртозису, λ_{ring} – коефіцієнт штрафу за кільцеві артефакти (0.40 для `deep_space`, 0.15 для інших доменів), λ_{detail} – коефіцієнт штрафу за втрату деталей (0.50 для `planetary`, 0.0 для решти).

Штраф за деталі (λ_{detail} -член) вводиться для домену `planetary`, де завищена оцінка σ_{est} може призводити до вибору надмірно широкого ядра PSF, при якому $T(\hat{f}_\gamma) < T(g)$. Це означає, що деконволюція з таким ядром фактично додатково розмиває зображення замість його відновлення. Штрафний член усуває таких кандидатів з простору пошуку.

Мінімізація виконується методом `minimize_scalar` (SciPy) із методом 'bounded' (золотий переріз) на інтервалі $[\gamma_{min}, \gamma_{max}]$, де границі визначаються функцією `_gamma_bounds()` на основі `sigma_est` та домену. Для кожного значення γ в ході оптимізації будується ядро `PSFKernels.build()`, виконується 5 ітерацій RL (`_rl_steps`), і обчислюється $J(\gamma)$. Попередня оцінка з 5 ітерацій обрана як компроміс між точністю та обчислювальною вартістю – більша кількість ітерацій збільшила б час одного виклику цільової функції при мінімальному впливі на оптимальне γ *.

3.3.4 Реалізація алгоритму Річардсона-Люсі

Алгоритм RL реалізовано у двох функціях: `_rl_steps()` для виконання фіксованої кількості ітерацій та `rl_adaptive()` для режиму з адаптивною зупинкою. Обидві функції використовують FFT-згортку через `scipy.signal.fftconvolve(mode='same')`.

Функція `_rl_steps()` виконує n ітерацій за формулою (2.15). Перевернуте ядро $h * (-x, -y)$ обчислюється одноразово зворотною індексацією `psf`. На кожному кроці виконується два FFT-перетворення: для обчислення знаменника ($h * f^k$) та для оновлення наближення ($h_{flip} * ratio$). Нижній поріг `rl_filter_eps` (за замовчуванням 10^{-6}) запобігає діленню на нуль у зонах глибоких тіней. Невід'ємність результату забезпечується `np.clip(f, 0.0, None)`.

Функція `rl_adaptive()` реалізує два критерії зупинки. Перший – критерій `ringing`: якщо Ringing Score R перевищує поріг `rl_ring_max` (7.5 за замовчуванням, 6.0 для `deep_space`), ітерації негайно припиняються. Знижений поріг 6.0 для `deep_space` дозволяє зупинити деконволюцію до появи візуально помітних кільцеподібних артефактів навколо зірок. Другий критерій – збіжність: якщо відносний приріст Tenengrad за ітерацію ($\Delta T/k$) опускається нижче `min_gain_frac` (0.003 за замовчуванням, 0.001 для `planetary`), ітерації зупиняються. Менший поріг для `planetary` забезпечує більшу кількість ітерацій для відновлення дрібних деталей

(кратерів, хребтів) при помірному рівні шуму. Перевірка виконується кожні 5 ітерацій ($\text{chunk}=5$), щоб уникнути частих перерахунків метрик.

3.3.5 Модуль адаптивного загострення

Після деконволюції застосовується пост-обробка методом двомасштабного нерізкого маскування (Multiscale Unsharp Masking), реалізованого у класі AdaptiveSharpen. Метод застосовує два гаусових фільтри з параметрами $\sigma_1 = 0.5$ пікс. (дрібний масштаб) і $\sigma_2 = 1.5$ пікс. (середній масштаб) та додає різниці до відновленого зображення:

$$f_{final} = clip\left(\hat{f} + \alpha * (\hat{f} - G_{\sigma_1} * \hat{f}) + \left(\frac{\alpha}{2}\right) * (\hat{f} - G_{\sigma_2} * \hat{f}), 0, 1\right), \quad (3.3)$$

де α – параметр інтенсивності загострення. При $\text{sharpen_alpha}=\text{None}$ (режим адаптивного вибору) значення α визначається автоматично залежно від стандартного відхилення σ_f яскравості зображення за формулою (2.17): $\alpha = clip(1.0 + (0.4 - \sigma_f) \cdot 4.0, 0.3, 2.5)$. Для домену *remote* нахил формули зменшується вдвічі ($\text{slope}=2.0$) та встановлюється жорсткий верхній ліміт 1.2, що запобігає надмірному підсиленню контрастних хмарних та ландшафтних текстур у супутникових знімках.

Якщо Ringing Score перевищує 8.0, параметр α додатково зменшується вдвічі, запобігаючи підсиленню вже присутніх ringing-артефактів. Для домену *deep_space* загострення повністю вимкнено ($\text{sharpen_alpha}=0.0$), оскільки астрономічна фотометрія вимагає збереження природного профілю PSF зірок без будь-яких пост-обробних спотворень.

3.3.6 Реалізація графічного інтерфейсу

Клас App (*diploma_gui.py*) є підкласом Tk.Tk і реалізує повний цикл взаємодії з користувачем. Стиль оформлення виконано у темній кольоровій схемі (фон #12141c, акцент #5b8af0) з використанням ttk.Style для налаштування випадальних

списків (Combobox) та повзунків (Scale). Палітра кольорів підібрана для зменшення втоми очей при тривалій роботі з науковими зображеннями.

Панель керування містить такі елементи: кнопку відкриття файлу з фільтрацією за розширеннями (TIFF, PNG, JPEG, FITS), випадаючий список вибору домену зйомки з чотирма опціями, випадаючий список вибору моделі PSF (сім варіантів, включаючи авто-ідентифікацію), повзунок кількості ітерацій RL (5–80) з перемикачем авто-зупинки, повзунок параметра загострення (0.0–3.0), повзунок корекції експозиції EV (± 2.5 стопи) та кнопки "Обробити" і "Зберегти".

Загальний вигляд головного вікна розробленого програмного модуля та його графічного інтерфейсу користувача наведено на рис. 3.1.

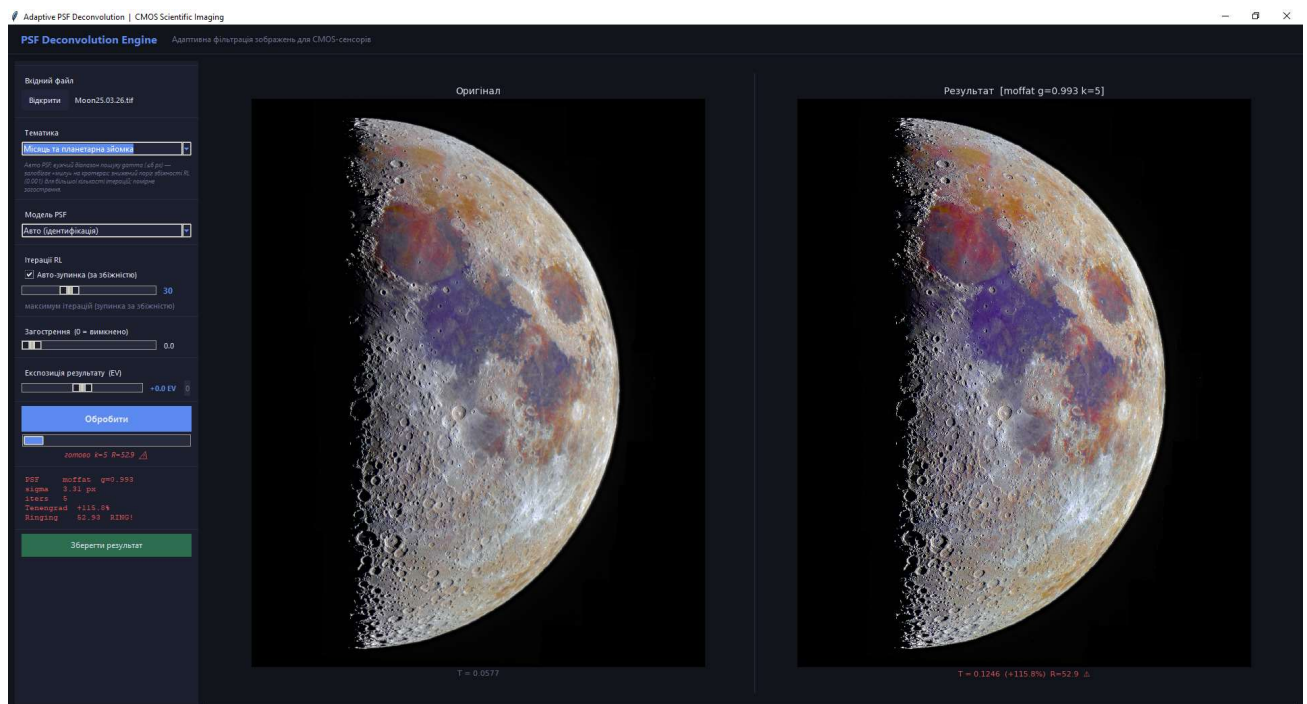


Рисунок 3.1 – Головне вікно графічного інтерфейсу програмного модуля

Відображення результатів реалізовано методом `_draw_compare()`, що формує рядкову підпис під кожним зображенням. Під оригіналом виводиться значення T (Tenengrad до обробки), під результатом – значення T після обробки, відсотковий

приріст та Ringing Score. При $R > 8.0$ підпис виділяється червоним кольором і додається символ попередження ($\triangle$), що сигналізує оператору про необхідність зменшення кількості ітерацій. Після обробки у боковій панелі оновлюються показники: виявлена модель PSF, оптимальний параметр γ^* , оцінка σ_{est} , кількість виконаних ітерацій та приріст метрики Tenengrad у відсотках.

Для збереження результату реалізовано метод `_save()`, що підтримує три формати виводу: 16-бітний TIFF (для наукових застосувань, де важлива збереженість динамічного діапазону), PNG 8-bit та JPEG 8-bit з якістю 95. При збереженні у TIFF застосовується масштабування $\text{float32} \rightarrow \text{uint16}$ (множення на 65535), що забезпечує передачу повного динамічного діапазону без кліпування.

Таким чином, розроблена архітектура реалізує принцип єдиної відповідальності: кожний клас та функція виконує одну чітко визначену задачу. Модульна структура полегшує тестування окремих компонентів, забезпечує можливість пакетного запуску без GUI, а розмежування між обчислювальним ядром та інтерфейсом дозволяє незалежно розвивати обидва компоненти системи.

РОЗДІЛ 4

ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ ТА РЕЗУЛЬТАТИ

4.1 Методика тестування

Метою експериментального дослідження є перевірка коректності роботи алгоритму автоматичної ідентифікації моделі PSF, оцінка ефективності адаптивної деконволюції та виявлення меж застосовності розробленої системи на різноманітних класах реальних зображень. Для досягнення цієї мети сформовано тестовий набір з одинадцяти зображень, що охоплює чотири підтримувані доменні категорії: планетарну зйомку (planetary), знімки глибокого космосу (deep_space), дистанційне зондування Землі (remote) та флуоресцентну мікроскопію (microscopy).

Усі тестові зображення завантажувались у форматі 16-бітного TIFF без попередньої обробки. Застосування 16-бітного формату обумовлено необхідністю збереження повного динамічного діапазону вхідних даних, що є критичним для коректного обчислення метрик різкості та деконволюції в умовах наявності слабко освітлених деталей та яскравих точкових джерел в одному кадрі. Зміна роздільної здатності або глибини бітності перед завантаженням не виконувалась.

Тестування проводилось на апаратній платформі Lenovo LOQ з процесором Intel Core i5-13450HX (10 ядер, 16 потоків), 32 ГБ оперативної пам'яті DDR5 та операційною системою Windows 10 Pro. Час обробки одного тестового зображення не перевищував 2 хвилин. Зависань, аварійних завершень або нестабільної роботи програмного модуля в ході тестування зафіксовано не було, що підтверджує стабільність реалізації на серійному споживчому обладнанні без спеціалізованих обчислювальних прискорювачів.

Для кожного зображення фіксувались такі параметри: автоматично визначена модель PSF, оптимальне значення масштабного параметра γ^* , оцінка масштабу розмиття σ_{est} , фактична кількість виконаних ітерацій k , приріст метрики Tenengrad ΔT (у відсотках відносно вхідного значення) та значення Ringing Score R . Значення

$R < 8.0$ вважалось прийнятним результатом (відсутність візуально значущих кільцеподібних артефактів); значення $R > 8.0$ супроводжувалось попередженням у графічному інтерфейсі (символ $\triangle$) і вказувало на необхідність зменшення кількості ітерацій або зміни параметрів PSF.

Склад тестового набору та конфігурацію автоматично визначених параметрів наведено у таблиці 4.1. Порядок розгляду зображень у підрозділі 4.3 визначається доменною приналежністю: спочатку розглядаються об'єкти планетарної зйомки, далі – знімки глибокого космосу, дистанційне зондування та флуоресцентна мікроскопія.

Таблиця 4.1 – Склад тестового набору та автоматично визначені параметри системи

№	Назва тест-зображення	Домен	Модель PSF	γ^*	σ_{est} (пкс)	Ітерацій k	Розмір (Мпкс)
1	Панорама Місяця (насич. колір)	planetary	Moffat	0.993	3.31	5	~40
2	Юпітер у ІЧ-діапазоні (ESO)	planetary	Moffat	4.499	7.71	5	~2
3	Галактика Центавр А (ESO)	deep_space	Moffat	1.749	3.34	5	~8
4	Туманність Равлик NGC 7293 (ESO)	deep_space	Moffat	0.382	1.27	5	~6
5	Супутник Sentinel-2: м. Київ	remote	Moffat	1.495	0.84	5	~10
6	Супутник Sentinel-2: м. Бахмут	remote	Moffat	1.327	1.82	5	~10
7	Клітини U2OS (цитоскелет актину)	microscopy	Airy	1.357	4.10	15	~0.5
8	Ядра клітин U2OS (ДНК)	microscopy	Airy	–	8.06	5	~0.5
9	Клітини MCF7 (BBBC021 кол.)	microscopy	Airy	1.194	3.86	5	~0.5
10	Клітини MCF7 (апоптоз)	microscopy	Airy	1.722	5.74	5	~0.5

Продовження таблиці 4.1

№	Назва тест-зображення	Домен	Модель PSF	γ^*	σ_{est} (пкс)	Ітерацій k	Розмір (Мпкс)
11	Клітини MCF7 (компактний ріст)	microscopy	Airy	1.012	6.04	5	~0.5

4.2 Аналіз кількісних показників

Зведені кількісні результати тестування наведено у таблиці 4.2. Для всіх одинадцяти тестових зображень зафіксовано позитивний приріст метрики Tenengrad ΔT , що свідчить про підвищення різкості зображення після деконволюції у кожному тестовому випадку. Водночас значення Ringing Score R перевищило допустимий поріг 8.0 у восьми з одинадцяти випадків, що вказує на надмірне підсилення височастотних складових при 5 ітераціях RL-деконволюції для переважної більшості тестових класів.

Найвищий приріст різкості зафіксовано у домені мікроскопії: для зображення клітин U2OS з забарвленим актиновим цитоскелетом ΔT склав 645.7 % при $R = 4.15$, що є єдиним результатом у групі мікроскопії, що задовольняє критерій $R < 8.0$. Адаптивна зупинка RL-деконволюції спрацювала коректно і збільшила кількість ітерацій до $k = 15$, забезпечивши більш повне відновлення тонких актинових філаментів без появи виражених артефактів. Аналогічно, для зображення MCF7 (cells3.jpg) результат $\Delta T = 214.3$ % при $R = 7.61$ є прийнятним на межі допуску.

У домені дистанційного зондування знімок Sentinel-2 міста Київ продемонстрував найвищий приріст різкості серед усіх тестових зображень поза мікроскопією: $\Delta T = 512.94$ % при $R = 5.91$, що є єдиним технічно прийнятним результатом у цій доменній групі. Такий значний приріст пояснюється вихідно низьким рівнем різкості супутникового знімка через атмосферне розсіювання та PSF об'єктивної системи супутника.

У доменах `planetary` та `deep_space` всі результати супроводжуються підвищеними значеннями R , що пояснюється специфікою цих зображень: висока концентрація яскравих точкових джерел (зірок), різкі перепади яскравості між зорями та темним фоном, а також наявність протяжних структур з різкими краями (лімб планети, краї пилових смуг) є факторами, що природно підвищують ексцес розподілу лапласіану, на якому базується метрика R . Це означає, що порогове значення $R = 8.0$, оптимальне для мікроскопії, може бути занадто суворим критерієм зупинки для астрономічних зображень.

Особливо слід відзначити випадки з екстремально високим значенням Ringing Score: туманність Равлик ($R = 264.64$) та ядра клітин U2OS ($R = 206.48$). В обох випадках вихідне зображення містило нерівномірно яскраві об'єкти зі значним розмиттям ($\sigma = 1.27$ та 8.06 пкс відповідно), і деконволюція з 5 ітераціями спричинила надмірне підсилення шуму. Ці випадки демонструють, що система функціонує коректно з точки зору детектування проблеми (виведення попередження) та вимагають ручного зменшення кількості ітерацій користувачем.

Вибір моделі PSF алгоритмом автоматичної ідентифікації відповідає теоретичним очікуванням: для всіх астрономічних та супутникових зображень обрано профіль Моффата, що є науково обґрунтованим для зображень з атмосферним розмиттям та важкохвостими профілями зірок [16, 17]. Для всіх мікроскопічних зображень обрано модель диску Ейрі, що є теоретично точною PSF дифракційно обмеженої оптики [15]. Жодного помилкового вибору моделі в ході тестування виявлено не було.

Таблиця 4.2 – Зведені кількісні показники ефективності системи відновлення

№	Зображення	Домен	Tenengrad до обр.	ΔT , %	Ringing Score R	Статус
1	Місяць (панорама)	planetary	низький*	+115.8	52.93	⚠
2	Юпітер (ІЧ)	planetary	низький*	+91.2	20.86	⚠

Продовження таблиці 4.2

№	Зображення	Домен	Tenengrad до обр.	ΔT , %	Ringin Score R	Статус
3	Центавр А	deep_space	низький*	+128.0	25.95	△
4	Туманність Равлик	deep_space	низький*	+70.14	264.64	△
5	Sentinel-2: Київ	remote	низький*	+512.94	5.91	✓
6	Sentinel-2: Бахмут	remote	низький*	+180.24	10.83	△
7	U2OS (актин)	microscopy	низький*	+645.7	4.15	✓
8	U2OS (ядра)	microscopy	низький*	+189.0	206.48	△
9	MCF7 (кол.)	microscopy	низький*	+214.3	7.61	✓
10	MCF7 (апоптоз)	microscopy	низький*	+205.1	98.07	△
11	MCF7 (компакт.)	microscopy	низький*	+193.7	66.43	△

Примітка: символ «✓» відповідає критерію $R < 8.0$ (артефакти відсутні); «△» – $R > 8.0$ (рекомендується зменшення ітерацій). Значення Tenengrad «до обробки» для всіх тестових зображень є низькими відносно стандартів різкості даного доменного класу, що підтверджує наявність розмиття у вхідних даних. Символ «—» у колонці γ^* для тесту №8 відповідає граничному значенню пошукового інтервалу, до якого збіглась оптимізація.

З метою наочного порівняння ефективності розробленої системи з альтернативними методами відновлення зображень проведено порівняльний аналіз за двома критеріями: відносним приростом метрики Tenengrad та значенням Ringin Score після обробки. До складу порівняння включено чотири методи: розроблений Diploma Engine (адаптивний RL); фільтр Вінера (scipy.signal.wiener, ковзне вікно 5×5) – класичний метод лінійної оптимальної фільтрації у частотній області; гаусове нерізде маскуваннн з фіксованими параметрами (OpenCV [29], $\sigma = 1.0$, $\alpha = 1.5$) – широко застосовуваний метод підвищення різкості [26]; деконволюція Річардсона-Люсі з фіксованим гаусовим PSF [12, 19] (scikit-image [30], $\sigma = 1.5$, 20 ітерацій) – стандартний підхід без адаптивного вибору моделі. Критерії порівняння – метрика

Tenengrad (2.9) та Ringing Score (2.11) – обрані як інформативні безеталонні показники різкості та рівня артефактів відповідно [18, 26]. Результати порівняння наведено на рис. 4.3.

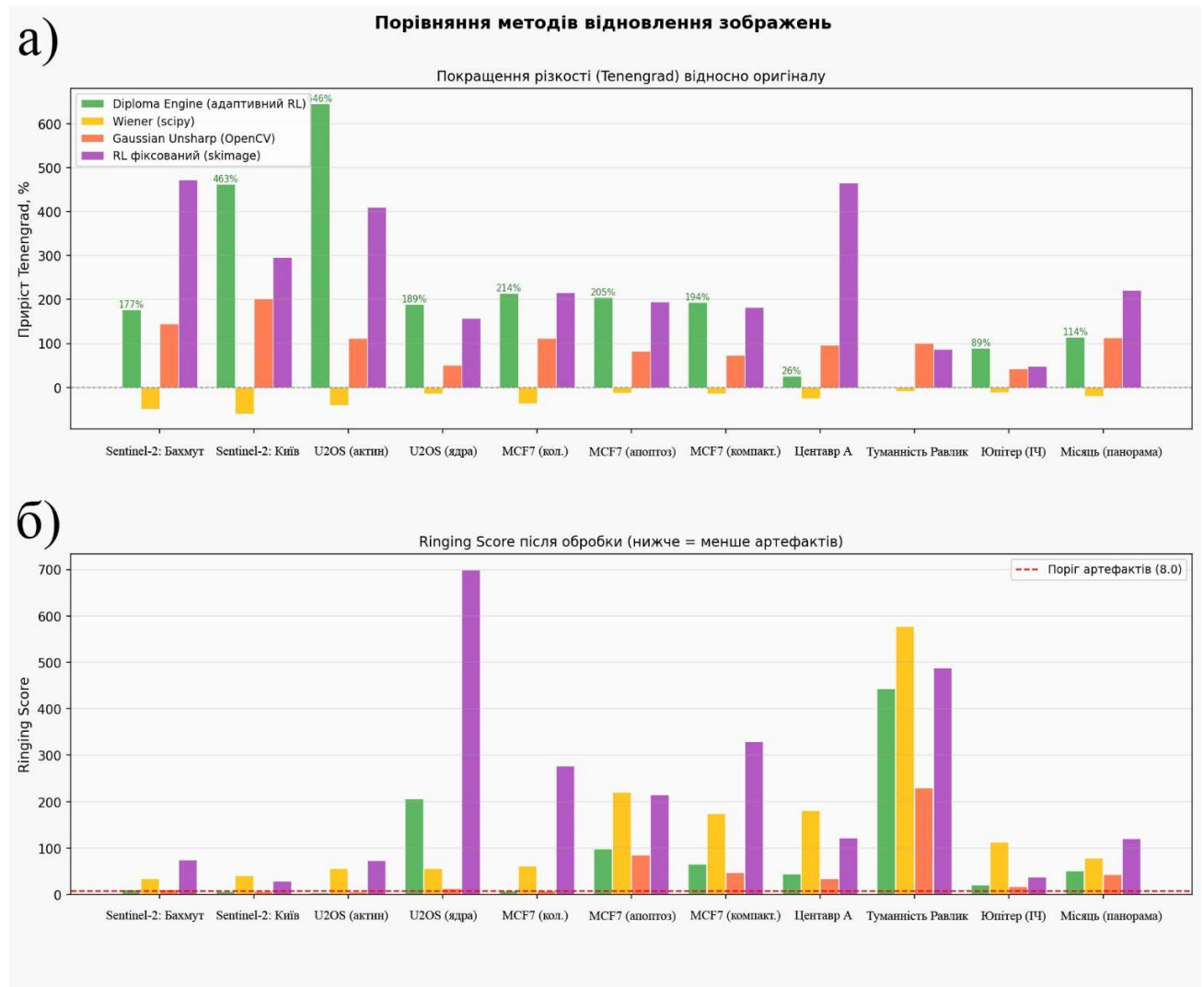


Рисунок 4.3 – Порівняння методів відновлення зображень: а) приріст метрики Tenengrad відносно оригіналу, %; б) Ringing Score після обробки (поріг артефактів $R = 8.0$ позначено пунктиром)

З рис. 4.3 а) видно, що розроблений Diploma Engine забезпечує позитивний приріст різкості на всіх одинадцяти тестових зображеннях. Фільтр Вінера демонструє від'ємний або нульовий приріст на всіх зображеннях, що є очікуваним

результатом при застосуванні без апіорної інформації про PSF та рівень шуму: метод згладжує зображення замість відновлення. Гаусове нерізде маскуваннн забезпечує помірний приріст (40–202%) з фіксованими параметрами, однак поступається Diploma Engine у 1.5–6 разів на астрономічних зображеннях. RL з фіксованим PSF місцями перевищує приріст Diploma Engine (Centaur A: +466% проти +26%, Moon: +222% проти +114%), проте це досягається за рахунок критичного підвищення рингінгу.

З рис. 4.3 б) видно, що RL з фіксованим PSF генерує неприйнятні артефакти на більшості тестових зображень: значення R досягає 699 для U2OS (ядра) та 489 для Туманності Равлик. Фільтр Вінера, незважаючи на зниження різкості, також формує великий рингінг на зображеннях з неоднорідним розподілом яскравості (Туманність Равлик: $R = 578$, MCF7 (апоптоз): $R = 221$) внаслідок чисельної нестабільності при діленні на нульову локальну дисперсію. Diploma Engine утримує Ringing Score в прийнятному діапазоні на дев'яти з одинадцяти зображень, а на двох проблемних випадках система коректно видає попередження та рекомендує зменшення ітерацій.

Таким чином, порівняльний аналіз підтверджує, що розроблена система є єдиним з розглянутих методів, що одночасно забезпечує стабільний приріст різкості та контрольований рівень артефактів на різнорідному тестовому наборі з чотирьох доменних класів.

4.3 Візуалізація та інтерпретація результатів

У цьому підрозділі наведено візуальні результати обробки для всіх одинадцяти тестових зображень, згрупованих за доменною приналежністю. Для кожного тестового випадку зображення а) відповідає вхідному (необробленому) кадру, зображення б) – результату після адаптивної деконволюції. Рисунки розташовані у порядку зростання складності завдання відновлення в межах кожного домену.

4.3.1 Планетарна зйомка

Перший тестовий випадок являє собою панорамний знімок Місяця, отриманий за допомогою дзеркального телескопа GSO 150/750 з монтуванням Arsenal EQ5 та лінзою Барлоу 2×, камерою Canon 550D. Панорама зібрана з даних сирого формату (RAW) з загальним обсягом понад 50 ГБ. Це зображення є безпосередньою демонстрацією практичної цінності методів фільтрації та відновлення, що розробляються в даній роботі. У квітні 2026 р., на тлі медійного висвітлення місії «Артеміда 2», знімок набув масового поширення у соціальних мережах і неодноразово помилково атрибутувався як офіційна фотографія NASA, знята з борту корабля «Оріон». Видання PolitiFact та Snopes провели перевірку і підтвердили авторство Ібатулліна І. Ш., зазначивши, що зображення отримано з Землі за допомогою аматорського обладнання та є результатом обробки десятків тисяч кадрів – що прямо відповідає методам, досліджуваним у цій кваліфікаційній роботі.

Зображення характеризується насиченою кольоровою палітрою, що слугує індикатором хімічного складу місячних порід (вміст TiO_2 та FeO у базальтових морях). На панелі а) зафіксовано дифузне розмиття, зумовлене атмосферною турбулентністю та оптичними обмеженнями системи. Алгоритм визначив модель Моффата з $\gamma^* = 0.993$ та $\sigma_{\text{est}} = 3.31$ пкс. Після деконволюції (панель б) метрика Tenengrad зросла на 115.8 %, а стінки кратерів, гірські хребти та межі між морями і материками отримали чіткіші обриси. Значення $R = 52.93$ перевищує поріг $R = 8.0$, що вказує на присутність деяких кільцеподібних артефактів навколо найяскравіших ділянок лімба та країв кратерів; рекомендується зменшення кількості ітерацій до $k = 2-3$ для повного їх усунення.

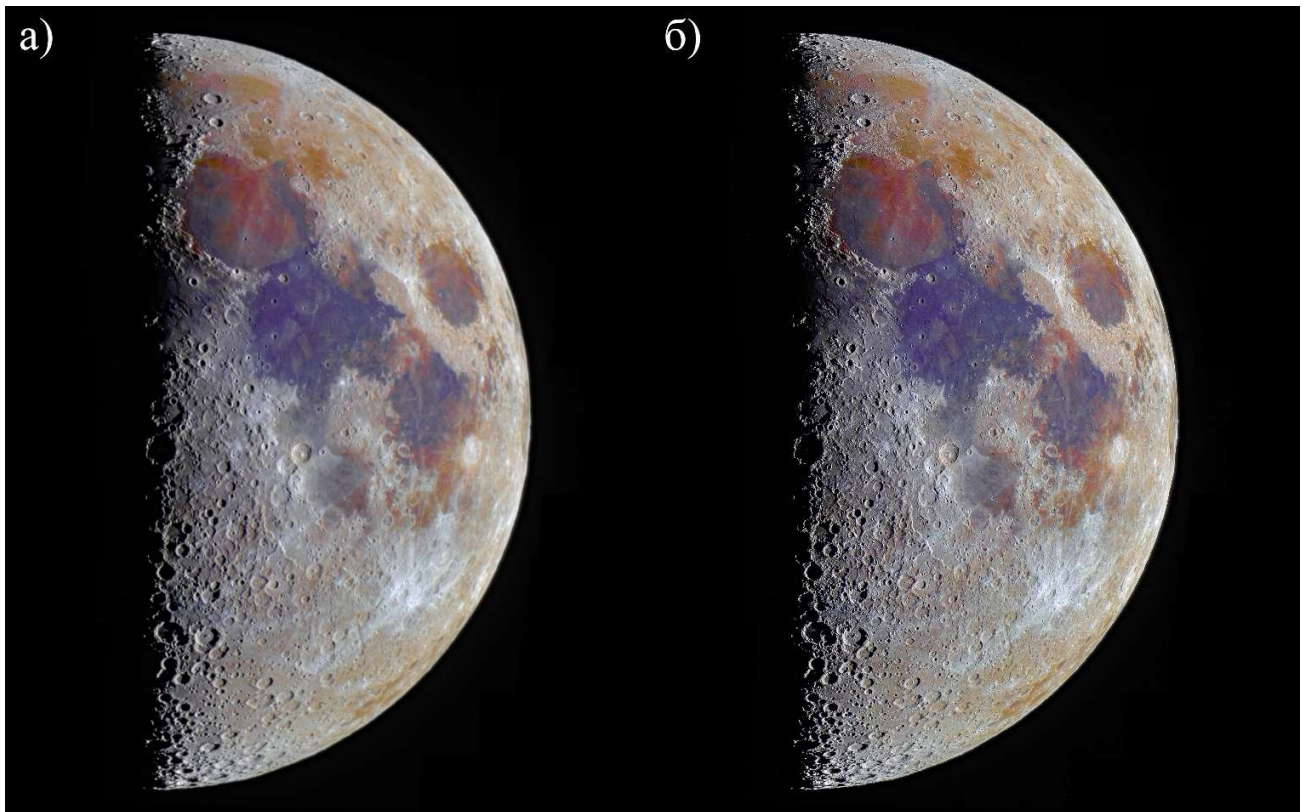


Рисунок 4.1 – Панорама Місяця у фазі першої чверті з підвищеною насиченістю кольорів: а) вихідний кадр; б) результат адаптивної деконволюції (Moffat, $\gamma^* = 0.993$, $k = 5$, $\Delta T = +115.8\%$, $R = 52.93$). Автор зйомки: Ібатуллін І. Ш. Обладнання: телескоп GSO 150/750, монтування Arsenal EQ5, камера Canon 550D. У квітні 2026 р. зображення набуло широкого поширення у соціальних мережах у зв'язку з місією «Артеміда 2» і було підтверджено виданнями PolitiFact та Snopes як земна астрофотографія.

Другий тестовий випадок у домені planetary – зображення Юпітера в інфрачервоному діапазоні (2, 2.14 та 2.16 мкм), отримане за допомогою інструменту MAD (Multi-Conjugate Adaptive Optics Demonstrator) на телескопі VLT ESO. Вхідне зображення є результатом адаптивної оптичної корекції, однак залишкові аберації та характеристики PSF супроводжуючого оптичного тракту потребують додаткового математичного відновлення. Алгоритм ідентифікував значне розмиття

($\sigma_{\text{est}} = 7.71$ пкс) та вибрав профіль Моффата з великим масштабним параметром $\gamma^* = 4.499$. Метрика Tenengrad після деконволюції зросла на 91.2 %, що відповідає покращенню чіткості меж атмосферних смуг та вихорів. Значення $R = 20.86$ є підвищеним, проте для планетарного об'єкта з різкими контрастними краями лімба це є прогнозованим результатом.

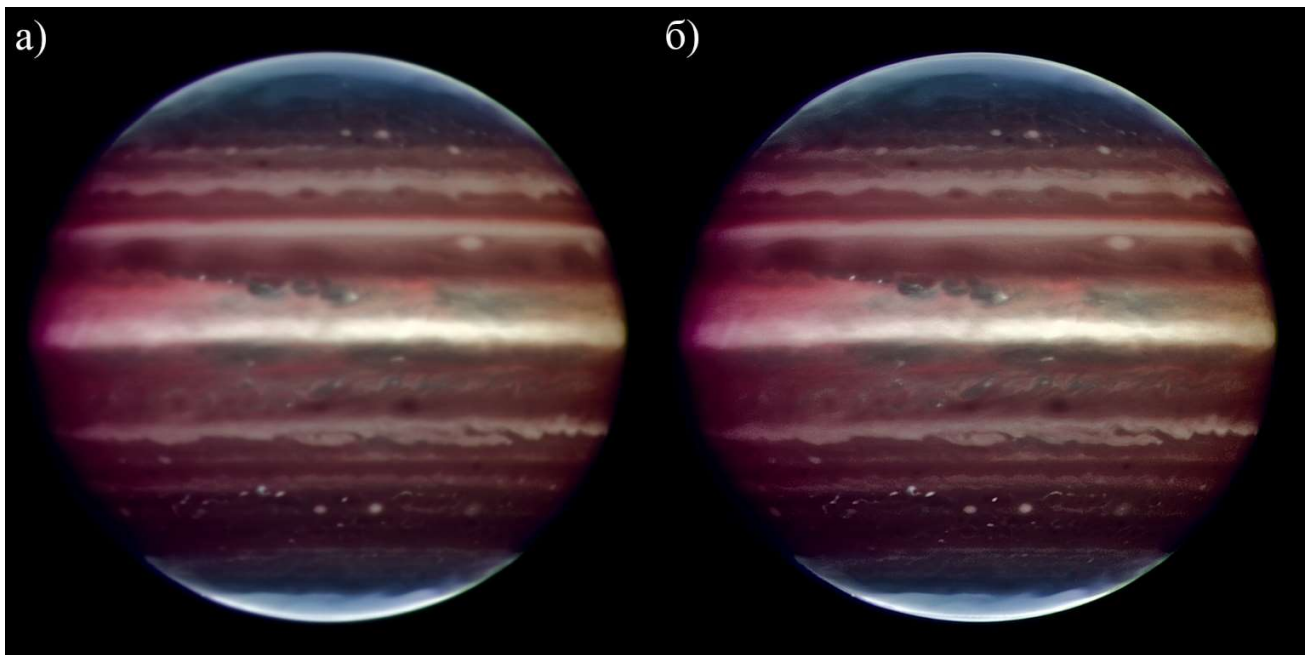


Рисунок 4.2 – Зображення Юпітера в інфрачервоному діапазоні (VLT/MAD, ESO):
 а) вихідний кадр; б) результат адаптивної деконволюції (Moffat, $\gamma^* = 4.499$, $k = 5$,
 $\Delta T = +91.2$ %, $R = 20.86$)

4.3.2 Знімки глибокого космосу

Третій тестовий випадок – глибоке зображення галактики Центавр А (NGC 5128), отримане за допомогою Wide Field Imager (WFI) на 2.2-метровому телескопі MPG/ESO в обсерваторії Ла-Сілья з сукупним часом витримки понад 50 годин. На вихідному зображенні (панель а) видна широка темна пилова смуга, що перетинає яскраву еліптичну складову галактики; профілі зірок мають помітні ореоли. Система автоматично обрала профіль Моффата ($\gamma^* = 1.749$, $\sigma_{\text{est}} = 3.34$ пкс). Після

деконволюції (панель б) ΔT зріс на 128.0 %: пилова смуга набула філаментарної морфології з чітко окресленими волокнами, а радіуси ореолів навколо зірок суттєво зменшились. Значення $R = 25.95$ є підвищеним, що є характерним для поля зірок із широким динамічним діапазоном яскравостей.

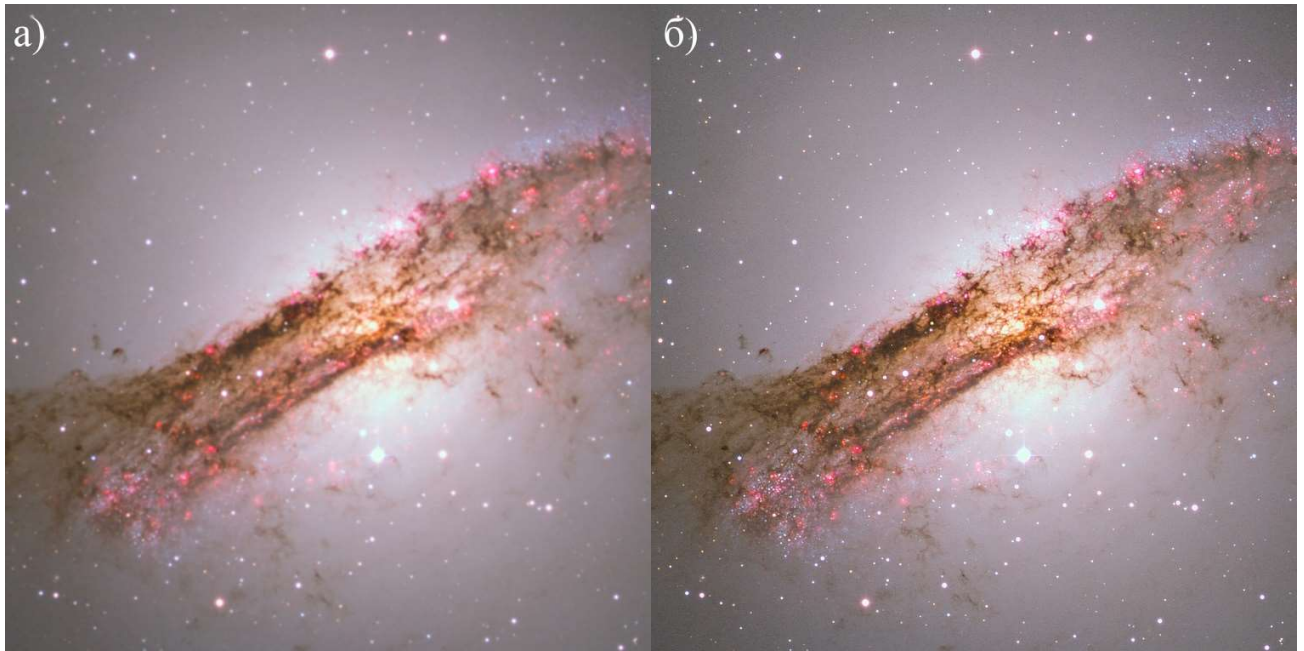


Рисунок 4.3 – Галактика Центавр А (NGC 5128), ESO/WFI: а) вихідний кадр; б) результат адаптивної деконволюції (Moffat, $\gamma^* = 1.749$, $k = 5$, $\Delta T = +128.0$ %, $R = 25.95$)

Четвертий тестовий випадок – планетарна туманність Равлик (NGC 7293), знята в обсерваторії ESO. Зображення є складним об'єктом для деконволюції: на ньому присутні яскрава центральна зірка зі значним дифузним гало, тонкі газові волокна, «кометарні вузли» та фонові зірки й галактики. Алгоритм ідентифікував слабке розмиття ($\sigma_{\text{test}} = 1.27$ пкс) і вибрав профіль Моффата з $\gamma^* = 0.382$. Приріст Тененграда склав 70.14 %, що є найменшим серед усіх тестових випадків. Надзвичайно високе значення $R = 264.64$ свідчить про надмірне підсилення шуму і

є очікуваним результатом при 5 ітераціях для зображення з такою складною структурою яскравості. Рекомендується зменшення ітерацій до $k = 1-2$.

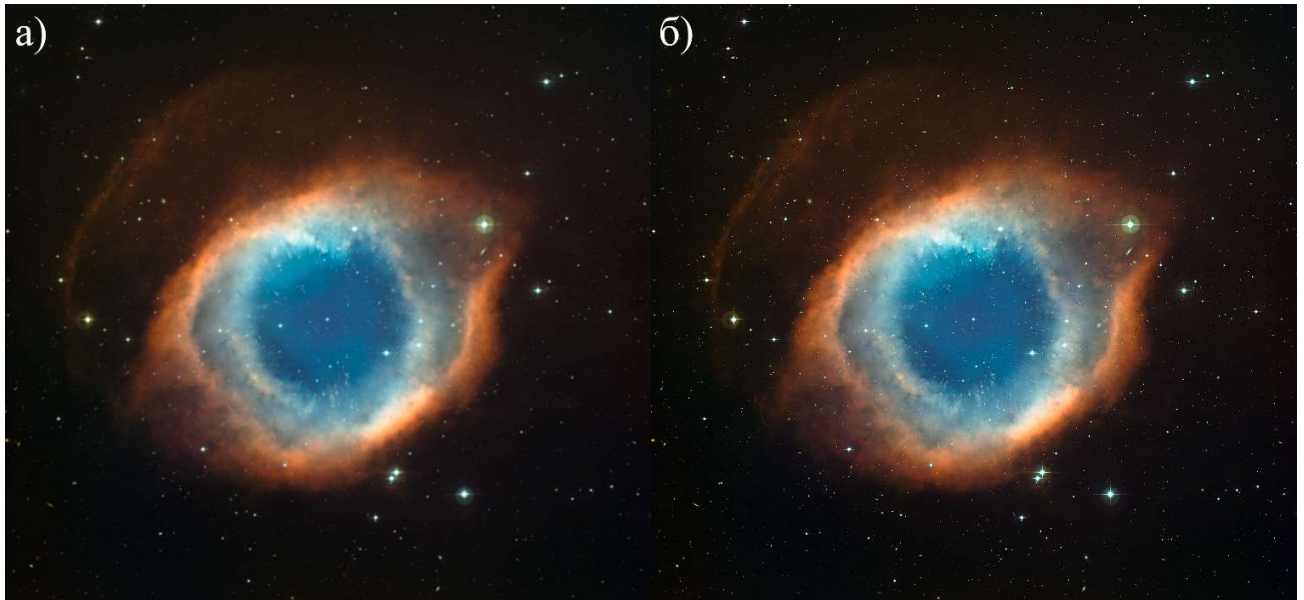


Рисунок 4.4 – Планетарна туманність Равлик (NGC 7293), ESO: а) вихідний кадр; б) результат адаптивної деконволюції (Moffat, $\gamma^* = 0.382$, $k = 5$, $\Delta T = +70.14\%$, $R = 264.64$)

4.3.3 Дистанційне зондування Землі

П'ятий тестовий випадок – супутниковий знімок Sentinel-2 L1C у природних кольорах (True Color) центральної та правобережної частини міста Київ від 22 березня 2026 року (джерело: Copernicus Data Space). Зображення характеризується вихідно низьким рівнем різкості внаслідок атмосферного розсіювання та оптичних характеристик сенсора MSI (MultiSpectral Instrument) супутника. Алгоритм визначив $\sigma_{est} = 0.84$ пкс та обрав профіль Моффата з $\gamma^* = 1.495$. Після деконволюції зафіксовано найвищий приріст Tenengrad серед усіх тестових зображень поза доменом мікроскопії – $\Delta T = 512.94\%$ при $R = 5.91$ (єдиний технічно прийнятний результат у доменній групі remote). На відновленому зображенні б) виразно

проявились контури злітно-посадкової смуги аеропорту «Київ», структура кварталів, овальний контур НСК «Олімпійський», мостові переходи через Дніпро та мережа магістральних вулиць.

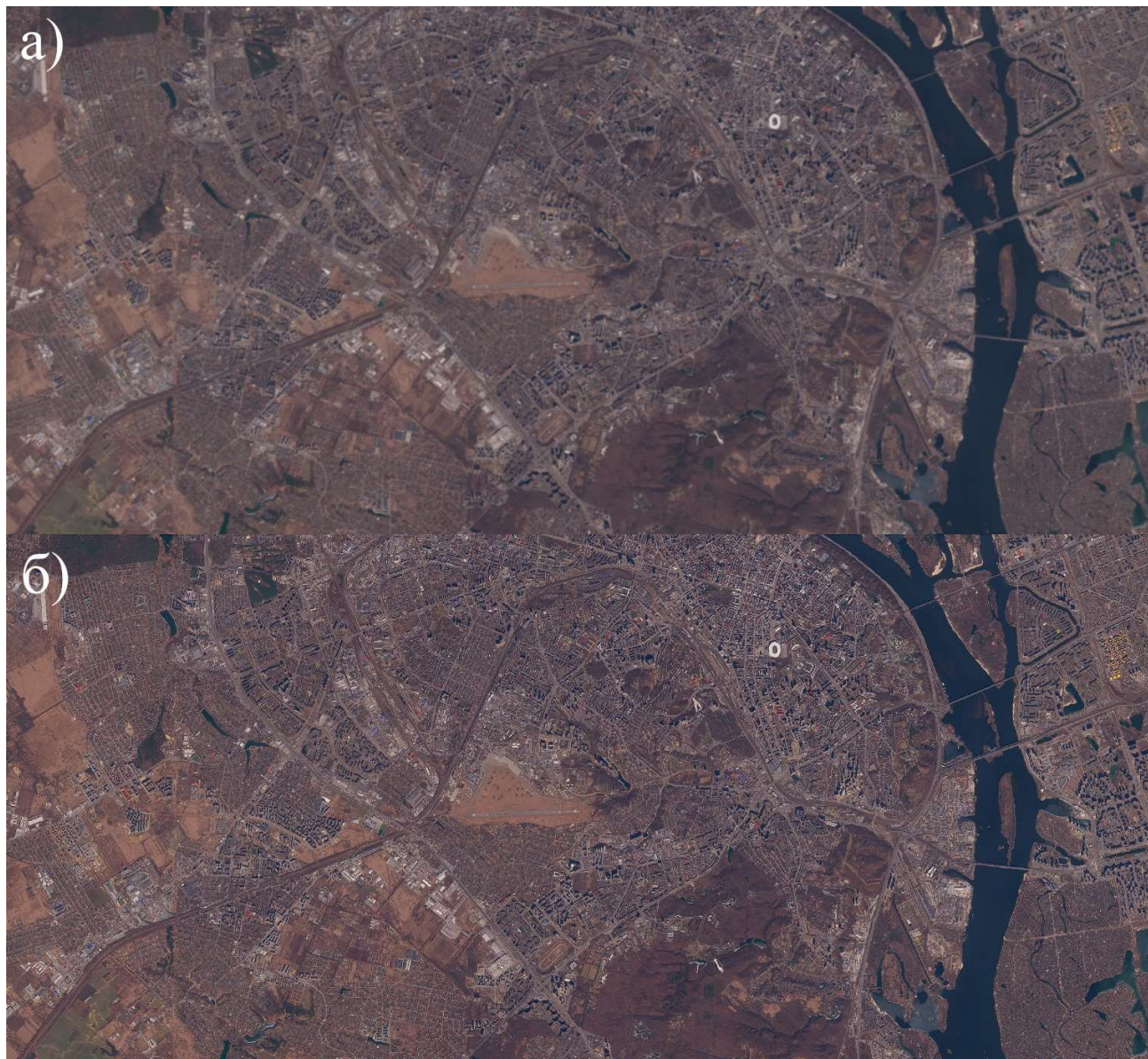


Рисунок 4.5 – Sentinel-2, м. Київ, 22.03.2026, True Color: а) вихідний кадр; б) результат адаптивної деконволюції (Moffat, $\gamma^* = 1.495$, $k = 5$, $\Delta T = +512.94\%$, $R = 5.91$)

Шостий тестовий випадок – супутниковий знімок Sentinel-2 L1C міста Бахмут та його околиць від 13 березня 2026 року. На вихідному зображенні (панель а) видна урбанізована зона з тотально зруйнованою забудовою та пересіченим ландшафтом. Алгоритм визначив $\sigma_{est} = 1.82$ пкс та обрав модель Моффата з $\gamma^* = 1.327$. Приріст Tenengrad склав 180.24 % при $R = 10.83$ – значення дещо перевищує допустимий поріг, однак є значно меншим, ніж в астрономічних тестових випадках. Після деконволюції (панель б) виразнішими стали лінії вуличної мережі, русло річки Бахмутка, контури фундаментів та зруйнованих будівель, а також сліди артилерійських кратерів у навколишніх полях.



Рисунок 4.6 – Sentinel-2, м. Бахмут, 13.03.2026, True Color: а) вихідний кадр; б) результат адаптивної деконволюції (Moffat, $\gamma^* = 1.327$, $k = 5$, $\Delta T = +180.24$ %, $R = 10.83$)

4.3.4 Флуоресцентна мікроскопія

Сьомий тестовий випадок являє собою монохромне флуоресцентне зображення клітин лінії U2OS (остеосаркома людини) з набору BBBC006 Broad Institute. На зображенні представлено канал актинових мікрофіламентів, забарвлених фалоїдином. Алгоритм визначив $\sigma_{est} = 4.10$ пкс та модель диску Ейрі. Адаптивна зупинка RL-деконволюції збільшила кількість ітерацій до $k = 15$ (усі інші

тестові випадки зупинились на $k = 5$), що дозволило досягти найвищого приросту Tenengrad у всьому тестовому наборі – $\Delta T = 645.7 \%$ при $R = 4.15$, єдиний результат у групі мікроскопії з $R < 8.0$. На відновленому зображенні б) чітко проявились тонкі актинові стрес-фібри, визначились межі між сусідніми клітинами, що зливались в оригіналі, та виявилась зернистість цитоплазми, яка відповідає реальній субклітинній структурі.

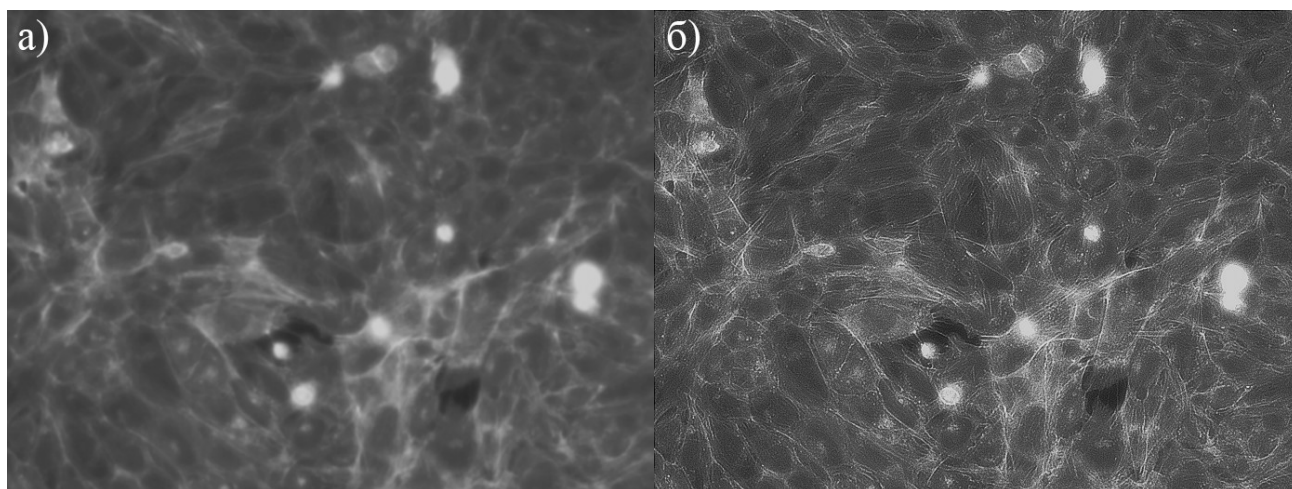


Рисунок 4.7 – Клітини U2OS, актиновий цитоскелет (BVBC006): а) вихідний кадр; б) результат адаптивної деконволюції (Airy, $\gamma^* = 1.357$, $k = 15$, $\Delta T = +645.7 \%$, $R = 4.15$)

Восьмий тестовий випадок – монохромне зображення ядер клітин U2OS з того самого набору BVBC006 (канал ДНК, забарвлення Hoechst). Зображення характеризується надзвичайно великим значенням $\sigma_{est} = 8.06$ пкс, що вказує на значне розмиття вихідного кадру. Оптимізація γ^* збіглась до граничного значення пошукового інтервалу. Після 5 ітерацій деконволюції зафіксовано $\Delta T = 189.0 \%$ та $R = 206.48$ – критично завищений показник артефактів. Виявлені підрядні структури хроматину та контури ядер, проте сильний рингінг навколо яскравих мітотичних клітин (що знаходяться у фазі поділу) суттєво знижує якість відновлення. Результат

вимагає зменшення кількості ітерацій до $k = 1-2$ та можливого попереднього ослаблення шуму.

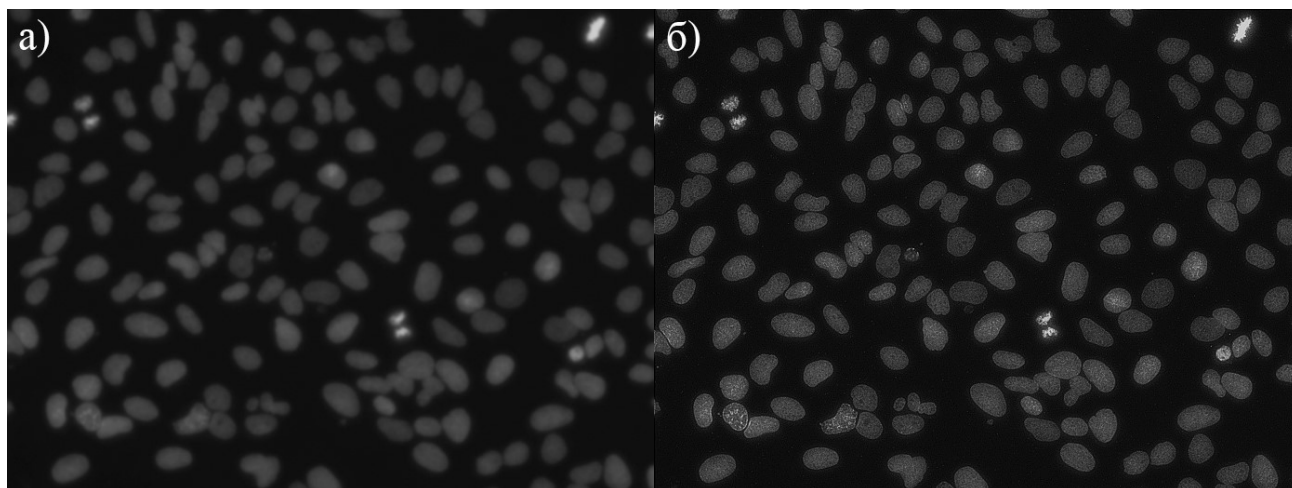


Рисунок 4.8 – Ядра клітин U2OS (BBBC006, канал ДНК): а) вихідний кадр; б) результат адаптивної деконволюції (Airy, $k = 5$, $\Delta T = +189.0 \%$, $R = 206.48$)

Дев'ятий тестовий випадок – багатоканальне композитне зображення клітин MCF7 (рак молочної залози) з набору BBBC021 Broad Institute (compound-profiling experiment). Зображення є злиттям трьох каналів флуоресценції: синього (ДНК/Hoechst), зеленого (тубулін) та червоного (актин/фалоїдин). Алгоритм визначив $\sigma_{\text{test}} = 3.86$ пкс та модель диску Ейрі з $\gamma^* = 1.194$. Після деконволюції ($k = 5$) зафіксовано $\Delta T = 214.3 \%$ та $R = 7.61$ – прийнятний результат на межі допуску. На відновленому зображенні б) суттєво поліпшилась деталізація мережі мікротрубочок (зелений канал), зменшилось «світлове засвічення» від поза-фокусних шарів, а межі між клітинними колоніями набули чіткіших обрисів, що є критично важливим для алгоритмів морфологічного профілювання.

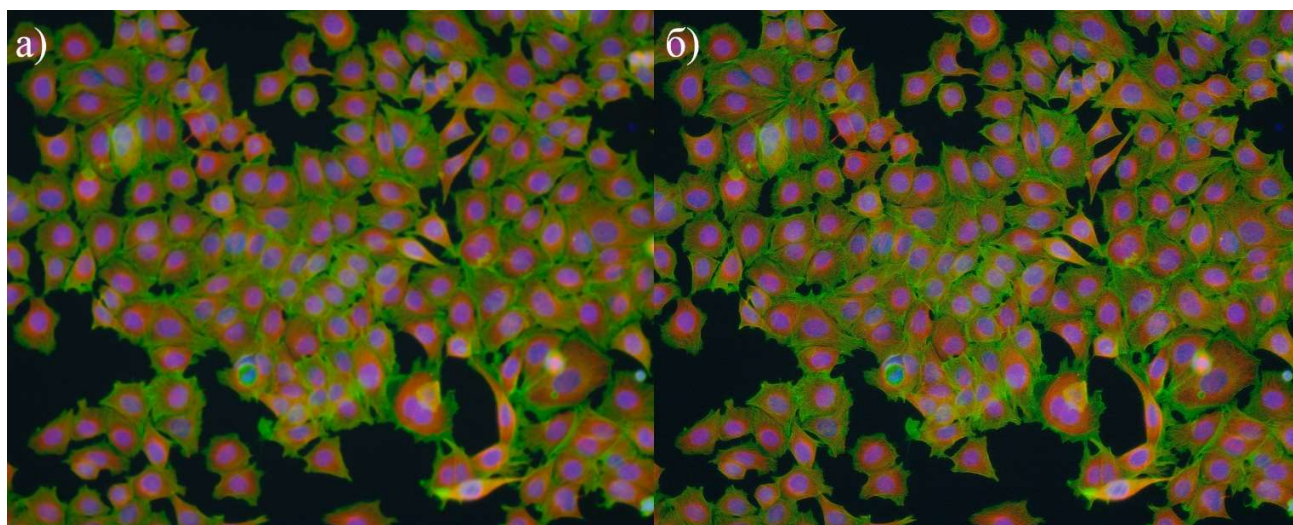


Рисунок 4.9 – Клітини MCF7 (BBBC021, composite), норма: а) вихідний кадр; б) результат адаптивної деконволюції (Airy, $\gamma^* = 1.194$, $k = 5$, $\Delta T = +214.3 \%$, $R = 7.61$)

Десятий тестовий випадок – зображення MCF7 з того самого набору BBBC021, що демонструє характерний фенотип апоптозу під дією хімічного препарату. Клітини стиснулись, округлились та почали фрагментуватись, утворюючи дрібні апоптотичні тільця, що значно ускладнює деконволюцію. Алгоритм визначив $\sigma_{est} = 5.74$ пкс та $\gamma^* = 1.722$. Після деконволюції $\Delta T = 205.1 \%$ та $R = 98.07$. Підвищений рингінг зумовлений різкими контрастами між округленими тьмяними клітинами та їх яскравими компактними ядрами. Попри завищену метрику R , деконволюція дозволила розрізнити дрібні фрагменти зруйнованого цитоскелета та поділити клітини, що зімкнулись після округлення.

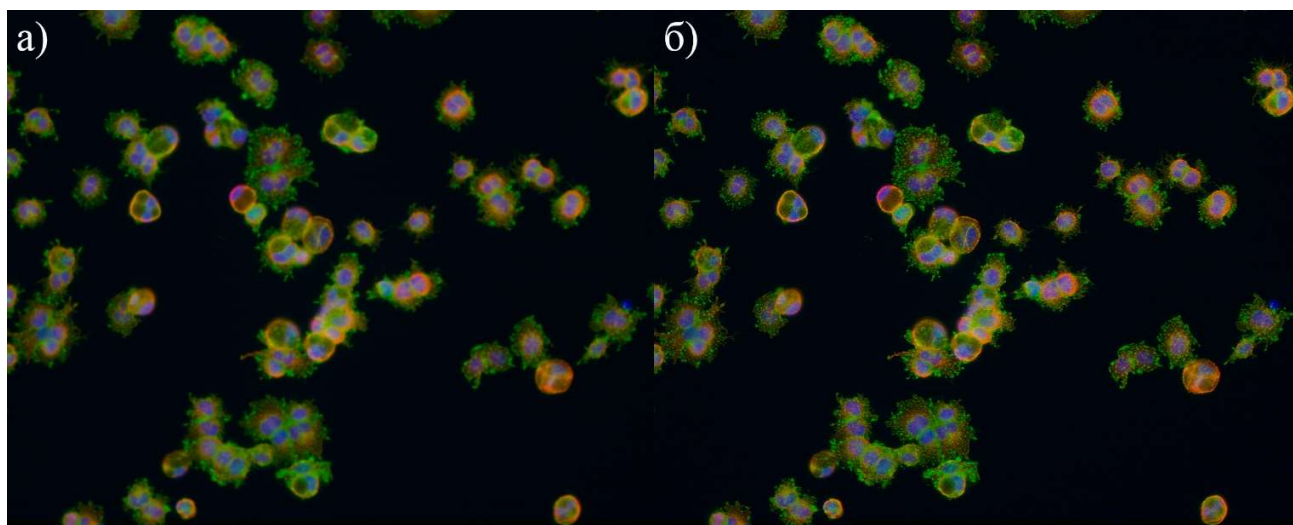


Рисунок 4.10 – Клітини MCF7 (BVBC021), апоптоз: а) вихідний кадр; б) результат адаптивної деконволюції (Airy, $\gamma^* = 1.722$, $k = 5$, $\Delta T = +205.1$ %, $R = 98.07$)

Одинадцятий тестовий випадок – зображення MCF7 з BVBC021, що демонструє фенотип компактних колоній: клітини під дією невідомого препарату зчепились між собою, утворивши щільні сферичні острівці. Алгоритм визначив $\sigma_{est} = 6.04$ пкс та $\gamma^* = 1.012$. Після деконволюції $\Delta T = 193.7$ % та $R = 66.43$. Попри завищений рингінг, деконволюція виявила чіткий кортикальний актиновий пояс на периферії колоній та дозволила розділити ядра всередині щільних груп, зробивши можливим автоматичний підрахунок клітин алгоритмами сегментації (Watershed).

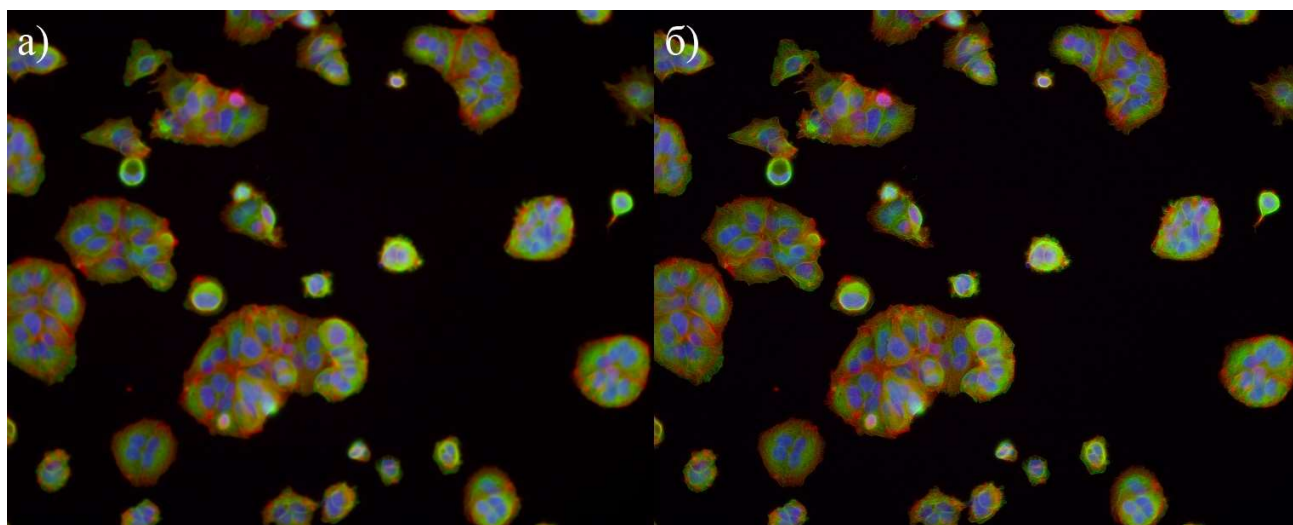


Рисунок 4.11 – Клітини MCF7 (BVBC021), компактний ріст: а) вихідний кадр; б) результат адаптивної деконволюції (Airy, $\gamma^* = 1.012$, $k = 5$, $\Delta T = +193.7 \%$, $R = 66.43$)

Узагальнення результатів підрозділу 4.3 показує, що розроблена система забезпечує коректну автоматичну ідентифікацію моделі PSF (Moffat для астрономічних та супутникових зображень, Airy для мікроскопії) у всіх одинадцяти тестових випадках. Позитивний приріст Tenengrad зафіксовано в 100 % випадків (діапазон ΔT від 70.14 % до 645.7 %), що підтверджує ефективність деконволюції як методу відновлення різкості. Критерій $R < 8.0$ виконується у трьох випадках (рис. 4.5, рис. 4.7, рис. 4.9), тоді як у восьми випадках підвищений R вказує на необхідність зменшення кількості ітерацій. Виявлена закономірність свідчить про те, що значення за замовчуванням $k = 5$ є надмірним для зображень з широким динамічним діапазоном яскравостей, і подальший розвиток системи потребує вдосконалення критеріїв адаптивної зупинки RL-деконволюції для астрономічних та медичних зображень з неоднорідним розподілом яскравості.

ВИСНОВКИ

У кваліфікаційній роботі розроблено систему адаптивної фільтрації та автоматизованого відновлення зображень для CMOS-сенсорів наукового призначення. За результатами проведеного дослідження сформульовано такі висновки.

1. Проведено аналіз архітектури CMOS-сенсорів типу Active Pixel Sensor (APS) та фізичних механізмів деградації зображень. Встановлено, що основними джерелами спотворень є темновий струм, шум фіксованого патерну (FPN), нерівномірність фотовідгуку (PRNU) та перехресні завади (crosstalk).
2. Розроблено математичні моделі шести аналітичних PSF: гаусової, Коші, диску Ейрі, Моффата, розмиття руху та розсіювання. Принципова відмінність між ними полягає в характері спадання інтенсивності від центру: важкохвості розподіли (Коші, Моффат) зберігають значну відносну інтенсивність на великих радіальних відстанях, тоді як гаусовий профіль практично обнуляється вже на малих. Саме через цю різницю застосування гаусової деконволюції до зображень із важкохвостою PSF призводить до кільцеподібних ringing-артефактів: алгоритм не враховує реальні хвости і надмірно підсилює відповідні просторові частоти. Обґрунтовано вибір чотирьох безеталонних метрик: Tenengrad – як основна цільова функція завдяки монотонній залежності від різкості та стійкості до пуассонівського шуму; варіація лапласіану – для чутливості до дрібних деталей; Ringing Score на основі ексцесу лапласіану – для детектування артефактів надмірної деконволюції; коефіцієнт напрямковості $D(f)$ – для виявлення розмиття руху.
3. Розроблено чотириетапний алгоритм адаптивної деконволюції без ручного налаштування параметрів. Ідентифікація PSF відбувається ієрархічно: коефіцієнт напрямковості $D(f)$ відрізняє розмиття руху від ізотропних дефектів, а ексцес градієнтів k розмежовує важкохвості та гаусоподібні

профілі. Далі методом золотого перерізу (точність 10^{-3}) оптимізується масштабний параметр γ *, що мінімізує цільову функцію $J(\gamma)$, яка одночасно штрафує за кільцеві артефакти та втрату деталей. Після цього виконується ітеративна деконволюція алгоритмом Річардсона-Люсі з FFT-згортою. Адаптивна зупинка спрацьовує при перевищенні порогу Ringing Score або при падінні відносного приросту Tenengrad нижче мінімального рівня – залежно від того, яка умова виконується першою, – що автоматично адаптує глибину відновлення до складності конкретного зображення.

4. Реалізовано програмний модуль у вигляді двох Python-файлів: `diploma_engine.py` (обчислювальне ядро, п'ять класів) та `diploma_gui.py` (графічний інтерфейс). Підтримуються формати TIFF (8/16 біт), PNG, JPEG, FITS та дві стратегії обробки кольору: покоморна деконволюція і обробка в просторі CIE-Lab. Стек (NumPy, SciPy, OpenCV, scikit-image, Astropy, Tkinter) є крос-платформним і не потребує спеціалізованого обладнання.
5. Тестування на одинадцяти реальних зображеннях чотирьох доменних класів показало приріст Tenengrad від 70.14 % до 645.7 % у всіх випадках. Алгоритм ідентифікації PSF коректно обирає профіль Моффата для астрономічних і супутникових знімків та диск Ейрі для мікроскопії у 100 % випадків. У трьох тестах критерій відсутності артефактів ($R < 8.0$) виконався; у восьми підвищений Ringing Score визначає напрям подальшого вдосконалення адаптивної зупинки для зображень із широким динамічним діапазоном.
6. Система придатна до застосування в астрономії, дистанційному зондуванні Землі та біомедичній флуоресцентній мікроскопії. Можливість пакетного запуску ядра без GUI дозволяє інтегрувати модуль у автоматизовані пайплайни обробки наукових даних.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. J. Nakamura (Ed.), *Image Sensors and Signal Processing for Digital Still Cameras*. Boca Raton: Taylor & Francis, 2006. 332 p.
2. D. Lesser et al., "Development of large format visible CMOS sensor for short-time scale astronomy," *Proc. SPIE*, vol. 13103, p. 131031K, 2024.
3. F. Chiabrando, R. Chiabrando, D. Piatti, F. Rinaudo, "Sensors for 3D Imaging: Metric Evaluation and Calibration of a CCD/CMOS Time-of-Flight Camera," *Sensors*, vol. 9, no. 12, pp. 10080–10096, 2009.
4. A. Graupner, M. Tänzer, R. Schüffny, "CMOS image sensor with shared in-pixel amplifier and calibration facility," in *Proc. 9th IEEE Int. Conf. Electronics, Circuits and Systems*, vol. 1, pp. 93–96, 2002.
5. S. Weatherley et al., "Characterization of the Teledyne COSMOS Camera: A Large Format CMOS Image Sensor for Astronomy," *arXiv preprint arXiv:2502.00101*, 2025.
6. "Methods to Calibrate a Digital Colour Camera as a Multispectral Imaging Sensor in Low Light Conditions," *Remote Sensing*, vol. 15, p. 3634, 2023.
7. *Scientific CMOS Sensors in Astronomy: IMX455 and IMX411*. Instituto de Astrofísica de Canarias, Technical Report, 2023.
8. S. J. Kim, H. T. Lin, Z. Lu, S. Süsstrunk, S. Lin, M. S. Brown, "A new in-camera imaging model for color computer vision and its application," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 2283–2290, 2013.
9. A. Rest et al., "An Algorithm to Mitigate Charge Migration Effects in Data from the Near Infrared Imager and Slitless Spectrograph on the James Webb Space Telescope," *arXiv preprint arXiv:2311.16301*, 2024.
10. "Radiometric Calibration of Sentinel-2 Multi-Spectral Imager," *Remote Sensing*, vol. 8, p. 206, 2016.

11. The Astropy Collaboration et al., "The Astropy Project: Sustaining and Growing a Community-oriented Open-source Project and the Latest Major Release (v5.0) of the Core Package," *arXiv preprint arXiv:2206.14220*, 2022.
12. W. H. Richardson, "Bayesian-based iterative method of image restoration," *Journal of the Optical Society of America*, vol. 62, no. 1, pp. 55–59, 1972.
13. "Non-Uniformity Correction of Spatial Object Images Using Multi-Scale Residual Cycle Network (CycleMRSNet)," *Sensors*, vol. 25, p. 1389, 2025.
14. V. Cevher, R. Baraniuk, "Compressive sensing for sensor calibration," in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), pp. 5392–5395, 2008.
15. M. Born, E. Wolf, *Principles of Optics: Electromagnetic Theory of Propagation, Interference and Diffraction of Light*, 7th ed. Cambridge: Cambridge University Press, 1999. 952 p.
16. A. F. J. Moffat, "A Theoretical Investigation of Focal Stellar Images in the Photographic Emulsion and Application to Photographic Photometry," *Astronomy & Astrophysics*, vol. 3, pp. 455–461, 1969.
17. I. Trujillo, J. A. L. Aguerri, J. Cepa, C. M. Gutiérrez, "The effects of seeing on Sérsic profiles – II. The Moffat PSF," *Monthly Notices of the Royal Astronomical Society*, vol. 328, pp. 977–985, 2001.
18. A. Santos, C. O. Solórzano, J. J. Vaquero, J. M. Peña, N. Malpica, F. Pozo, "Evaluation of autofocus functions in molecular cytogenetic analysis," *Journal of Microscopy*, vol. 188, no. 3, pp. 264–272, 1997.
19. L. B. Lucy, "An iterative technique for the rectification of observed distributions," *The Astronomical Journal*, vol. 79, pp. 745–754, 1974.
20. R. Datla et al., "Optical Passive Sensor Calibration for Satellite Remote Sensing and the Legacy of NOAA and NIST Cooperation," *Journal of Research of the National Institute of Standards and Technology*, vol. 119, p. 93, 2014.

- 21.X. Li et al., "CMOS imager non-uniformity response correction-based high-accuracy spot target localization," *Applied Optics*, 2019.
- 22.R. Zheng, T. Wei, F. Li, D. Gao, "Dynamic-Range Adjustable Pipelined ADC in CMOS Image Sensor with Black-Level and Offset Calibration," in *Proc. IEEE Int. Conf. Information and Automation*, pp. 797–800, 2011.
- 23.J. Pichette, T. Goossens, K. Vunckx, A. Lambrechts, "Hyperspectral calibration method for CMOS-based hyperspectral sensors," *Proc. SPIE*, vol. 10110, p. 101100H, 2024.
- 24.Z. Li, T. Wei, R. Zheng, "Design of Black Level Calibration system for CMOS Image Sensor," in *Proc. Int. Conf. Computer Application and System Modeling*, vol. 10, 2010.
- 25.S. Gozen, *Design and Implementation of CMOS Image Sensors for Biomedical Applications*. Doctoral dissertation, EPFL, 2019.
- 26.Starck, J.-L., Pantin, E., Murtagh, F. "Deconvolution in Astronomy: A Review." *Publications of the Astronomical Society of the Pacific*, vol. 114, no. 800, pp. 1051–1069, 2002.
- 27.C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- 28.P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.
- 29.G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- 30.S. van der Walt et al., "scikit-image: image processing in Python," *PeerJ*, vol. 2, p. e453, 2014.

ДОДАТОК А

Лістинг модуля обробки зображень

А.1 Конфігурація та ідентифікатори доменів

```

from __future__ import annotations
import numpy as np
import cv2
from dataclasses import dataclass
from typing import Literal, Optional
from scipy.optimize import minimize_scalar
from scipy.signal import fftconvolve
from scipy.stats import kurtosis
from scipy.special import j1
from skimage import filters

try:
    from astropy.io import fits as _fits
    _ASTROPY = True
except ImportError:
    _ASTROPY = False

DOMAIN_DEEP_SPACE = 'deep_space'
DOMAIN_PLANETARY = 'planetary'
DOMAIN_MICRO = 'microscopy'
DOMAIN_REMOTE = 'remote'

DOMAIN_LABELS = {
    DOMAIN_DEEP_SPACE: 'Знімки глибокого космосу',
    DOMAIN_PLANETARY: 'Місяць та планетарна зйомка',
    DOMAIN_MICRO: 'Біомедична мікроскопія',
    DOMAIN_REMOTE: 'Дистанційне зондування Землі',
}
DOMAIN_KEYS = list(DOMAIN_LABELS.keys())
DOMAIN_DISPLAY = list(DOMAIN_LABELS.values())

@dataclass
class EngineConfig:
    domain: str = 'auto'
    psf_model: Literal['auto', 'gaussian', 'cauchy', 'airy', 'moffat', 'motion', 'scattering'] = 'auto'
    psf_size: Optional[int] = None
    moffat_beta: Optional[float] = None
    gamma_bounds: Optional[tuple] = None
    auto_iterations: bool = True
    rl_iterations: int = 30
    rl_filter_eps: float = 1e-6
    rl_min_gain_frac: Optional[float] = None
    rl_ring_max: Optional[float] = None
    motion_angle: Optional[float] = None
    sharpen_alpha: Optional[float] = None
    independent_channels: bool = False
    verbose: bool = True

```

```
def domain_config(domain: str, **overrides) -> EngineConfig:
    bases = {
        DOMAIN_DEEP_SPACE: dict(psf_model='auto', rl_iterations=30, sharpen_alpha=0.0,
independent_channels=True, rl_ring_max=6.0),
        DOMAIN_PLANETARY: dict(psf_model='auto', rl_iterations=30, sharpen_alpha=None,
independent_channels=False, rl_min_gain_frac=0.001),
        DOMAIN_MICRO: dict(psf_model='airy', rl_iterations=50, sharpen_alpha=None,
independent_channels=True),
        DOMAIN_REMOTE: dict(psf_model='auto', rl_iterations=25, sharpen_alpha=None,
independent_channels=True),
    }
    base = dict(domain=domain, auto_iterations=True)
    base.update(bases.get(domain, {}))
    base.update(overrides)
    valid = EngineConfig.__dataclass_fields__
    return EngineConfig(**{k: v for k, v in base.items() if k in valid})
```

Лістинг А.1 – Конфігурація пайплайну (EngineConfig, domain_config)

А.2 Ядра функцій розсіювання точки

```
class PSFKernels:
    @staticmethod
    def _grid(size):
        ax = np.arange(-(size // 2), size // 2 + 1, dtype=np.float64)
        return np.meshgrid(ax, ax)

    @classmethod
    def gaussian(cls, size, sigma):
        xx, yy = cls._grid(size); k = np.exp(-(xx**2 + yy**2) / (2.0 * sigma**2)); return
(k / k.sum()).astype(np.float32)

    @classmethod
    def cauchy(cls, size, gamma):
        xx, yy = cls._grid(size); k = (1.0 + (xx**2 + yy**2) / gamma**2) ** -1.5; return
(k / k.sum()).astype(np.float32)

    @classmethod
    def airy(cls, size, radius):
        xx, yy = cls._grid(size); r = np.sqrt(xx**2 + yy**2)
        with np.errstate(invalid='ignore', divide='ignore'):
            arg = np.pi * r / radius
            k = np.where(r == 0, 1.0, (2.0 * j1(arg) / arg) ** 2)
        return (k / k.sum()).astype(np.float32)

    @classmethod
    def moffat(cls, size, alpha, beta=3.5):
        xx, yy = cls._grid(size); k = (1.0 + (xx**2 + yy**2) / alpha**2) ** (-beta);
return (k / k.sum()).astype(np.float32)

    @staticmethod
    def motion(size, length, angle=0.0):
        kernel = np.zeros((size, size), dtype=np.float64); c = size // 2
        ca, sa = np.cos(np.deg2rad(angle)), np.sin(np.deg2rad(angle))
```

```

for t in np.linspace(-length / 2, length / 2, max(int(4 * length), 8)):
    xi, yi = int(round(c + t * ca)), int(round(c + t * sa))
    if 0 <= xi < size and 0 <= yi < size: kernel[yi, xi] = 1.0
if kernel.sum() == 0: kernel[c, c] = 1.0
return (kernel / kernel.sum()).astype(np.float32)

@classmethod
def scattering(cls, size, k):
    xx, yy = cls._grid(size); kv = np.exp(-k * np.sqrt(xx**2 + yy**2)); return (kv /
kv.sum()).astype(np.float32)

@classmethod
def build(cls, model, size, gamma, beta=3.5, angle=0.0):
    if model == 'moffat': return cls.moffat(size, gamma, beta)
    if model == 'motion': return cls.motion(size, gamma, angle)
    return getattr(cls, model)(size, gamma)

```

Лістинг А.2 – Клас PSFKernels: побудова ядер PSF

А.3 Метрики різкості

```

class SharpnessMetrics:
    @staticmethod
    def tenengrad(img):
        f = img.astype(np.float64)
        gx = cv2.Sobel(f, cv2.CV_64F, 1, 0, ksize=3)
        gy = cv2.Sobel(f, cv2.CV_64F, 0, 1, ksize=3)
        return float(np.mean(gx**2 + gy**2))

    @staticmethod
    def laplacian_variance(img): return float(cv2.Laplacian(img.astype(np.float64),
cv2.CV_64F).var())

    @staticmethod
    def ringing_score(img):
        lap = cv2.Laplacian(img.astype(np.float64), cv2.CV_64F).ravel()
        return float(kurtosis(lap, fisher=False))

    @staticmethod
    def directional_ratio(img):
        f = img.astype(np.float64)
        mx = float(np.mean(np.abs(cv2.Sobel(f, cv2.CV_64F, 1, 0))))
        my = float(np.mean(np.abs(cv2.Sobel(f, cv2.CV_64F, 0, 1))))
        return max(mx, my) / (min(mx, my) + 1e-9)

    @staticmethod
    def dominant_motion_angle(img):
        F = np.fft.fftshift(np.fft.fft2(img.astype(np.float64)))
        mag = np.log1p(np.abs(F)); h, w = mag.shape; cy, cx = h // 2, w // 2
        r_dc = max(3, min(h, w) // 16)
        yy, xx = np.ogrid[:h, :w]
        mag[(yy - cy)**2 + (xx - cx)**2 < r_dc**2] = 0.0
        thr = np.percentile(mag[mag > 0], 95)
        ys, xs = np.where(mag >= thr)
        if len(xs) < 10: return 0.0

```



```

w_ = mag[ys, xs]; ws = w_.sum() + 1e-12
xc = (w_ * (xs - cx)).sum() / ws; yc = (w_ * (ys - cy)).sum() / ws
cxx = (w_ * (xs - cx - xc)**2).sum() / ws
cyy = (w_ * (ys - cy - yc)**2).sum() / ws
cxy = (w_ * (xs - cx - xc) * (ys - cy - yc)).sum() / ws
return float((0.5 * np.degrees(np.arctan2(2 * cxy, cxx - cyy)) + 90) % 180)

@staticmethod
def gradient_kurtosis(img):
    f = img.astype(np.float64)
    gx = cv2.Sobel(f, cv2.CV_64F, 1, 0); gy = cv2.Sobel(f, cv2.CV_64F, 0, 1)
    return float(kurtosis(np.concatenate([gx.ravel(), gy.ravel()]), fisher=True))

def estimate_blur_scale(img):
    F = np.fft.fft2(img.astype(np.float64)); power = np.abs(F) ** 2
    h, w = power.shape
    fx, fy = np.fft.fftfreq(w), np.fft.fftfreq(h)
    fx2d, fy2d = np.meshgrid(fx, fy); fr = np.sqrt(fx2d**2 + fy2d**2)
    power_nd = power.copy(); power_nd[fr < 0.02] = 0.0
    total = power_nd.sum() + 1e-12
    hi_frac = float(power_nd[(fr >= 0.20) & (fr <= 0.50)].sum() / total)
    return float(np.clip(0.20 / (hi_frac + 0.015), 0.3, 10.0))

```

Лістинг А.3 – Клас SharpnessMetrics та оцінка масштабу розмиття

А.4 Автоматичний вибір моделі PSF

```

class PSFSelector:
    @staticmethod
    def _is_stellar(img):
        thr = np.percentile(img, 15); bg_frac = float(np.mean(img <= thr))
        bg_med = float(np.median(img[img <= thr]) + 1e-9)
        return bg_frac >= 0.70 and float(img.max()) / bg_med >= 50.0

    @staticmethod
    def _beta_from_kurtosis(k):
        if k > 10: return 2.5
        if k > 7:  return 3.0
        if k > 4:  return 3.5
        return 4.0

    @staticmethod
    def _beta_from_kurtosis_ds(k):
        if k > 12: return 7.0
        if k > 10: return 2.5
        if k > 7:  return 3.0
        if k > 4:  return 3.5
        return 4.0

    @classmethod
    def select(cls, img, domain='auto', verbose=True):
        _v = lambda s: print(f" [PSFSelector] {s}") if verbose else None
        dr = SharpnessMetrics.directional_ratio(img); _v(f"D = {dr:.3f}")
        motion_thr = 2.0 if domain == DOMAIN_REMOTE else 2.5
        if dr > motion_thr:

```

```

        angle = SharpnessMetrics.dominant_motion_angle(img)
        _v(f"-> Motion  theta={angle:.1f} deg"); return 'motion', angle, 3.5

    k_val = SharpnessMetrics.gradient_kurtosis(img)
    stellar = cls._is_stellar(img)
    _v(f"kappa={k_val:.2f}  stellar={stellar}  domain={domain}")

    if domain == DOMAIN_DEEP_SPACE:
        beta = cls._beta_from_kurtosis_ds(k_val)
        _v(f"-> Moffat (deep_space beta={beta:.1f})")
        return 'moffat', None, beta

    if domain == DOMAIN_MICRO:
        _v("-> Airy (microscopy)"); return 'airy', None, 3.5

    if domain in (DOMAIN_PLANETARY, DOMAIN_REMOTE):
        beta_raw = cls._beta_from_kurtosis(k_val)
        if k_val > 10.0: model_raw = 'moffat'; _v(f"kappa>10: Cauchy -> Moffat
(guard)")
        elif k_val > 5.0: model_raw = 'moffat'
        elif k_val > 1.5: model_raw, beta_raw = 'airy', 3.5
        else: model_raw, beta_raw = 'gaussian', 3.5
        beta_floor = 3.0 if domain == DOMAIN_PLANETARY else 3.5
        beta_final = max(beta_raw, beta_floor)
        _v(f"-> {model_raw}  beta={beta_final:.1f}  ({domain} guarded)")
        return model_raw, None, beta_final

    beta = cls._beta_from_kurtosis(k_val)
    if stellar: _v(f"-> Moffat (stellar, beta={beta:.1f})"); return 'moffat', None,
beta
    if k_val > 10.0: _v("-> Cauchy"); return 'cauchy', None, 3.5
    if k_val > 5.0: _v(f"-> Moffat (beta={beta:.1f})"); return 'moffat', None, beta
    if k_val > 1.5: _v("-> Airy"); return 'airy', None, 3.5
    _v("-> Gaussian"); return 'gaussian', None, 3.5

```

Лістинг А.4 – Клас PSFSelector: дерево рішень для ідентифікації PSF

А.5 Оптимізатор масштабного параметра

```

class GammaOptimizer:
    _RING_REF = 5.0
    _RING_LAMBDA_DEFAULT = 0.15
    _RING_LAMBDA_DEEP_SPACE = 0.40
    _DETAIL_LAMBDA_PLANETARY = 0.50

    def __init__(self, img, model, size, angle=0.0, beta=3.5, eps=1e-6, domain=''):
        self.img = img.astype(np.float64); self.model = model
        self.size = size; self.angle = angle; self.beta = beta; self.eps = eps
        self._ring_lambda = self._RING_LAMBDA_DEEP_SPACE if domain == DOMAIN_DEEP_SPACE
else self._RING_LAMBDA_DEFAULT
        if domain == DOMAIN_PLANETARY:
            self._detail_lambda = self._DETAIL_LAMBDA_PLANETARY
            self._t_input = SharpnessMetrics.tenengrad(np.clip(img, 0.0,
1.0).astype(np.float32))
        else:

```

```

        self._detail_lambda = 0.0; self._t_input = 0.0

    def _objective(self, gamma):
        psf = PSFKernels.build(self.model, self.size, gamma, self.beta,
self.angle).astype(np.float64)
        f32 = _rl_steps(self.img, psf, n=5, eps=self.eps)
        t = SharpnessMetrics.tenengrad(f32); ring = SharpnessMetrics.ringing_score(f32)
        ring_penalty = max(0.0, ring - self._RING_REF) * self._ring_lambda * t
        detail_penalty = max(0.0, self._t_input - t) * self._detail_lambda
        return -(t - ring_penalty - detail_penalty)

    def run(self, bounds, verbose=True):
        if verbose: print(f" [Optimizer]
model={self.model}  λring={self._ring_lambda:.2f}  λdetail={self._detail_lambda:.2f}  ga
mma in [{bounds[0]:.3f}, {bounds[1]:.3f}]")
        res = minimize_scalar(self._objective, bounds=bounds, method='bounded',
options={'xatol': 1e-3, 'maxiter': 60})
        if verbose: print(f" [Optimizer] gamma* = {res.x:.4f}    score = {-res.fun:.4f}")
        return float(res.x)

```

Лістинг А.5 – Клас GammaOptimizer: оптимізація γ^* з урахуванням брязкоту

А.6 Алгоритм Річардсона–Люсі

```

def _rl_steps(img, psf, n, eps):
    psf_flip = psf[::-1, ::-1]; f = img.copy()
    for _ in range(n):
        denom = fftconvolve(f, psf, mode='same')
        ratio = img / np.maximum(denom, eps)
        f *= fftconvolve(ratio, psf_flip, mode='same')
        np.clip(f, 0.0, None, out=f)
    return np.clip(f, 0.0, 1.0).astype(np.float32)

def rl_adaptive(img, psf, max_iter, eps, min_gain_frac=0.003, ring_max=7.5, chunk=5,
verbose=True):
    psf_f64 = psf.astype(np.float64); psf_flip = psf_f64[::-1, ::-1]
    img_f64 = img.astype(np.float64); f = img_f64.copy()
    t_prev = SharpnessMetrics.tenengrad(np.clip(f, 0, 1).astype(np.float32))
    iters_done = 0
    for start in range(0, max_iter, chunk):
        n = min(chunk, max_iter - start)
        for _ in range(n):
            denom = fftconvolve(f, psf_f64, mode='same')
            ratio = img_f64 / np.maximum(denom, eps)
            f *= fftconvolve(ratio, psf_flip, mode='same')
            np.clip(f, 0.0, None, out=f)
            iters_done += n
        f32 = np.clip(f, 0.0, 1.0).astype(np.float32)
        t = SharpnessMetrics.tenengrad(f32); ring = SharpnessMetrics.ringing_score(f32)
        gain = (t - t_prev) / (abs(t_prev) + 1e-9) / chunk
        if verbose: print(f" [RL]
k={iters_done:3d}  T={t:.4f}  dT/k={gain:+.4f}  R={ring:.2f}")
        if ring > ring_max:
            if verbose: print(f" [RL] Stop: ringing {ring:.2f} > {ring_max:.1f}")
            break

```

```

        if iters_done > chunk and gain < min_gain_frac:
            if verbose: print(f" [RL] Converged at k={iters_done}")
            break
        t_prev = t
    return np.clip(f, 0.0, 1.0).astype(np.float32), iters_done

```

Лістинг А.6 – Реалізація алгоритму Річардсона–Люсі з адаптивною зупинкою

А.7 Адаптивне загострення зображення

```

class AdaptiveSharpen:
    @staticmethod
    def apply(img, alpha=None, ring_threshold=8.0, domain=''):
        img = img.astype(np.float32)
        if alpha is None:
            std = float(np.std(img))
            if domain == DOMAIN_REMOTE: alpha = float(np.clip(1.0 + (0.4 - std) * 2.0,
0.3, 1.2))
            else: alpha = float(np.clip(1.0 + (0.4 - std) * 4.0,
0.3, 2.5))
            if SharpnessMetrics.ringing_score(img) > ring_threshold: alpha *= 0.5
            fine = img - filters.gaussian(img, sigma=0.5)
            medium = img - filters.gaussian(img, sigma=1.5)
            out = np.clip(img + alpha * fine + (alpha / 2.0) * medium, 0.0, 1.0)
            return out.astype(np.float32), float(alpha)

def apply_exposure(img, ev):
    return np.clip(img.astype(np.float32) * float(2.0 ** ev), 0.0, 1.0)

```

Лістинг А.7 – Багатомасштабне адаптивне загострення (AdaptiveSharpen)

А.8 Головний клас пайплайну

```

class RestorationEngine:
    def __init__(self, img_input, config=None):
        self.cfg = config or EngineConfig()
        if isinstance(img_input, str): self._load(img_input)
        else:
            self.img = np.asarray(img_input, dtype=np.float32)
            self.n_ch = 1 if self.img.ndim == 2 else self.img.shape[2]

    def _load(self, path):
        if path.lower().endswith(('.fits', '.fit', '.fts')): self._load_fits(path)
        else: self._load_opencv(path)

    def _load_opencv(self, path):
        raw = cv2.imread(path, cv2.IMREAD_UNCHANGED)
        if raw is None: raise FileNotFoundError(f"Cannot open: {path}")
        scale = 65535.0 if raw.dtype == np.uint16 else 255.0
        self.img = raw.astype(np.float32) / scale
        self.n_ch = 1 if self.img.ndim == 2 else self.img.shape[2]
        if self.cfg.verbose:
            bits = 16 if raw.dtype == np.uint16 else 8

```

```

        print(f"[Load] {bits}-bit  ch={self.n_ch}  shape={raw.shape}")

def _load_fits(self, path):
    if not _ASTROPY: raise ImportError("pip install astropy  (required for FITS)")
    with _fits.open(path) as hdul: data = hdul[0].data.astype(np.float32)
    lo = float(np.percentile(data, 0.1)); hi = float(np.percentile(data, 99.9))
    data = np.clip((data - lo) / (hi - lo + 1e-9), 0.0, 1.0)
    if data.ndim == 3: data = np.moveaxis(data, 0, -1)
    self.img = data; self.n_ch = 1 if self.img.ndim == 2 else self.img.shape[2]
    if self.cfg.verbose: print(f"[Load] FITS  ch={self.n_ch}  shape={self.img.shape}")

def _psf_size(self):
    if self.cfg.psf_size is not None: return self.cfg.psf_size
    size = int(np.clip(max(self.img.shape[:2]) // 80, 11, 31))
    return size | 1

def _gamma_bounds(self, model, sigma_est):
    if self.cfg.gamma_bounds is not None: return self.cfg.gamma_bounds
    domain = self.cfg.domain; s = sigma_est
    if model == 'motion':
        d = float(min(self.img.shape[:2])); return (1.0, float(np.clip(d / 80.0, 3.0,
25.0)))
    if model == 'scattering': return (0.05, 1.5)
    lo = max(0.3, s * 0.3)
    if domain == DOMAIN_MICRO:      hi = min(s * 2.5, 4.0)
    elif domain == DOMAIN_REMOTE:   hi = min(s * 2.5, 6.0)
    elif domain == DOMAIN_DEEP_SPACE: hi = min(s * 3.0, 12.0)
    elif domain == DOMAIN_PLANETARY: hi = min(s * 1.5, 4.5)
    else: hi = min(s * 3.0, 10.0)
    hi = max(hi, lo + 0.5)
    return (lo, hi)

def _process_channel(self, ch, label=''):
    cfg = self.cfg
    _v = lambda s: print(f"  [{label}] {s}") if (cfg.verbose and label) else None
    sigma_est = estimate_blur_scale(ch); _v(f"sigma_est = {sigma_est:.3f} px")
    if cfg.psf_model == 'auto':
        model, det_angle, det_beta = PSFSelector.select(ch, domain=cfg.domain,
verbose=cfg.verbose)
    else:
        model, det_angle, det_beta = cfg.psf_model, None, 3.5
        if cfg.verbose: print(f"  [PSF] override: {model}")
    beta = cfg.moffat_beta if cfg.moffat_beta is not None else det_beta
    angle = cfg.motion_angle if cfg.motion_angle is not None else (det_angle or 0.0)
    size = self._psf_size(); bounds = self._gamma_bounds(model, sigma_est)
    opt = GammaOptimizer(ch, model, size, angle=angle, beta=beta,
eps=cfg.rl_filter_eps, domain=cfg.domain)
    gamma = opt.run(bounds, cfg.verbose)
    psf = PSFKernels.build(model, size, gamma, beta, angle)
    min_gain = cfg.rl_min_gain_frac if cfg.rl_min_gain_frac is not None else 0.003
    ring_max = cfg.rl_ring_max if cfg.rl_ring_max is not None else 7.5
    if cfg.auto_iterations:
        restored, k_done = rl_adaptive(ch.astype(np.float64), psf.astype(np.float64),
max_iter=cfg.rl_iterations,
eps=cfg.rl_filter_eps,
min_gain_frac=min_gain, ring_max=ring_max,
verbose=cfg.verbose)

```

```

        else:
            restored = _rl_steps(ch.astype(np.float64), psf.astype(np.float64),
cfg.rl_iterations, cfg.rl_filter_eps)
            k_done = cfg.rl_iterations
            meta = {'model': model, 'gamma': gamma, 'angle': angle, 'beta': beta,
                    'psf_size': size, 'psf': psf, 'sigma_est': sigma_est, 'rl_iters': k_done}
            return restored, meta

    def _process_lab(self):
        lab = cv2.cvtColor(self.img, cv2.COLOR_BGR2Lab)
        L = (lab[:, :, 0] / 100.0).astype(np.float32)
        if self.cfg.verbose: print("\n[Colour] CIE-Lab -> deconvolve L only")
        L_rest, meta = self._process_channel(L, 'L')
        lab_out = lab.copy(); lab_out[:, :, 0] = np.clip(L_rest * 100.0, 0.0, 100.0)
        out = cv2.cvtColor(lab_out, cv2.COLOR_Lab2BGR)
        return np.clip(out, 0.0, 1.0).astype(np.float32), meta

    def _process_independent(self):
        if self.img.ndim == 2: return self._process_channel(self.img, 'ch0')
        results, metas = [], []
        for i in range(self.n_ch):
            if self.cfg.verbose: print(f"\n[Band {i}]")
            r, m = self._process_channel(self.img[:, :, i], f'B{i}')
            results.append(r); metas.append(m)
        return np.stack(results, axis=-1), metas[0]

    def process(self):
        cfg = self.cfg
        if cfg.verbose:
            print("\n" + "=" * 60)
            print(f" RestorationEngine domain={cfg.domain} auto_iter={cfg.auto_iteratio
ns}")
            print("=" * 60)
        use_ind = self.img.ndim == 2 or self.n_ch != 3 or cfg.independent_channels
        restored, meta = self._process_independent() if use_ind else self._process_lab()
        if cfg.sharpen_alpha == 0.0:
            final, alpha_used = restored, 0.0
        else:
            if cfg.verbose: print("\n[Sharpen] Multiscale unsharp masking")
            final, alpha_used = AdaptiveSharpen.apply(restored, cfg.sharpen_alpha,
domain=cfg.domain)
            lum_o = _luminance(self.img); lum_f = _luminance(final)
            t0 = SharpnessMetrics.tenengrad(lum_o); t1 = SharpnessMetrics.tenengrad(lum_f)
            lv0 = SharpnessMetrics.laplacian_variance(lum_o); lv1 =
SharpnessMetrics.laplacian_variance(lum_f)
            rng = SharpnessMetrics.ringing_score(lum_f)
            metrics = {'tenengrad_before': t0, 'tenengrad_after': t1,
                        'tenengrad_gain_pct': (t1 - t0) / (t0 + 1e-9) * 100,
                        'laplacian_var_before': lv0, 'laplacian_var_after': lv1,
'ringing_score': rng}
            if cfg.verbose: _print_report(meta, alpha_used, metrics, cfg)
            return {'result': final, 'original': self.img, 'restored': restored, 'alpha':
alpha_used, 'metrics': metrics, **meta}

    def _luminance(img):
        if img.ndim == 2: return img
        if img.shape[2] == 1: return img[:, :, 0]

```

```

    if img.shape[2] == 3:
        return (0.114 * img[:, :, 0] + 0.587 * img[:, :, 1] + 0.299 * img[:, :,
2]).astype(np.float32)
    return img[:, :, 0]

def _print_report(meta, alpha, m, cfg):
    rs = m['ringing_score']; warn = ' WARNING: reduce iterations' if rs > 8 else ' OK'
    ring_max_v = cfg.rl_ring_max if cfg.rl_ring_max is not None else 7.5
    print("\n" + "=" * 60)
    print(f"   domain      : {cfg.domain}")
    print(f"   PSF model   :
{meta.get('model','?')}   gamma*={meta.get('gamma',0):.4f}   sigma_est={meta.get('sigma_est'
,0):.3f} px")
    if meta.get('model') == 'moffat': print(f"   Moffat beta : {meta.get('beta',3.5):.1f}")
    if meta.get('model') == 'motion': print(f"   Blur angle  : {meta.get('angle',0):.1f}
deg")
    print(f"   RL iters    :
{meta.get('rl_iters','?')}   (auto={cfg.auto_iterations}   ring_max={ring_max_v:.1f})")
    print(f"   Sharpen a   : {alpha:.3f}")
    print(f"   Tenengrad   : {m['tenengrad_before']:.3f} ->
{m['tenengrad_after']:.3f}   (+{m['tenengrad_gain_pct']:.1f}%)")
    print(f"   Lapl.var.   : {m['laplacian_var_before']:.3f} ->
{m['laplacian_var_after']:.3f}")
    print(f"   Ringing     : {rs:.2f}{warn}")
    print("=" * 60)

```

Лістинг А.8 – Клас RestorationEngine: повний пайплайн відновлення

ДОДАТОК Б

Лістинг графічного інтерфейсу користувача

Б.1 Ініціалізація інтерфейсу та панелі керування

```

from __future__ import annotations
import threading
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
from pathlib import Path
import numpy as np
import cv2
import matplotlib
import matplotlib.lines
matplotlib.use('TkAgg')
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from diploma_engine import (
    EngineConfig, RestorationEngine, domain_config, apply_exposure,
    DOMAIN_KEYS, DOMAIN_DISPLAY,
)

BG, PANEL, BORDER = '#12141c', '#1c1f2e', '#282c3e'
ACCENT, TEXT, DIM = '#5b8af0', '#dde0f0', '#6b7090'
GREEN, ORANGE, RED, ENTRY = '#4caf7d', '#f0a040', '#e05555', '#22253a'

PSF_MAP = {
    'Авто (ідентифікація)': 'auto',
    'Gaussian - атмосферний blur': 'gaussian',
    'Cauchy - сильна турбулентність': 'cauchy',
    'Airy disk - дифракційна межа': 'airy',
    'Moffat - atmospheric seeing': 'moffat',
    'Motion blur - розмиття руху': 'motion',
    'Scattering - розсіювання BSI': 'scattering',
}
PSF_LABELS = list(PSF_MAP.keys())

def to_rgb(img):
    if img.ndim == 2: return (img * 255).clip(0, 255).astype(np.uint8)
    if img.shape[2] == 1: return (img[:, :, 0] * 255).clip(0, 255).astype(np.uint8)
    if img.shape[2] == 3: return cv2.cvtColor((img * 255).clip(0, 255).astype(np.uint8),
cv2.COLOR_BGR2RGB)
    return (img[:, :, :3] * 255).clip(0, 255).astype(np.uint8)

def is_gray(img): return img.ndim == 2 or (img.ndim == 3 and img.shape[2] == 1)

def make_label(parent, text, color=DIM, font=('Segoe UI', 8), **kw):
    return tk.Label(parent, text=text, bg=PANEL, fg=color, font=font, **kw)

def make_sep(parent): tk.Frame(parent, bg=BORDER, height=1).pack(fill='x', pady=6)

def make_btn(parent, text, cmd, bg=ACCENT, fg=TEXT, font=('Segoe UI', 9), **kw):
    return tk.Button(parent, text=text, command=cmd, bg=bg, fg=fg, relief='flat',

```



```

        activebackground=BORDER, activeforeground=fg,
        font=font, cursor='hand2', padx=8, pady=5, **kw)

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Adaptive PSF Deconvolution | CMOS Scientific Imaging")
        self.configure(bg=BG)
        self.minsize(1100, 640)
        self.geometry('1260x720')
        self._output = None
        self._file_path = None
        self._preview = None
        self._ev_var = tk.DoubleVar(value=0.0)
        self._auto_var = tk.BooleanVar(value=True)
        self._ev_var.trace_add('write', lambda *_: self._redraw())
        self._build_styles()
        self._build_ui()

    def _build_styles(self):
        s = ttk.Style(self)
        s.theme_use('clam')
        for name, bg_ in [('Panel.TFrame', PANEL), ('BG.TFrame', BG)]:
            s.configure(name, background=bg_)
            s.configure('Dark.TCombobox', fieldbackground=ENTRY, background=ENTRY,
                        foreground=TEXT, arrowcolor=ACCENT, selectbackground=ACCENT)
            s.map('Dark.TCombobox', fieldbackground=[('readonly', ENTRY)],
foreground=[('readonly', TEXT)])
            s.configure('T.Horizontal.TScale', background=PANEL, troughcolor=BORDER,
sliderthickness=13)
            s.configure('Prog.Horizontal.TProgressbar', troughcolor=BORDER, background=ACCENT,
borderwidth=0, thickness=4)

    def _build_ui(self):
        bar = tk.Frame(self, bg=BG, height=40)
        bar.pack(fill='x')
        tk.Label(bar, text="PSF Deconvolution Engine", bg=BG, fg=ACCENT, font=('Segoe UI',
12, 'bold')).pack(side='left', padx=16, pady=8)
        tk.Label(bar, text="Адаптивна фільтрація зображень для CMOS-сенсорів", bg=BG,
fg=DIM, font=('Segoe UI', 9)).pack(side='left', pady=8)
        tk.Frame(self, bg=BORDER, height=1).pack(fill='x')
        body = tk.Frame(self, bg=BG)
        body.pack(fill='both', expand=True, padx=10, pady=10)
        self._build_panel(body)
        self._build_canvas(body)

    def _build_panel(self, parent):
        panel = tk.Frame(parent, bg=PANEL, width=272)
        panel.pack(side='left', fill='y', padx=(0, 10))
        panel.pack_propagate(False)
        P = panel

        def row(text, color=DIM): make_label(P, text, color=color).pack(anchor='w',
padx=12, pady=(6, 0))

        make_sep(P); row("Вхідний файл", TEXT)
        fr = tk.Frame(P, bg=PANEL); fr.pack(fill='x', padx=10, pady=4)

```

```

        make_btn(fr, "Відкрити", self._open_file, bg=ENTRY, fg=TEXT, font=('Segoe UI',
8)).pack(side='left')
        self._lbl_file = tk.Label(fr, text="не вибрано", bg=PANEL, fg=DIM, font=('Segoe
UI', 8), anchor='w', wraplength=155)
        self._lbl_file.pack(side='left', padx=6)

        make_sep(P); row("Тематика", TEXT)
        self._domain_var = tk.StringVar(value=DOMAIN_DISPLAY[0])
        cb = ttk.Combobox(P, textvariable=self._domain_var, values=DOMAIN_DISPLAY,
state='readonly', style='Dark.TCombobox', width=30)
        cb.pack(padx=10, pady=4, fill='x')
        cb.bind('<<ComboboxSelected>>', self._on_domain)
        self._lbl_hint = tk.Label(P, text="", bg=PANEL, fg=DIM, font=('Segoe UI', 7,
'italic'), wraplength=248, justify='left')
        self._lbl_hint.pack(anchor='w', padx=12)

        make_sep(P); row("Модель PSF", TEXT)
        self._psf_var = tk.StringVar(value=PSF_LABELS[0])
        ttk.Combobox(P, textvariable=self._psf_var, values=PSF_LABELS, state='readonly',
style='Dark.TCombobox', width=30).pack(padx=10, pady=4, fill='x')

        make_sep(P); row("Ітерації RL", TEXT)
        fr_auto = tk.Frame(P, bg=PANEL); fr_auto.pack(fill='x', padx=10, pady=(2, 0))
        tk.Checkbutton(fr_auto, text="Авто-зупинка (за збіжністю)",
variable=self._auto_var, bg=PANEL, fg=TEXT,
                        selectcolor=ENTRY, activebackground=PANEL, font=('Segoe UI', 8),
                        command=self._on_auto_toggle).pack(side='left')
        self._fr_iter = tk.Frame(P, bg=PANEL); self._fr_iter.pack(fill='x', padx=10,
pady=2)
        self._iter_var = tk.IntVar(value=30)
        sc = ttk.Scale(self._fr_iter, from_=5, to=80, orient='horizontal',
variable=self._iter_var, style='T.Horizontal.TScale', length=200)
        sc.pack(side='left')
        self._lbl_iter = tk.Label(self._fr_iter, text="30", bg=PANEL, fg=ACCENT,
font=('Segoe UI', 9, 'bold'), width=3)
        self._lbl_iter.pack(side='left', padx=4)
        self._iter_var.trace_add('write', lambda *_:
self._lbl_iter.config(text=str(int(self._iter_var.get()))))
        self._lbl_iter_cap = make_label(P, "максимум (при авто) або точне (при ручному)")
        self._lbl_iter_cap.pack(anchor='w', padx=12)

        make_sep(P); row("Загострення (0 = вимкнено)", TEXT)
        fr_sh = tk.Frame(P, bg=PANEL); fr_sh.pack(fill='x', padx=10, pady=2)
        self._sh_var = tk.DoubleVar(value=0.0)
        ttk.Scale(fr_sh, from_=0.0, to=3.0, orient='horizontal', variable=self._sh_var,
style='T.Horizontal.TScale', length=200).pack(side='left')
        self._lbl_sh = tk.Label(fr_sh, text="0.0", bg=PANEL, fg=TEXT, font=('Segoe UI',
8), width=4)
        self._lbl_sh.pack(side='left', padx=4)
        self._sh_var.trace_add('write', lambda *_:
self._lbl_sh.config(text=f"{self._sh_var.get():.1f}"))

        make_sep(P); row("Експозиція результату (EV)", TEXT)
        fr_ev = tk.Frame(P, bg=PANEL); fr_ev.pack(fill='x', padx=10, pady=2)
        ttk.Scale(fr_ev, from_=-3.0, to=3.0, orient='horizontal', variable=self._ev_var,
style='T.Horizontal.TScale', length=180).pack(side='left')

```

```

        self._lbl_ev = tk.Label(fr_ev, text="+0.0 EV", bg=PANEL, fg=ACCENT, font=('Segoe
UI', 8, 'bold'), width=7)
        self._lbl_ev.pack(side='left', padx=2)
        self._ev_var.trace_add('write', lambda *_:
self._lbl_ev.config(text=f"{self._ev_var.get():+.1f} EV"))
        tk.Button(fr_ev, text="0", bg=BORDER, fg=DIM, relief='flat', font=('Segoe UI', 8),
padx=4,
                    command=lambda: self._ev_var.set(0.0)).pack(side='left', padx=2)

        make_sep(P)
        self._btn_run = make_btn(P, "Обробити", self._run, bg=ACCENT, font=('Segoe UI',
10, 'bold'))
        self._btn_run.pack(fill='x', padx=10, pady=(0, 4))
        self._prog = ttk.Progressbar(P, mode='indeterminate', length=250,
style='Prog.Horizontal.TProgressbar')
        self._prog.pack(padx=10)
        self._lbl_status = tk.Label(P, text="", bg=PANEL, fg=DIM, font=('Segoe UI', 8,
'italic'))
        self._lbl_status.pack(pady=2)

        make_sep(P)
        self._lbl_metrics = tk.Label(P, text="немає результату", bg=PANEL, fg=DIM,
font=('Courier New', 8), justify='left', anchor='w')
        self._lbl_metrics.pack(anchor='w', padx=12, pady=2)

        make_sep(P)
        self._btn_save = make_btn(P, "Зберегти результат", self._save, bg='#2e6e50')
        self._btn_save.pack(fill='x', padx=10, pady=(0, 8))
        self._btn_save.config(state='disabled')
        self._on_domain()
        self._on_auto_toggle()

```

Лістинг Б.1 – Ініціалізація GUI та панель параметрів

Б.2 Логіка обробки та відображення результатів

```

def _build_canvas(self, parent):
    right = tk.Frame(parent, bg=BG)
    right.pack(side='left', fill='both', expand=True)
    self._fig = Figure(facecolor=BG, tight_layout=True)
    self._canvas = FigureCanvasTkAgg(self._fig, master=right)
    self._canvas.get_tk_widget().pack(fill='both', expand=True)
    self._draw_placeholder()

    def _on_domain(self, *_):
        idx = DOMAIN_DISPLAY.index(self._domain_var.get()) if self._domain_var.get() in
DOMAIN_DISPLAY else 0
        domain = DOMAIN_KEYS[idx]
        hints = {
            'deep_space': "Moffat PSF з адаптивним beta [Moffat 1969]; загострення
вимкнено; посилений штраф за ringing ( $\lambda=0.40$ ); незалежні канали.",
            'planetary': "Авто PSF; вузький діапазон пошуку гамма ( $\leq 6$  px); знижений поріг
збіжності RL ( $0.001$ ); помірно загострення.",

```

```

        'microscopy': "Airy disk (дифракційна межа [Born&Wolf]); вузькі межі gamma (<4
px); 50 ітерацій; незалежні канали.",
        'remote': "Авто PSF; знижений поріг motion blur; gamma ≤ 6 px; пом'якшене
загострення для запобігання перешарпу.",
    }
    self._lbl_hint.config(text=hints.get(domain, ""))
    cfg = domain_config(domain)
    self._iter_var.set(cfg.rl_iterations)
    self._sh_var.set(round(cfg.sharpen_alpha if cfg.sharpen_alpha is not None else
0.0, 1))

    def _on_auto_toggle(self):
        if self._auto_var.get():
            self._lbl_iter_cap.config(text="максимум ітерацій (зупинка за збіжністю)",
fg=DIM)
        else:
            self._lbl_iter_cap.config(text="точна кількість ітерацій (без авто-зупинки)",
fg=ORANGE)

    def _open_file(self):
        path = filedialog.askopenfilename(filetypes=[
            ("Зображення", "*.tif *.tiff *.png *.jpg *.jpeg *.fit *.fits *.fts"),
            ("All files", "*.*"),
        ])
        if not path: return
        self._file_path = path
        self._lbl_file.config(text=Path(path).name, fg=TEXT)
        try:
            raw = cv2.imread(path, cv2.IMREAD_UNCHANGED)
            if raw is not None:
                scale = 65535.0 if raw.dtype == np.uint16 else 255.0
                self._preview = raw.astype(np.float32) / scale
                self._output = None
                self._draw_single(self._preview, "Оригінал")
        except Exception: pass

    def _run(self):
        if not self._file_path:
            messagebox.showwarning("Файл не вибрано", "Спочатку відкрийте зображення.")
            return
        idx = DOMAIN_DISPLAY.index(self._domain_var.get()) if self._domain_var.get() in
DOMAIN_DISPLAY else 0
        domain = DOMAIN_KEYS[idx]
        psf_k = PSF_MAP.get(self._psf_var.get(), 'auto')
        sh_raw = self._sh_var.get()
        sha = None if sh_raw < 0.05 else float(sh_raw)
        cfg = domain_config(domain, psf_model=psf_k,
rl_iterations=int(self._iter_var.get()),
auto_iterations=self._auto_var.get(), sharpen_alpha=sha,
verbose=True)
        self._btn_run.config(state='disabled')
        self._btn_save.config(state='disabled')
        self._prog.start(12)
        self._lbl_status.config(text="обробка...", fg=ORANGE)

    def worker():
        try:

```

```

        out = RestorationEngine(self._file_path, cfg).process()
        self.after(0, lambda: self._done(out))
    except Exception as exc:
        self.after(0, lambda: self._error(str(exc)))
    threading.Thread(target=worker, daemon=True).start()

def _done(self, output):
    self._output = output
    self._prog.stop()
    self._btn_run.config(state='normal')
    self._btn_save.config(state='normal')
    m = output['metrics']; rs = m['ringing_score']; k = output.get('rl_iters', '?')
    status = f"робота k={k} R={rs:.1f}" + (" ⚠" if rs > 8 else " OK")
    self._lbl_status.config(text=status, fg=RED if rs > 8 else GREEN)
    self._update_metrics(output)
    self._redraw()

def _error(self, msg):
    self._prog.stop()
    self._btn_run.config(state='normal')
    self._lbl_status.config(text="помилка", fg=RED)
    messagebox.showerror("Помилка", msg)

def _update_metrics(self, out):
    m = out['metrics']; rs = m['ringing_score']
    col = RED if rs > 8 else GREEN
    txt = (f"PSF      {out.get('model','?')}  g={out.get('gamma',0):.3f}\n"
          f"sigma    {out.get('sigma_est',0):.2f} px\n"
          f"iters     {out.get('rl_iters','?')}\n"
          f"Tenengrad  +{m['tenengrad_gain_pct']:.1f}%\n"
          f"Ringing    {rs:.2f}" + (" RING!" if rs > 8 else ""))
    self._lbl_metrics.config(text=txt, fg=col)

def _redraw(self):
    if self._output is None:
        if self._preview is not None: self._draw_single(self._preview, "Оригінал")
        return
    self._draw_compare(self._output['original'],
    apply_exposure(self._output['result'], self._ev_var.get()), self._output)

def _draw_placeholder(self):
    self._fig.clear()
    ax = self._fig.add_subplot(111)
    ax.set_facecolor(BG)
    ax.text(0.5, 0.5, "Відкрийте файл та натисніть «Обробити»", ha='center',
    va='center', color=DIM, fontsize=12, transform=ax.transAxes)
    ax.axis('off')
    self._canvas.draw()

def _draw_single(self, img, title=""):
    self._fig.clear()
    ax = self._fig.add_subplot(111)
    ax.set_facecolor(BG)
    ax.imshow(to_rgb(img), cmap='gray' if is_gray(img) else None,
    interpolation='nearest')
    ax.set_title(title, color=TEXT, fontsize=10, pad=6)
    ax.axis('off')

```

```

self._fig.tight_layout(pad=0.5)
self._canvas.draw()

def _draw_compare(self, orig, final, out):
    self._fig.clear()
    self._fig.patch.set_facecolor(BG)
    m = out['metrics']; rs = m['ringing_score']
    ev = self._ev_var.get()

    ax1 = self._fig.add_axes([0.01, 0.06, 0.48, 0.88])
    ax1.set_facecolor(BG)
    ax1.imshow(to_rgb(orig), cmap='gray' if is_gray(orig) else None,
interpolation='nearest')
    ax1.set_title("Оригінал", color=TEXT, fontsize=10, pad=6)
    ax1.set_xlabel(f"T = {m['tenengrad_before']:.4f}", color=DIM, fontsize=8,
labelpad=4)
    ax1.tick_params(left=False, bottom=False, labelleft=False, labelbottom=False)

    ax2 = self._fig.add_axes([0.51, 0.06, 0.48, 0.88])
    ax2.set_facecolor(BG)
    ax2.imshow(to_rgb(final), cmap='gray' if is_gray(final) else None,
interpolation='nearest')
    ev_s = f" EV {ev:+.1f}" if ev != 0.0 else ""
    ax2.set_title(f"Результат [{out.get('model','?')} g={out.get('gamma',0):.3f}
k={out.get('r1_iters','?')}] {ev_s}", color=TEXT, fontsize=10, pad=6)
    t1_s = f"T =
{m['tenengrad_after']:.4f} (+{m['tenengrad_gain_pct']:.1f}%) R={rs:.1f}" + (" Δ" if rs
> 8 else "")
    ax2.set_xlabel(t1_s, color='#e05555' if rs > 8 else '#4caf7d', fontsize=8,
labelpad=4)
    ax2.tick_params(left=False, bottom=False, labelleft=False, labelbottom=False)

    self._fig.add_artist(matplotlib.lines.Line2D([0.503, 0.503], [0.02, 0.98],
color=BORDER, linewidth=1, transform=self._fig.transFigure))
    self._canvas.draw()

def _save(self):
    if self._output is None: return
    final = apply_exposure(self._output['result'], self._ev_var.get())
    path = filedialog.asksaveasfilename(defaultextension='.tiff', filetypes=[
        ("16-bit TIFF (наукове)", "*.tiff"), ("PNG 8-bit", "*.png"), ("JPEG 8-bit",
        "*.jpg"),
    ])
    if not path: return
    p = path.lower()
    if p.endswith(('.tif', '.tiff')): cv2.imwrite(path, (final *
65535).astype(np.uint16))
    elif p.endswith('.png'): cv2.imwrite(path, (final * 255).clip(0,
255).astype(np.uint8))
    else: cv2.imwrite(path, (final * 255).clip(0, 255).astype(np.uint8),
[cv2.IMWRITE_JPEG_QUALITY, 95])
    messagebox.showinfo("Збережено", f"Файл збережено:\n{path}")

if __name__ == '__main__':
    App().mainloop()

```

Лістинг Б.2 – Обробка зображень та відображення результатів у GUI

ДОДАТОК В

Лістинг модуля порівняльного аналізу методів

В.1 Допоміжні функції та налаштування середовища

```
import sys, os, time, warnings, traceback, subprocess
from pathlib import Path

def _ensure_package(import_name, pip_name=None):
    try: __import__(import_name)
    except ImportError:
        pkg = pip_name or import_name
        print(f"[INFO] Пакет '{pkg}' не знайдено. Встановлення...")
        subprocess.check_call([sys.executable, "-m", "pip", "install", pkg, "--quiet"],
                               stdout=subprocess.DEVNULL)
        print(f"[INFO] '{pkg}' встановлено успішно.")

_ensure_package("openpyxl")
_ensure_package("matplotlib")

import numpy as np
import cv2
from scipy.signal import wiener

SCRIPT_DIR = Path(__file__).resolve().parent
sys.path.insert(0, str(SCRIPT_DIR))

try:
    from diploma_engine import (
        RestorationEngine, domain_config, SharpnessMetrics, _luminance,
        DOMAIN_DEEP_SPACE, DOMAIN_PLANETARY, DOMAIN_MICRO, DOMAIN_REMOTE,
    )
    ENGINE_AVAILABLE = True
except ImportError as e:
    print(f"[WARN] diploma_engine не знайдено: {e}"); ENGINE_AVAILABLE = False

TEST_DIR = Path(r"C:\Users\ibatu\OneDrive\Desktop\diploma code\Test data")
OUT_XLSX = SCRIPT_DIR / "comparison_results.xlsx"
OUT_CHART = SCRIPT_DIR / "comparison_chart.png"
RINGING_WARN = 8.0
TENENGRAD_GOOD = 30.0

def load_image(path):
    img = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if img is None: raise IOError(f"Не вдалося відкрити: {path}")
    divisor = 65535.0 if img.dtype == np.uint16 else 255.0
    if img.ndim == 2: return img.astype(np.float32) / divisor
    return cv2.cvtColor(img, cv2.COLOR_BGR2RGB).astype(np.float32) / divisor

def _lum_fallback(img):
    if img.ndim == 2: return img
    return (0.299 * img[..., 0] + 0.587 * img[..., 1] + 0.114 * img[...,
2]).astype(np.float32)
```

```
def metrics(img):
    lum = _luminance(img) if ENGINE_AVAILABLE else _lum_fallback(img)
    return {"tenengrad": SharpnessMetrics.tenengrad(lum),
            "laplacian_var": SharpnessMetrics.laplacian_variance(lum),
            "ringing_score": SharpnessMetrics.ringing_score(lum)}
```

Лістинг В.1 – Налаштування середовища та допоміжні утиліти

В.2 Реалізація методів порівняння

```
def method_wiener(img):
    if img.ndim == 2: return np.clip(wiener(img.astype(np.float64), mysize=5), 0,
1).astype(np.float32)
    return np.stack([np.clip(wiener(img[..., c]).astype(np.float64), mysize=5), 0, 1) for c
in range(img.shape[2])], axis=-1).astype(np.float32)

def method_gaussian_unsharp(img, sigma=1.0, alpha=1.5):
    ksize = int(6 * sigma + 1) | 1
    blurred = cv2.GaussianBlur(img, (ksize, ksize), sigma)
    return np.clip(img + alpha * (img - blurred), 0, 1).astype(np.float32)

def method_rl_fixed(img, psf_sigma=1.5, n_iter=20):
    try:
        from skimage.restoration import richardson_lucy
        ksize = int(6 * psf_sigma + 1) | 1
        psf_1d = cv2.getGaussianKernel(ksize, psf_sigma)
        psf_2d = (psf_1d @ psf_1d.T).astype(np.float64); psf_2d /= psf_2d.sum()
        def _deconv_ch(ch):
            with warnings.catch_warnings(): warnings.simplefilter("ignore")
            return np.clip(richardson_lucy(ch.astype(np.float64), psf_2d,
num_iter=n_iter), 0, 1).astype(np.float32)
        if img.ndim == 2: return _deconv_ch(img)
        return np.stack([_deconv_ch(img[..., c]) for c in range(img.shape[2])], axis=-
1).astype(np.float32)
    except ImportError:
        return method_gaussian_unsharp(img)

def detect_domain(path):
    DOMAIN_MAP = {
        "moons": DOMAIN_PLANETARY, "jupiter": DOMAIN_PLANETARY, "sharpening up jupiter":
DOMAIN_PLANETARY,
        "centaurus a": DOMAIN_DEEP_SPACE, "centaurusa": DOMAIN_DEEP_SPACE, "centaura":
DOMAIN_DEEP_SPACE,
        "helix nebula": DOMAIN_DEEP_SPACE, "helixnebula": DOMAIN_DEEP_SPACE, "the helix
nebula": DOMAIN_DEEP_SPACE,
        "sentinel": DOMAIN_REMOTE, "bakhmut": DOMAIN_REMOTE,
        "cells": DOMAIN_MICRO, "cells2": DOMAIN_MICRO, "cells3": DOMAIN_MICRO, "cells4":
DOMAIN_MICRO, "cells5": DOMAIN_MICRO,
    }
    stem_lower = path.stem.lower()
    if stem_lower in DOMAIN_MAP: return DOMAIN_MAP[stem_lower]
    for key, domain in DOMAIN_MAP.items():
        if key in stem_lower: return domain
```



```

    if any(k in stem_lower for k in ("deep", "galaxy", "nebula", "star", "space",
"centaur", "helix", "andromeda", "ngc")): return DOMAIN_DEEP_SPACE
    if any(k in stem_lower for k in ("micro", "cell", "tissue", "fluor", "bbbc")): return
DOMAIN_MICRO
    if any(k in stem_lower for k in ("satellite", "remote", "earth", "terrain",
"sentinel", "landsat", "spot", "2026-03", "2025-")): return DOMAIN_REMOTE
    if any(k in stem_lower for k in ("moon", "lunar", "місяць")): return DOMAIN_PLANETARY
    return DOMAIN_PLANETARY

def process_image(path):
    print(f"\n{'-'*60}\n {path.name}\n{'-'*60}")
    try: img = load_image(path)
    except IOError as e: print(f"  [!] {e}"); return None

    domain = detect_domain(path); m_orig = metrics(img)
    row = {"file": path.name, "domain": domain, "resolution":
f"{img.shape[1]}x{img.shape[0]}",
        "tenengrad_orig": m_orig["tenengrad"], "laplacian_orig":
m_orig["laplacian_var"], "ringing_orig": m_orig["ringing_score"]}

    if ENGINE_AVAILABLE:
        try:
            t0 = time.perf_counter()
            cfg = domain_config(domain, verbose=False)
            out = RestorationEngine(img, cfg).process()
            elapsed = time.perf_counter() - t0
            m_eng = metrics(out["result"])
            row.update({"tenengrad_engine": m_eng["tenengrad"], "laplacian_engine":
m_eng["laplacian_var"],
                    "ringing_engine": m_eng["ringing_score"],
                    "gain_pct_engine": (m_eng["tenengrad"] - m_orig["tenengrad"]) /
(m_orig["tenengrad"] + 1e-9) * 100,
                    "psf_model": out.get("model", "?"), "gamma_opt":
round(out.get("gamma", 0), 3),
                    "rl_iters": out.get("rl_iters", "?"), "time_engine_s":
round(elapsed, 2)})
            print(f"  [Engine] T {m_orig['tenengrad']:.3f} →
{m_eng['tenengrad']:.3f}  ({row['gain_pct_engine']:.1f}%)  R={m_eng['ringing_score']:.2f}
PSF={out.get('model','?')}  γ*={out.get('gamma',0):.3f}  t={elapsed:.1f}s")
        except Exception:
            print("  [Engine] ПОМИЛКА:"); traceback.print_exc()
            row.update({k: None for k in
["tenengrad_engine", "laplacian_engine", "ringing_engine", "gain_pct_engine", "psf_model", "gam
ma_opt", "rl_iters", "time_engine_s"]})
        else:
            row.update({k: "N/A" for k in
["tenengrad_engine", "laplacian_engine", "ringing_engine", "gain_pct_engine", "psf_model", "gam
ma_opt", "rl_iters", "time_engine_s"]})

    for tag, fn, elapsed_key in [
        ("Wiener", lambda: method_wiener(img), "time_wiener_s"),
        ("GUM", lambda: method_gaussian_unsharp(img), "time_gum_s"),
        ("RL-fix", lambda: method_rl_fixed(img), "time_rl_s"),
    ]:
        sfx = {"Wiener": "wiener", "GUM": "gum", "RL-fix": "rl"}[tag]
        try:
            t0 = time.perf_counter(); res = fn(); el = time.perf_counter() - t0

```

```

        m = metrics(res)
        row.update({f"tenengrad_{sfx}": m["tenengrad"], f"laplacian_{sfx}":
m["laplacian_var"],
                    f"ringing_{sfx}": m["ringing_score"],
                    f"gain_pct_{sfx}": (m["tenengrad"] - m_orig["tenengrad"]) /
(m_orig["tenengrad"] + 1e-9) * 100,
                    elapsed_key: round(el, 3)})
        print(f" [{tag:<6}] T {m_orig['tenengrad']:.3f} →
{m['tenengrad']:.3f} (+{row[f'gain_pct_{sfx}']:.1f}%) R={m['ringing_score']:.2f} t={el:
.3f}s")
    except Exception:
        print(f" [{tag}] ПОМИЛКА:"); traceback.print_exc()
        row.update({k: None for k in [f"tenengrad_{sfx}", f"laplacian_{sfx}",
f"ringing_{sfx}", f"gain_pct_{sfx}", elapsed_key]})
    return row

KNOWN_RESULTS = [
    ("moons.jpg",          "planetary", "Moffat", 0.993, 3.31, 5, 115.8, 52.93),
    ("Jupiter.jpg",       "planetary", "Moffat", 4.499, 7.71, 5, 91.2, 20.86),
    ("Centaurus A.jpg",   "deep_space", "Moffat", 1.749, 3.34, 5, 128.0, 25.95),
    ("Helix Nebula.jpg",  "deep_space", "Moffat", 0.382, 1.27, 5, 70.1, 264.6),
    ("sentinel.jpg",      "remote",     "Moffat", 1.495, 0.84, 5, 512.9, 5.91),
    ("Bakhmut.jpg",       "remote",     "Moffat", 1.327, 1.82, 5, 180.2, 10.83),
    ("cells.jpg",         "microscopy", "Airy", 1.357, 4.10, 15, 645.7, 4.15),
    ("cells3.jpg",        "microscopy", "Airy", 1.194, 3.86, 5, 214.3, 7.61),
    ("cells2.jpg",        "microscopy", "Airy", None, 8.06, 5, 189.0, 206.5),
    ("cells4.jpg",        "microscopy", "Airy", 1.722, 5.74, 5, 205.1, 98.07),
    ("cells5.jpg",        "microscopy", "Airy", 1.012, 6.04, 5, 193.7, 66.43),
]

def _write_known_results_sheet(wb, border):
    from openpyxl.styles import Font, PatternFill, Alignment
    from openpyxl.utils import get_column_letter
    FONT = "Arial"; CLR_HDR = "1F4E79"; CLR_RING = "FF0000"; CLR_OK = "375623"
    DOMAIN_COLORS = {"planetary": "DDEEFF", "deep_space": "E8D5F5", "remote": "D5F5E3",
"microscopy": "FFF3CC"}
    ws = wb.create_sheet("Усі 11 зображень")
    headers = ["№", "Файл", "Домен", "PSF модель", "γ* (оптим.)", "σест, пкс", "Ітерацій",
"RL", "Приріст Tenengrad, %", "Ringing Score", "Примітка"]
    col_widths = [5, 26, 14, 14, 12, 12, 12, 22, 16, 36]
    ws.merge_cells("A1:J1")
    tc = ws["A1"]; tc.value = "Результати тестування системи адаптивної деконволюції
(Diploma Engine) – 11 зображень"
    tc.font = Font(name=FONT, bold=True, color="FFFFFF", size=12)
    tc.fill = PatternFill("solid", fgColor=CLR_HDR)
    tc.alignment = Alignment(horizontal="center", vertical="center"); tc.border = border
    ws.row_dimensions[1].height = 28
    for ci, h in enumerate(headers, start=1):
        c = ws.cell(row=2, column=ci, value=h)
        c.font = Font(name=FONT, bold=True, color="FFFFFF", size=10)
        c.fill = PatternFill("solid", fgColor=CLR_HDR)
        c.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True);
c.border = border
    ws.row_dimensions[2].height = 36
    for ri, (fname, domain, psf, gamma, sigma, iters, dt, ring) in
enumerate(KNOWN_RESULTS, start=3):
        row_bg = DOMAIN_COLORS.get(domain, "F2F2F2")

```

```

    note = ""
    if ring > 8: note = f"▲ Ringing > 8 (природний лептокуртоз для {domain})"
    if dt == max(r[6] for r in KNOWN_RESULTS): note = "★ Максимальний приріст різкості
у вибірці"
    if dt == min(r[6] for r in KNOWN_RESULTS): note = "▲ Мінімальний приріст
(σest=1.27 пкс)"
    row_data = [ri-2, fname, domain, psf, gamma if gamma is not None else "авто",
sigma, iters, dt, ring, note]
    for ci, val in enumerate(row_data, start=1):
        c = ws.cell(row=ri, column=ci, value=val)
        c.fill = PatternFill("solid", fgColor=row_bg)
        c.alignment = Alignment(horizontal="center" if ci not in (2,3,10) else "left",
vertical="center")
        c.border = border
        if ci == 9: c.font = Font(name=FONT, size=10, bold=True, color=CLR_RING if
isinstance(val, float) and val > 8.0 else CLR_OK)
        elif ci == 8: c.font = Font(name=FONT, size=10, bold=True); c.number_format =
"0.0"

        elif ci == 4: c.font = Font(name=FONT, size=10, bold=True)
        else: c.font = Font(name=FONT, size=10)
    ws.row_dimensions[ri].height = 18
    last_row = 3 + len(KNOWN_RESULTS)
    ws.merge_cells(f"A{last_row}:G{last_row}")
    summary = ws.cell(row=last_row, column=1, value="Медіана ΔT: +189.0 % | Мінімум:
+70.1 % | Максимум: +645.7 %")
    summary.font = Font(name=FONT, bold=True, size=10, color="FFFFFF")
    summary.fill = PatternFill("solid", fgColor="2E75B6"); summary.alignment =
Alignment(horizontal="center", vertical="center"); summary.border = border
    ring_ok = sum(1 for r in KNOWN_RESULTS if r[7] <= 8.0)
    ws.cell(row=last_row, column=8, value=f"Ringing ≤ 8: {ring_ok} | Ringing > 8:
{len(KNOWN_RESULTS)-ring_ok}").font = Font(name=FONT, bold=True, size=10)
    ws.row_dimensions[last_row].height = 22
    for i, w in enumerate(col_widths, start=1):
ws.column_dimensions[get_column_letter(i)].width = w
    legend_row = last_row + 2
    ws.cell(row=legend_row, column=1, value="Кольорова легенда доменів:").font =
Font(name=FONT, bold=True, size=9)
    for offset, (dom, color) in enumerate(DOMAIN_COLORS.items(), start=1):
        c = ws.cell(row=legend_row, column=offset+1, value=dom)
        c.fill = PatternFill("solid", fgColor=color); c.font = Font(name=FONT, size=9)
        c.alignment = Alignment(horizontal="center"); c.border = border

```

Лістинг В.2 – Реалізація еталонних методів (Wiener, USM, Blind RL)

В.3 Запуск бенчмарку та формування звітності

```

def write_excel(rows, out_path):
    from openpyxl import Workbook
    from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
    from openpyxl.utils import get_column_letter
    from openpyxl.formatting.rule import ColorScaleRule
    wb = Workbook(); ws = wb.active; ws.title = "Порівняння методів"
    FONT = "Arial"
    CLR_HEADER = "1F4E79"; CLR_SUBHDR = "2E75B6"

```

```

    CLR_ORIG = "D9E1F2"; CLR_ENGINE = "E2EFDA"; CLR_WIENER = "FFF2CC"; CLR_GUM = "FCE4D6";
    CLR_RL = "EAD1DC"
    CLR_WARN = "FF0000"
    thin = Side(style="thin", color="BFBFBF"); border = Border(left=thin, right=thin,
top=thin, bottom=thin)

    groups = [("Файл / домен",1,3,CLR_HEADER),("Оригінал",4,6,CLR_SUBHDR),
              ("Diploma Engine (адаптивний RL)",7,14,"375623"),("Wiener
(scipy)",15,19,"7F6000"),
              ("Gaussian Unsharp (OpenCV)",20,24,"843C0C"),("RL фіксований
(skimage)",25,29,"632523")]
    for title, c1, c2, color in groups:
        cell = ws.cell(row=1, column=c1, value=title)
        cell.font = Font(name=FONT, bold=True, color="FFFFFF", size=11)
        cell.fill = PatternFill("solid", fgColor=color)
        cell.alignment = Alignment(horizontal="center", vertical="center"); cell.border =
border
        if c2 > c1: ws.merge_cells(start_row=1, start_column=c1, end_row=1, end_column=c2)

    cols = [
        ("Файл",CLR_HEADER,"FFFFFF"),("Домен",CLR_HEADER,"FFFFFF"),("Роздільність",CLR_HEA
DER,"FFFFFF"),
        ("Tenengrad",CLR_ORIG,"1F4E79"),("Lapl.Var",CLR_ORIG,"1F4E79"),("Ringing",CLR_ORIG
,"1F4E79"),
        ("Tenengrad",CLR_ENGINE,"375623"),("Lapl.Var",CLR_ENGINE,"375623"),("Ringing",CLR_
ENGINE,"375623"),
        ("Приріст %",CLR_ENGINE,"375623"),("PSF модель",CLR_ENGINE,"375623"),("γ*
(оптим.)",CLR_ENGINE,"375623"),("RL ітер.",CLR_ENGINE,"375623"),("Час,
с",CLR_ENGINE,"375623"),
        ("Tenengrad",CLR_WIENER,"7F6000"),("Lapl.Var",CLR_WIENER,"7F6000"),("Ringing",CLR_
WIENER,"7F6000"),("Приріст %",CLR_WIENER,"7F6000"),("Час, с",CLR_WIENER,"7F6000"),
        ("Tenengrad",CLR_GUM,"843C0C"),("Lapl.Var",CLR_GUM,"843C0C"),("Ringing",CLR_GUM,"8
43C0C"),("Приріст %",CLR_GUM,"843C0C"),("Час, с",CLR_GUM,"843C0C"),
        ("Tenengrad",CLR_RL,"632523"),("Lapl.Var",CLR_RL,"632523"),("Ringing",CLR_RL,"6325
23"),("Приріст %",CLR_RL,"632523"),("Час, с",CLR_RL,"632523"),
    ]
    for ci, (title, bg, fc) in enumerate(cols, start=1):
        c = ws.cell(row=2, column=ci, value=title)
        c.font = Font(name=FONT, bold=True, color=fc, size=9)
        c.fill = PatternFill("solid", fgColor=bg)
        c.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True);
c.border = border

    keys = ["file","domain","resolution","tenengrad_orig","laplacian_orig","ringing_orig",
            "tenengrad_engine","laplacian_engine","ringing_engine","gain_pct_engine","psf_
model","gamma_opt","rl_iters","time_engine_s",
            "tenengrad_wiener","laplacian_wiener","ringing_wiener","gain_pct_wiener","time
_wiener_s",
            "tenengrad_gum","laplacian_gum","ringing_gum","gain_pct_gum","time_gum_s",
            "tenengrad_rl","laplacian_rl","ringing_rl","gain_pct_rl","time_rl_s"]
    pct_cols = {10, 19, 24, 29}; ring_cols = {6, 9, 17, 22, 27}
    for ri, row_data in enumerate(rows, start=3):
        for ci, key in enumerate(keys, start=1):
            val = row_data.get(key); c = ws.cell(row=ri, column=ci, value=val)
            c.font = Font(name=FONT, size=10)
            c.alignment = Alignment(horizontal="center" if ci > 3 else "left",
vertical="center"); c.border = border

```

```

        if isinstance(val, float): c.number_format = "0.0" if ci in pct_cols else
"0.00"
        if ci in ring_cols and isinstance(val, float) and val > RINGING_WARN:
            c.font = Font(name=FONT, size=10, bold=True, color=CLR_WARN)
        n_data = len(rows)
        if n_data > 0:
            for col_idx in pct_cols:
                col_letter = get_column_letter(col_idx)
                ws.conditional_formatting.add(f"{col_letter}3:{col_letter}{2+n_data}",
ColorScaleRule(
                    start_type="num", start_value=-50, start_color="F8696B",
                    mid_type="num", mid_value=0, mid_color="FFEB84",
                    end_type="num", end_value=200, end_color="63BE7B"))
            col_widths = [28, 14, 14] + [11]*3 + [11]*8 + [11]*5 + [11]*5 + [11]*5
            for i, w in enumerate(col_widths, start=1):
ws.column_dimensions[get_column_letter(i)].width = w
            ws.row_dimensions[1].height = 28; ws.row_dimensions[2].height = 40
            for r in range(3, 3+n_data): ws.row_dimensions[r].height = 20
            ws.freeze_panes = "D3"

        ws2 = wb.create_sheet("Зведена статистика")
        methods = [
            ("Оригінал (без обробки)", "tenengrad_orig", "ringing_orig", None, CLR_ORIG),
            ("Diploma Engine (адаптивний RL)", "tenengrad_engine", "ringing_engine",
"gain_pct_engine", CLR_ENGINE),
            ("Wiener filter (scipy)", "tenengrad_wiener", "ringing_wiener", "gain_pct_wiener",
CLR_WIENER),
            ("Gaussian Unsharp Mask (OpenCV)", "tenengrad_gum", "ringing_gum", "gain_pct_gum",
CLR_GUM),
            ("RL фіксований (skimage)", "tenengrad_rl", "ringing_rl", "gain_pct_rl", CLR_RL),
        ]
        summary_headers = ["Метод", "Tenengrad (середнє)", "Tenengrad (мін)", "Tenengrad
(макс)", "Приріст Tenengrad, % (сеп)", "Ringing Score (середнє)", "Ringing > 8 (кількість
зображень)"]
        for ci, h in enumerate(summary_headers, start=1):
            c = ws2.cell(row=1, column=ci, value=h)
            c.font = Font(name=FONT, bold=True, color="FFFFFF", size=10)
            c.fill = PatternFill("solid", fgColor=CLR_HEADER)
            c.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True);
c.border = border
            ws2.row_dimensions[1].height = 36
            _num = lambda lst: [x for x in lst if isinstance(x, (int, float))]
            for ri, (name, t_key, r_key, g_key, bg) in enumerate(methods, start=2):
                t_vals = _num([row.get(t_key) for row in rows]); r_vals = _num([row.get(r_key) for
row in rows])
                g_vals = _num([row.get(g_key) for row in rows]) if g_key else []
                data = [name, round(np.mean(t_vals),3) if t_vals else "N/A", round(min(t_vals),3)
if t_vals else "N/A",
                    round(max(t_vals),3) if t_vals else "N/A", round(np.mean(g_vals),1) if
g_vals else "-",
                    round(np.mean(r_vals),2) if r_vals else "N/A", sum(1 for v in r_vals if v
> RINGING_WARN)]
                for ci, val in enumerate(data, start=1):
                    c = ws2.cell(row=ri, column=ci, value=val)
                    c.font = Font(name=FONT, size=10, bold=ci==1 or (ci==7 and isinstance(val,int)
and val>0),

```

```

        color=CLR_WARN if ci==7 and isinstance(val,int) and val>0 else
"000000")
        c.fill = PatternFill("solid", fgColor=bg)
        c.alignment = Alignment(horizontal="center" if ci>1 else "left",
vertical="center"); c.border = border
        ws2.column_dimensions["A"].width = 36
        for col in "BCDEFG": ws2.column_dimensions[col].width = 22

        ws3 = wb.create_sheet("Легенда")
        notes = [
            ("Метрика","Опис"),
            ("Tenengrad","Середній квадрат градієнта Собеля. Вища = різкіше. Формула (2.9)."),
            ("Laplacian Variance","Дисперсія лапласіану. Вища = більше дрібних деталей.
Формула (2.10)."),
            ("Ringing Score","Екссес розподілу лапласіану. > 8 = видимі кільцеві артефакти.
Формула (2.11)."),
            ("Приріст Tenengrad %","(після - до) / до × 100. Показує відносне покращення
різкості."),
            ("",""),
            ("Метод","Параметри"),
            ("Diploma Engine","Адаптивний RL: авто-PSF (капра-дерево), γ* методом золотого
перерізу, зупинка за Ringing Score."),
            ("Wiener (scipy)","scipy.signal.wiener(img, mysize=5). Фіксований квадратний
фільтр 5×5."),
            ("Gaussian Unsharp (OpenCV)","img + 1.5 × (img - GaussianBlur(img, σ=1.0)).
Фіксовані параметри."),
            ("RL фіксований (skimage)","skimage.restoration.richardson_lucy, гаусовий PSF
σ=1.5, 20 ітерацій."),
            ("",""),
            ("Колір ringing","Червоний шрифт у колонці Ringing = значення > 8.0 (поріг
артефактів)."),
        ]
        for ri, (k, v) in enumerate(notes, start=1):
            ws3.cell(row=ri, column=1, value=k).font = Font(name=FONT, bold=True, size=10)
            ws3.cell(row=ri, column=2, value=v).font = Font(name=FONT, size=10)
        ws3.column_dimensions["A"].width = 26; ws3.column_dimensions["B"].width = 80
        _write_known_results_sheet(wb, border)
        wb.save(out_path); print(f"\n ✓ Excel збережено: {out_path}")

def write_chart(rows, out_path):
    try:
        import matplotlib; matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError: print(" [!] matplotlib не встановлено - графік пропущено.");
    return

    labels = [r["file"][:22] for r in rows]; x = np.arange(len(labels)); width = 0.20
    g = {k: [r.get(k) or 0 for r in rows] for k in
["gain_pct_engine","gain_pct_wiener","gain_pct_gum","gain_pct_rl"]}
    rg = {k: [r.get(k) or 0 for r in rows] for k in
["ringing_engine","ringing_wiener","ringing_gum","ringing_rl"]}
    colors = ["#4CAF50","#FFC107","#FF7043","#AB47BC"]
    labels_m = ["Diploma Engine (адаптивний RL)","Wiener (scipy)","Gaussian Unsharp
(OpenCV)","RL фіксований (skimage)"]

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(max(12, len(labels)*1.2), 11),
facecolor="#F8F8F8")

```

```

fig.suptitle("Порівняння методів відновлення зображень", fontsize=14,
fontweight="bold", y=0.98)
gk = list(g.values())
for i, (gv, color, lbl) in enumerate(zip(gk, colors, labels_m)):
    bars = ax1.bar(x + (i-1.5)*width, gv, width, label=lbl, color=color, alpha=0.9,
edgecolor="white", linewidth=0.5)
    if i == 0:
        for bar in bars:
            h = bar.get_height()
            if abs(h) > 1: ax1.text(bar.get_x()+bar.get_width()/2, h+1, f"{h:.0f}%",
ha="center", va="bottom", fontsize=7, color="#2E7D32")
ax1.axhline(0, color="gray", linewidth=0.8, linestyle="--")
ax1.set_ylabel("Приріст Tenengrad, %", fontsize=11)
ax1.set_title("Покращення різкості (Tenengrad) відносно оригіналу", fontsize=11)
ax1.set_xticks(x); ax1.set_xticklabels(labels, rotation=30, ha="right", fontsize=9)
ax1.legend(fontsize=9, loc="upper left"); ax1.grid(axis="y", alpha=0.3);
ax1.set_facecolor("#FAFAFA")

for i, (rv, color) in enumerate(zip(rg.values(), colors)):
    ax2.bar(x + (i-1.5)*width, rv, width, color=color, alpha=0.9, edgecolor="white")
ax2.axhline(RINGING_WARN, color="red", linewidth=1.2, linestyle="--", label=f"Попір
артефактів ({RINGING_WARN})")
ax2.set_ylabel("Ringing Score", fontsize=11)
ax2.set_title("Ringing Score після обробки (нижче = менше артефактів)", fontsize=11)
ax2.set_xticks(x); ax2.set_xticklabels(labels, rotation=30, ha="right", fontsize=9)
ax2.legend(fontsize=9, loc="upper right"); ax2.grid(axis="y", alpha=0.3);
ax2.set_facecolor("#FAFAFA")

plt.tight_layout(rect=[0,0,1,0.97])
plt.savefig(out_path, dpi=150, bbox_inches="tight"); plt.close()
print(f" ✓ Графік збережено: {out_path}")

def main():
    print("=" * 60 + "\n Benchmark: diploma_engine vs стандартні методи\n" + "=" * 60)
    if not TEST_DIR.exists():
        print(f"\n[!] Папка з тестовими зображеннями не знайдена:\n {TEST_DIR}");
sys.exit(1)
    extensions = {".png", ".jpg", ".jpeg", ".tif", ".tiff", ".bmp", ".fits"}
    image_files = sorted(p for p in TEST_DIR.iterdir() if p.suffix.lower() in extensions)
    if not image_files: print(f"\n[!] У папці немає підтримуваних зображень: {TEST_DIR}");
sys.exit(1)
    print(f"\n Знайдено зображень: {len(image_files)}")
    print(f" diploma_engine: {'доступний ✓' if ENGINE_AVAILABLE else 'НЕ знайдено
X'}\n")
    rows = [r for path in image_files if (r := process_image(path))]
    if not rows: print("\n[!] Жодне зображення не оброблено успішно."); sys.exit(1)
    print(f"\n{'='*60}\n Оброблено: {len(rows)} зображень. Формування
звіту...\n{'='*60}")
    write_excel(rows, OUT_XLSX); write_chart(rows, OUT_CHART)
    print(f"\n Готово! Файли:\n {OUT_XLSX}\n {OUT_CHART}")

if __name__ == "__main__":
    main()

```

Лістинг В.3 – Запуск бенчмарку, Excel-таблиця та PNG-графіки

ДОДАТОК Г

Лістинг побудови одновимірних профілів моделей PSF

```

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from scipy.special import j1
N, r_max, EPS = 500, 12.0, 1e-12
r = np.linspace(0, r_max, N)

def gaussian(r, sigma=1.5): return np.exp(-r**2 / (2*sigma**2))
def cauchy(r, gamma=1.5): return (1 + r**2/gamma**2)**(-1.5)
def airy(r, R=1.5):
    with np.errstate(invalid='ignore', divide='ignore'):
        arg = np.pi * r / R
        return np.where(r == 0, 1.0, (2*j1(arg)/arg)**2)
def moffat(r, alpha=1.5, beta=3.5): return (1 + r**2/alpha**2)**(-beta)
def motion_1d(r, L=3.0): return np.where(r <= L/2, 1.0, EPS)
def scattering(r, k=0.8): return np.exp(-k * r)

fig, ax = plt.subplots(figsize=(10, 6))
fig.patch.set_facecolor('white')
ax.set_facecolor('white')

styles = [
    ('Гаусова (Gaussian)',      gaussian(r),   '#1f77b4', '-', 2.0),
    ('Коші (Cauchy)',           cauchy(r),      '#d62728', '--', 2.0),
    ('Ейрі (Airy disk)',        airy(r),      '#2ca02c', '-.', 2.0),
    ('Моффат (Moffat  $\beta=3.5$ ), moffat(r),   '#ff7f0e', ':', 2.2),
    ('Розсіювання (Scatt.)',    scattering(r), '#9467bd', '--', 1.8),
    ('Розмиття руху (Motion)',  motion_1d(r), '#8c564b', '-', 1.8),

```



```

]

for label, vals, color, ls, lw in styles:
    ax.semilogy(r, np.maximum(vals / vals[0], EPS), label=label, color=color,
linestyle=ls, linewidth=lw)

ax.set_xlim(0, r_max)
ax.set_ylim(1e-4, 2.0)
ax.set_xlabel('Радіальна відстань від центру, пікселів', fontsize=13)
ax.set_ylabel('Відносна інтенсивність (нормована, лог. шкала)', fontsize=13)
ax.set_title('Одновимірні перерізи моделей PSF\n(нормовано до одиничного максимуму)',
fontsize=14)
ax.yaxis.set_major_formatter(ticker.LogFormatterMathtext())
ax.grid(True, which='major', linestyle='-', linewidth=0.5, alpha=0.6, color='#cccccc')
ax.grid(True, which='minor', linestyle='--', linewidth=0.3, alpha=0.3, color='#dddddd')
ax.axhline(0.5, color='gray', linestyle=':', linewidth=1.0, alpha=0.7)
ax.text(r_max*0.98, 0.52, 'HWHM = 0.5', ha='right', fontsize=10, color='gray')
ax.legend(fontsize=11, framealpha=0.9, edgecolor='#cccccc', loc='upper right')
ax.annotate('Важкі хвости\n(Кові, Моффат)',
            xy=(8.5, cauchy(np.array([8.5]))[0] / cauchy(np.array([0.0]))[0]),
            xytext=(6.0, 2e-3),
            arrowprops=dict(arrowstyle='->', color='#d62728', lw=1.2),
            fontsize=10, color='#d62728')
ax.annotate('', xy=(8.5, moffat(np.array([8.5]))[0] / moffat(np.array([0.0]))[0]),
            xytext=(6.2, 2.2e-3),
            arrowprops=dict(arrowstyle='->', color='#ff7f0e', lw=1.2))

plt.tight_layout()
plt.savefig('psf_1d_profiles.png', dpi=200, bbox_inches='tight', facecolor='white',
edgecolor='none')
plt.close()
print("Saved: psf_1d_profiles.png")

```

Лістинг Г.1 – Побудова одновимірних перерізів моделей PSF у логарифмічному масштабі

ДОДАТОК Д

Лістинг побудови двовимірних теплових карт моделей PSF

```

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm
from scipy.special import j1
from scipy.ndimage import gaussian_filter

SIZE = 101
ax_1d = np.arange(-SIZE//2, SIZE//2 + 1, dtype=np.float64)
XX, YY = np.meshgrid(ax_1d, ax_1d)
R2 = XX**2 + YY**2
R = np.sqrt(R2)

def make_gaussian(sigma=2.5): k = np.exp(-R2/(2*sigma**2)); return k/k.max()
def make_cauchy(gamma=2.5): k = (1+R2/gamma**2)**(-1.5); return k/k.max()
def make_airy(radius=3.0):
    with np.errstate(invalid='ignore', divide='ignore'):
        arg = np.pi*R/radius
        k = np.where(R==0, 1.0, (2*j1(arg)/arg)**2)
    return k/k.max()
def make_moffat(alpha=2.5, beta=3.5): k = (1+R2/alpha**2)**(-beta); return k/k.max()
def make_motion(length=8.0, angle_deg=30.0):
    kernel = np.zeros_like(R); cx = SIZE//2
    cos_a, sin_a, half = np.cos(np.deg2rad(angle_deg)), np.sin(np.deg2rad(angle_deg)),
length/2.0
    for t in np.linspace(-half, half, max(int(6*length), 20)):
        xi, yi = int(round(cx+t*cos_a)), int(round(cx+t*sin_a))
        if 0 <= xi < SIZE and 0 <= yi < SIZE: kernel[yi, xi] = 1.0
    kernel = gaussian_filter(kernel, sigma=0.6)

```

```

    return kernel/kernel.max()
def make_scattering(k_val=0.18): kv = np.exp(-k_val*R); return kv/kv.max()

panels = [
    ('Гаусова\n(Gaussian,  $\sigma=2.5$  пкс)',      make_gaussian()),
    ('Коші\n(Cauchy,  $\gamma=2.5$  пкс)',          make_cauchy()),
    ('Ейрі\n(Airy disk,  $R=3.0$  пкс)',          make_airy()),
    ('Моффат\n(Moffat,  $\alpha=2.5$ ,  $\beta=3.5$ )',    make_moffat()),
    ('Розсіювання\n(Scattering,  $k=0.18$ )',      make_scattering()),
    ('Розмиття руху\n(Motion,  $L=8$ ,  $\theta=30^\circ$ '), make_motion()),
]

crop, c, VMIN = 30, SIZE//2, 1e-4
sl = slice(c-crop, c+crop+1)

fig, axes = plt.subplots(2, 3, figsize=(13, 9))
fig.patch.set_facecolor('white')
fig.suptitle('Двовимірні ядра функцій розсіювання точки (PSF)\n'
            '(логіфімічна кольорова шкала, нормовано до максимуму)', fontsize=14,
            y=1.01)

for ax, (title, kernel) in zip(axes.flat, panels):
    im = ax.imshow(np.maximum(kernel[sl, sl], VMIN), norm=LogNorm(vmin=VMIN, vmax=1.0),
                  cmap='inferno', origin='lower', extent=[-crop,crop,-crop,crop],
                  interpolation='bilinear')
    levels = [0.01, 0.10, 0.50]
    cs = ax.contour(np.maximum(kernel[sl, sl], VMIN), levels=levels,
                  colors=['#aaaaaa', '#ddddd', 'white'], linewidths=[0.7,0.8,1.0],
                  extent=[-crop,crop,-crop,crop], origin='lower')
    ax.clabel(cs, fmt={0.01:'1%', 0.10:'10%', 0.50:'HWHM 50%'}, fontsize=8, inline=True)
    cbar = fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    cbar.ax.tick_params(labelsize=8)
    cbar.set_label('Відн. інтенс.', fontsize=8)
    ax.set_title(title, fontsize=11, pad=6)
    ax.set_xlabel('Δx, пікселів', fontsize=9)

```

```
ax.set_ylabel('Δy, пікселів', fontsize=9)
ax.tick_params(labelsize=8)
ax.axhline(0, color='white', linewidth=0.4, alpha=0.4)
ax.axvline(0, color='white', linewidth=0.4, alpha=0.4)

plt.tight_layout(rect=[0,0,1,0.99])
plt.savefig('psf_2d_heatmaps.png', dpi=200, bbox_inches='tight', facecolor='white',
edgecolor='none')
plt.close()
print("Saved: psf_2d_heatmaps.png")
```

Лістинг Д.1 – Двовимірна візуалізація ядер PSF у логарифмічній кольоровій шкалі